NG PULSE

Rail-Signature Workload Integrity Monitor

EEL 4914 Senior Design I

Midterm Milestone Report

Group Number: G12

Group Members: Joey Esposito, Sean Kenney, Jacob Daleandro, Dylan Ruiz

Degree Majors: 2EE, 2CpE

Sponsor/Funding: Northrop Grumman

Advisor/Reviewer(s): Dr. Mike Borowczak, Jared S. Burrows, Prof. Amir Jazayeri

Date: [MM/DD/2026]

Table of Contents

List of Tables

List of Figures

Acknowledgements

[TO DO: Optional. Acknowledge Northrop Grumman, advisors/reviewers, the DRACO laboratory and anyone else who contributed. Delete this page if not used.]

Chapter 1 – Executive Summary

[TO DO: Target 1–2 pages. Page count starts here. Not required for the midterm report; text below is carried over from the Divide and Conquer and should be updated for the final SD1 report.]

NG PULSE is a hardware-security monitoring project intended to detect physical attacks against a protected processor by observing its supply-rail current behavior. The proposed monitor will attach to any printed circuit board (PCB), acquire current signatures from the host processor, compare measured behavior against a known-good reference or golden profile, and generate an alert when an abnormal condition is detected. An advanced implementation may also initiate a protective system shutdown after an attack is identified.

The planned architecture combines a current-sense front end, a 12-bit analog-to-digital converter (ADC), windowed feature extraction, and threshold or classification logic. The project will first establish reference current waveforms for selected RISC-V processor functions or workloads. The detection logic will then be implemented in Verilog and verified before physical prototyping. The prototype is planned as a PCB containing the current-sense design and MCU for processing, to monitor a ESP32-C (board with a RISC-V processor) during normal function.

The project is funded by Northrop Grumman with a maximum budget of $1,000. Key performance targets include 100% coverage of the reference waveforms defined by the project test plan, no more than a 5% reduction in RISC-V computing efficiency, no more than 1 second of attack-detection latency, at least a 95% attack-detection rate, no more than a 5% false-positive rate, 12-bit ADC resolution, and operation from either a 3.3 V or 5 V input supply.

Chapter 2 – Project Description

[TO DO: Target ~10 pages. Required for the midterm report.]

2.1 Motivation and Background

Modern computing systems depend on integrated circuits that may be exposed not only to software-based threats but also to physical attacks that interfere directly with hardware operation. NG PULSE is motivated by the need for an additional layer of hardware-level security that can identify abnormal electrical behavior in a protected processor. A representative threat identified for this project is abnormal or high-current injection intended to disturb normal chip operation.

The central concept is to monitor the processor's supply-rail current signature. Under normal operation, processor activity produces characteristic electrical behavior. By establishing a golden representation of that behavior and continuously comparing later measurements against it, NG PULSE is intended to identify departures that may indicate an attack. Because the monitor does not

need to sit in the processor's normal control path, the design goal is to provide security monitoring while limiting the impact on host computing performance.

The project also provides a mixed-signal design flow spanning analog sensing, analog-to-digital conversion, digital signal processing, classification or threshold logic, Verilog/RTL development, CAD-based schematic integration, and eventual physical prototyping. The final design is intended to be suitable for attachment to an existing PCB rather than requiring a complete redesign of the protected system.

2.2 Goals and Objectives

2.2.1 Goals

Basic Goals:

- Develop a functional hardware-security monitor capable of detecting abnormal supply-rail current behavior from an ESP32-C target and asserting an error flag when an abnormal condition is detected.
- Monitor one predefined ESP32-C work mode by comparing measured supply-rail current behavior against an established golden profile representing normal operation.
- Monitor the ESP32-C while in a Not-in-Use state to detect current behavior indicative of unauthorized processor activity.
- Implement NG PULSE as a separate monitoring system that can interface with an external target without requiring the monitoring hardware to be integrated into the target PCB.
- Demonstrate detection of abnormal supply-rail current behavior resulting from a defined physical/electrical attack on the ESP32-C.
- Produce digital design of ASIC in Cadence to contain comparison logic and error flag generation

Advanced Goals:

- Expand NG PULSE from monitoring one predefined work mode to monitoring four predefined ESP32-C work modes in addition to the Not-in-Use state.
- Automatically apply the appropriate golden profile according to the operating mode reported by the ESP32-C.
- Maintain continuous monitoring as the ESP32-C transitions between its predefined operating modes and the Not-in-Use state.

Stretch Goals:

- Integrate the current-sensing hardware, Nordic MCU, and FPGA components onto a single custom NG PULSE PCB while maintaining the ESP32-C as an externally connected target.

- Incorporate Bluetooth connectivity to provide system status and detection information to a mobile device.
- Demonstrate a protective response in which the NG PULSE error flag can initiate an automatic shutdown of the ESP32-C following detection of abnormal current behavior.
- Include ADC functionality in digitally designed ASIC
- Include memory storage in digitally designed ASIC
- Include current capture in digitally designed ASIC

2.2.2 Objectives

Basic Objectives:

- Design and implement a shunt-based current-sensing front end on a custom NG PULSE PCB to convert ESP32-C supply-rail current behavior into an analog voltage signal measurable by the Nordic MCU's integrated ADC.
- Connect the Nordic MCU, FPGA, and ESP32-C target to the NG PULSE current-sensing PCB through external jumper connections.
- Characterize one predefined authorized ESP32-C work mode by collecting 1,024 current measurements using the Nordic MCU's integrated ADC.
- Use the 1,024 characterization measurements to calculate an average current value and acceptable deviation representing the golden profile for the predefined work mode, and store the resulting profile on the Nordic MCU.
- Establish communication from the ESP32-C to the Nordic MCU so that the target can report whether it is operating in the predefined work mode or the Not-in-Use state.
- Configure the Nordic MCU to acquire current measurements and provide the FPGA with the measured ADC values and stored golden-profile parameters required for detection.
- Develop Verilog-based FPGA logic that evaluates successive current measurements against the acceptable range established by the golden profile and asserts an error flag when abnormal current behavior satisfies the defined detection criteria.
- Use developed RTL code to produce a logic gate schematic in Cadence
- Continue monitoring supply-rail current while the ESP32-C reports the Not-in-Use state and assert an error flag when current behavior indicates unauthorized processor activity.
- Simulate and verify the Verilog-based comparison, detection, and error-flag logic prior to physical FPGA implementation.

- Physically demonstrate that NG PULSE can detect abnormal ESP32-C supply-rail current behavior resulting from an induced voltage-surge attack and successfully assert an error flag.

Advanced Objectives:

- Characterize four predefined ESP32-C work modes using 1,024 current measurements per mode and establish a separate golden average and acceptable deviation for each mode.
- Store the golden-profile parameters for all four predefined work modes on the Nordic MCU.
- Expand ESP32-C-to-Nordic communication to identify the Not-in-Use state and each of the four predefined work modes.
- Configure the Nordic MCU to automatically select the appropriate stored golden profile according to the operating mode reported by the ESP32-C.
- Provide the FPGA with the applicable golden-profile parameters and current ADC measurements as the ESP32-C transitions between predefined operating modes.
- Expand FPGA-based detection to evaluate abnormal current behavior across all four predefined work modes while maintaining monitoring of the Not-in-Use state.

Stretch Objectives:

- Develop a custom NG PULSE PCB incorporating the current-sensing circuitry, Nordic MCU, and FPGA components on a single board, eliminating the need for a separate FPGA board while maintaining an external connection to the ESP32-C target.
- Utilize the Nordic MCU's Bluetooth capability to communicate the ESP32-C's reported operating mode and NG PULSE's normal, abnormal, or error status to a mobile device.
- Implement and demonstrate an automatic ESP32-C shutdown response initiated by the FPGA-generated error flag as an example of a protective action that can be taken following detection.
- Design a 4-bit ADC digitally
- Expand ADC resolution to 12 bits and integrate into ASIC
- Design memory storage, access, and data transfer system for ASIC
- Design voltage amplification and straight through power rail for current sense in ASIC

2.3 System Features and Functional Description

NG PULSE will observe the supply-rail current of the protected processor and convert the sensed analog behavior into digital data. The digital subsystem will

process the measurements over defined time windows and compare extracted information against the golden reference profile. When the measured behavior satisfies the project's criteria for an anomalous or attack condition, an attack flag will be asserted.

- Supply-rail current sensing.
- 12-bit analog-to-digital conversion.
- Acquisition and storage/use of golden reference waveforms.
- Windowed feature extraction or equivalent waveform-processing method.
- Threshold-based or classifier-based anomaly decision logic.
- Attack flag generation within the specified latency.
- Alert/status reporting; PMBus or I²C is identified in the project concept, with the final interface to be confirmed.
- Optional system shutdown response for the stretch implementation.
- Operation from either a 3.3 V input supply.

[TO DO: Guideline: reference any input from customers (Northrop Grumman) or marketing analysis of comparable products/projects used to identify these features.]

2.4 Existing Products and Prior Related Work

The NG PULSE project overlaps with several established areas of technology that exist in the market today. These include current-sense amplifiers, ADC-based power monitors, threshold and comparator protection circuits, processor power-signature analysis, and physical fault-injection security research. Although each of these areas provides technologies relevant to the proposed system, the information provided by Northrop Grumman does not establish a specific commercial product that directly combines these capabilities. The following products were therefore selected as comparable due to their demonstration of individual functions that overlap with the NG PULSE architecture. The differences that separate the NG PULSE from these systems are also discussed.

2.4.1 Texas Instruments INA233

The Texas Instruments INA233 is a ten-pin Very Thin Shrink Small Outline Package (VSSOP) digital current-sense monitor. This monitor is designed to measure and report electrical signals from a monitored power rail. The INA233 takes measurements of the current, voltage, and power. It then reports these values to enable accurate decision making in controlled systems. It has an integrated analog-to-digital converter (ADC) with programmable conversion times and averages modes. This device also has programmable limits for voltage, current, and power that has the ability to send alert notifications when the thresholds are exceeded. This combination of features allows for continuous monitoring with a notification system for abnormal behaviors [DATASHEET].

The INA233 shares several characteristics with the NG PULSE. The integration of current sensing, the analog-to-digital conversion, digital processing, threshold detection, and notification system. This device contains a shunt resistor that is used for the current sensing. Measuring the voltage drop through the shunt resistor allows for a measurement of the current that is being sent through the component, with no significant impedance affecting it. The NG PULSE will also use a shunt resistor for measurements to be taken for current sensing. The INA233 also uses an analog-to-digital conversion. This conversion is done by taking samples of continuous analog signals and converting them into digital values. These digital values are then able to be used for digital processing, which uses computational methods to analyze signals that are received. The threshold detection incorporated into the INA233 is another feature that is similar to the NG PULSE. This feature is used to identify anomalous behavior, specifically when a measured value crosses a predefined limit. By continuously comparing the measured data against the golden profile, the threshold detection will identify when the measured values are too different from the established normal behavior. If abnormal behavior is identified, the INA233 has an integrated notification system that has the ability to send alerts. With this feature, the NG PULSE will be able to promptly send an alert when the threshold is crossed, and further action needs to be taken. The last similarity these two share is that they are both designed to be placed directly onto a programmable circuit board. This allows the components to monitor a power rail so that any abnormalities are caught quickly and negative effects can be mitigated.

The main difference between the Texas Instruments INA233 and the NG PULSE comes from the recognition of what is defined as abnormal behavior. Mentioned previously, the INA233 has programmable limits to act as the threshold. This threshold is also set as a steady value that will act as an upper or lower limit, or both. Meanwhile the NG PULSE will analyze the current during processor activity to identify abnormal behavior. The NG PULSE will not have a steady value acting as a threshold and will instead compare the values being measured to the golden profile. This allows for continuous monitoring against the golden profile and will not limit it to a maximum or minimum value.

2.4.2 WattsUpDoc

The WattsUpDoc device is a power-monitoring system designed to detect malware on embedded devices by analyzing the device's power consumption. The system uses a side-channel analysis to identify abnormal behavior without requiring any modifications to the hardware or software of the device that is being monitored. This device operates externally by measuring the overall power being consumed by the device through an AC power outlet by using a shunt resistor to measure the voltage drop, which shows the current being drawn through the device. These voltage measurements are then sampled and saved for analysis. The system uses machine-learning algorithms, multilayer perceptron, and random classifiers to analyze the collected power measurements and distinguish between normal and abnormal behavior. This method allows the WattsUpDoc to detect malware that exists on the device by identifying higher than normal

workloads. Researchers have shown that the detection accuracies of WattsUpDoc is approximately 94% on a pharmaceutical compounder and 99% on a substation computer when previously known malware is identified [WATTSUPDOC].

WattsUpDoc shares several characteristics with the NG PULSE. Both of these systems will use measurements taken to identify abnormal behavior in the devices. The WattsUpDoc uses a $0.1\ \Omega$ resistor in series with the outlet to take the measurement of the voltage drop to determine how much current is being drawn through the monitored device. Similarly, the NG PULSE will use a shunt resistor to measure the current through the power rail of a processor. The WattsUpDoc uses a data acquisition unit (DAQ) to sample the voltage measurements and convert them from analog to digital to be analyzed. This DAQ contains an ADC but also contains more parts alongside the ADC. Both products will analyze the collected measurements to identify any abnormal behavior. Certain characteristics that are evaluated, such as mean and variance, will be measured and evaluated by both products. The way the measurements are analyzed are different – which will be brought up later. These evaluations allow both systems to identify abnormal behavior that could indicate malicious activity. Most importantly, both of these systems are designed to monitor a system without interfering while normal operations are carried out.

The main difference between the WattsUpDoc and NG PULSE comes from how the monitoring occurs and what is being monitored. As mentioned previously, the WattsUpDoc is an external monitoring device that utilizes side-channel analysis to identify abnormal activity. This is done by monitoring the entirety of the power consumption on the device being monitored. While effective, this means that the actual activity of the individual processor or power rail cannot be monitored exclusively because the WattsUpDoc cannot isolate those signals. Since the NG PULSE is designed to sit directly on the power rail from the processor, the specific activity from the processor being monitored is isolated from the overall power consumption. There are also differences in how these two analyze behaviors to identify any malicious activity. The NG PULSE uses a golden profile to represent expected activity depending on the processor's activity. The measurements that are taken are then compared to that profile to determine whether the currently measured activity is deemed to be regular or abnormal. The WattsUpDoc devices uses machine-learning classifiers that evaluate certain metrics – such as the mean, maximum, and variance. Once the classifiers are trained, they are then used to determine if the power being measured represents normal or abnormal behavior. The final difference between these two is the ADC. The WattsUpDoc uses a DAQ, which has an integrated ADC alongside other components like sensors and filters. The NG PULSE does not use a DAQ, instead implementing an ADC into the design of the ASIC.

[TO DO: Consider adding prior related work: the previous UCF/Northrop Grumman senior design project NGRAD and any power side-channel or fault-injection research papers.]

2.5 Engineering Specifications

The following specifications define the current quantitative targets for the NG PULSE system. The final test plan should define the exact workloads, attack cases, measurement equipment, and pass/fail procedures used to demonstrate each requirement.

Table 2.1: Engineering Specifications

Parameter	Requirement	Verification Method
Reference waveform coverage	# of target processes: 4 Sample requirement of each process: 1024 samples averaged Output: golden profile	Golden-profile acquisition records
RISC-V Computing Efficiency	0% loss (zero tolerance with NG PULSE operation)	Benchmark with/without monitor
Attack detection latency	Maximum 1 s from detectable attack/anomaly to asserted flag	Timed attack/flag measurement
Attack detection rate	Minimum 95%	Repeated defined attack trials
False-positive detection rate	Maximum 5%	Repeated normal-workload trials
F1 score	Minimum 0.85	Repeated and defined attack trials and normal operation monitoring
Current sensing range	10mA to 1.65A	Thermal imaging camera under high load and minimum current detection within specified accuracy.
Signal to noise ratio	Minimum 20dB	Golden profile comparison to actual waveform
Long time overcurrent shutoff	< 20s for 1.05 - 1.3 pu surge	Repeated and defined attack trials
Short time overcurrent shutoff	< 5s for 1.3 - 2.0 pu surge	Repeated and defined attack trials
Instantaneous overcurrent shutoff	< 0.1s for >= 2.0 pu surge	Repeated and defined attack trials
Weight	< 5 lbs	Digital scale measurement of complete system
Size	< 1 ft^2	Physical dimension and footprint measurement
Cost	< $1000 (Shipping not included)	BOM and purchase receipt cost calculation

[TO DO: Guideline: highlight 3 demonstrable specifications (e.g., shade those rows), and consider splitting into two tables — overall system specs and component specs.]

2.6 Hardware Block Diagram

At a high level, the protected processor is powered through a rail that is observed by the NG PULSE sensing circuitry. The current-sense front end produces a measurable analog representation of the rail current. A 12-bit ADC digitizes this signal. Digital processing then extracts waveform information and compares the measured behavior against the golden profile. The decision engine asserts an attack flag when the anomaly criteria are satisfied. The flag may be reported externally and, in the advanced implementation, may initiate a shutdown response.

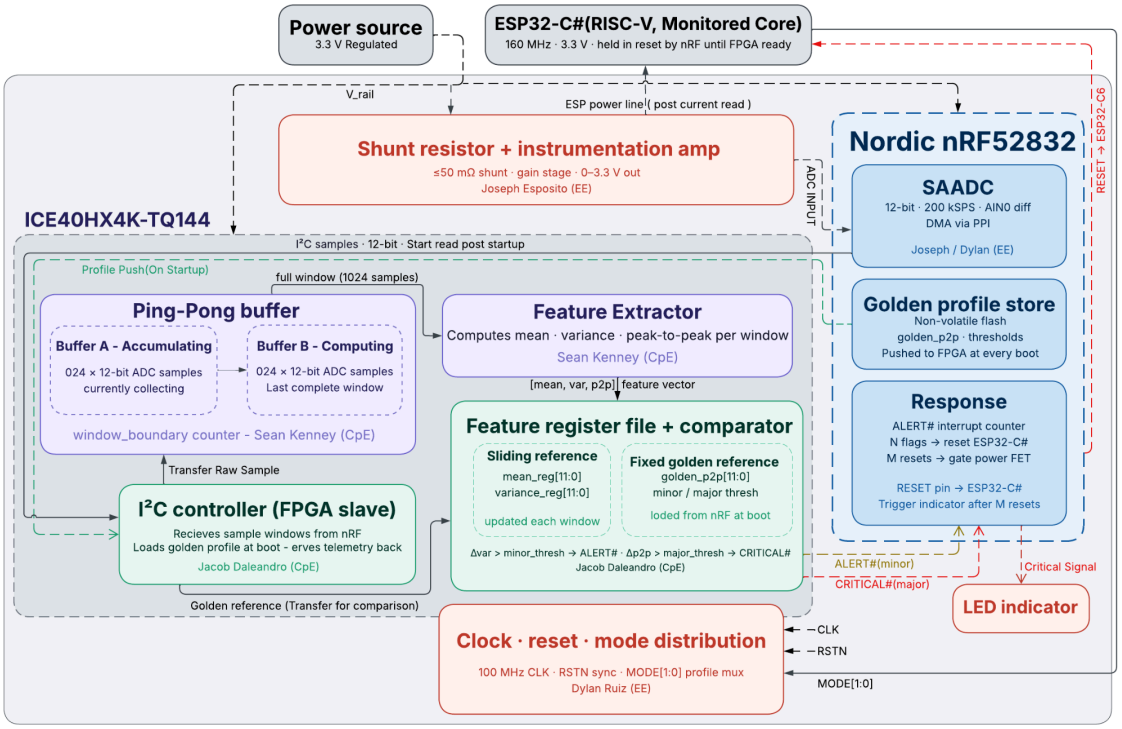


Figure 2.1: Hardware Block Diagram

[TO DO: Guideline: the block diagram must show the status of each block (to be acquired, acquired, investigating, designing, prototyping, completed), a legend, and distinct line styles for wired, wireless and power signals.]

2.6.1 Preliminary Component Categories

- Shunt resistor
- Op-amp
- MCU: Nordic nrf52832

- FPGA: ICE40HX4k-TQ144
- Target RISC-V processor/platform: ESP32-C
- Communication/interface components:TBD
- Protection/shutdown circuitry: MOSFET

2.7 Software Diagram

The NG PULSE digital subsystem will be developed in Verilog. The intended processing flow begins with digitized current samples from the ADC. Samples will be organized into appropriate analysis windows, processed to obtain the features or values required by the detector, and evaluated against the golden profile or configured decision thresholds. When the decision logic identifies an attack condition, it will assert the attack flag and provide the required reporting or protection response.

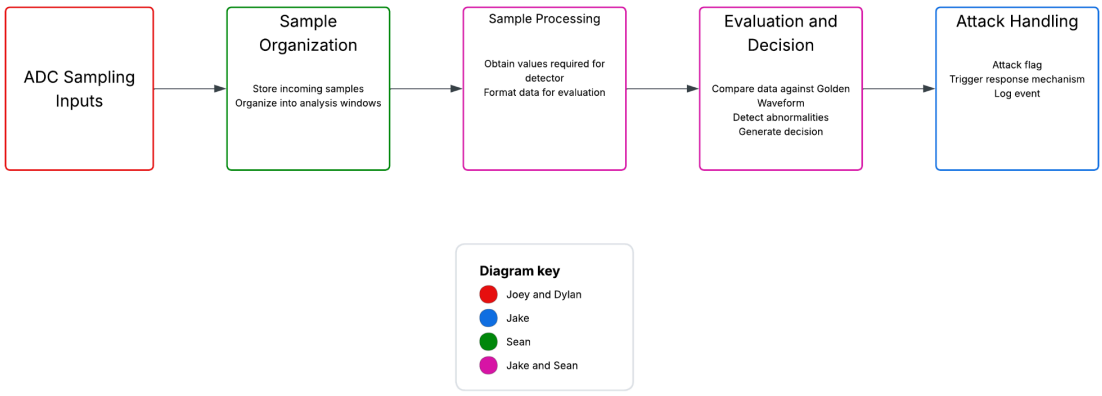


Figure 2.2: Software Flowchart

[TO DO: Guideline: indicate the input and output of each block.]

2.8 Prototype Illustration

The initial physical detector will be implemented on a PCB using commercial off-the-shelf components. The PCB will contain the sensing, controls, and clock signals, as well as pin headers for the ESP32-C, FPGA, and Nordic board, and supporting power circuitry required to evaluate the NG PULSE concept.

The preferred prototype will connect the detector to an FPGA available in the Senior Design laboratory so that the Verilog logic can be implemented, tested, and modified rapidly. The team will connect the prototype to a RISC-V processor board (ESP32-C) in the DRACO laboratory. This would allow the monitor to acquire and evaluate current signatures from an actual RISC-V target.

Prototype testing will verify current acquisition, golden-profile comparison, attack flag generation, detection latency, detection rate, false-positive rate, and all other specifications listed in table 2.1.

[TO DO: Guideline: add a prototype illustration/blueprint figure (e.g., a rendering or sketch of the two-board assembly and the ESP32-C target).]

2.9 House of Quality

A formal House of Quality will be developed to connect sponsor/customer needs to measurable engineering characteristics. Based on the current project definition, likely customer needs include reliable attack detection, low false alarms, rapid response, minimal performance impact, compatibility with existing hardware, and practical prototyping cost.

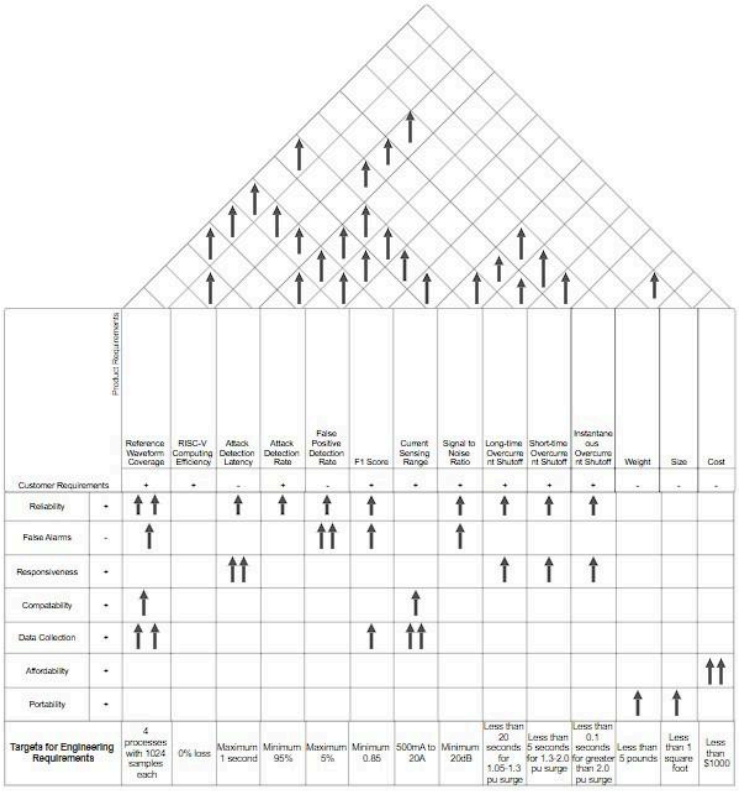


Figure 2.3: House of Quality

Chapter 3 – Research and Investigation

[TO DO: Target 45–60 pages. Required for the midterm report. Order: hardware, then communication protocol, then software. In hardware, the power supply goes last. For each item, discuss technology comparison and selection first, followed immediately by part comparison and selection (at least 3 options, one table each). Every comparison needs: critical parameters and their impact, a quantitative summary table, and a justification for the selection.]

3.1 Hardware Technology and Part Selection

3.1.1 Current Sensing

NG PULSE monitors changes in the electrical current drawn by an ESP32-based target while it executes defined workloads. The sensing method must capture both the steady-state current and time-varying features associated with workload transitions without materially disturbing the target supply rail. Three approaches were considered: resistive shunt sensing, Hall-effect sensing, and current-transformer sensing. Their operating principles and limitations are discussed below. This discussion evaluates technology categories; final part numbers, current ranges, bandwidth, and acceptable insertion losses remain subject to measurement and verification.

3.1.1.1 Resistive Shunt Current Sensing

Resistive shunt sensing measures current indirectly by inserting a low-value resistor in series with the load and measuring the voltage developed across it. The principle follows Ohm's law:

$$V_{shunt} = I_{load} \times R_{shunt} \quad (3.1)$$

where V_{shunt} is the voltage across the resistor, I_{load} is the load current, and R_{shunt} is the selected shunt resistance. The measured voltage can be amplified, filtered, and digitized to reconstruct the target current waveform. For example, a hypothetical 0.10 Ω shunt carrying 0.20 A produces 20 mV. These values are illustrative and are not finalized NG PULSE specifications.

A shunt resistor can be installed on the high side, between the supply and the load, or on the low side, between the load return and ground. High-side sensing preserves the target's ground reference but requires a sensing circuit that tolerates the supply common-mode voltage. Low-side sensing simplifies common-mode requirements, but the additional resistance in the return path can shift the target ground potential and affect interfaces connected to other equipment. Because NG PULSE is intended to be an inline monitor rather than a controller, the location of the resistor must be chosen to minimize disturbance of the target and its external connections.

The principal electrical tradeoff is that increasing shunt resistance produces a larger voltage signal but also increases voltage drop and dissipated power:

$$V_{drop} = I_{load} \times R_{shunt} \quad (3.2)$$

$$P_{shunt} = I_{load}^2 \times R_{shunt} \quad (3.3)$$

Excessive voltage drop can alter the target's operating voltage and therefore its current signature, while power dissipation can cause resistor heating and resistance drift. A low-temperature-coefficient resistor with an appropriate power rating is consequently desirable. Four-terminal (Kelvin) sensing, where practical, reduces errors caused by voltage drops along current-carrying PCB traces and connections.

A resistive shunt can measure DC and changing current, and its output is approximately linear over its specified operating range. It is well suited to a compact PCB and can provide useful waveform fidelity when paired with an amplifier and ADC of sufficient bandwidth. However, the small sense voltage may be affected by amplifier offset, electrical noise, resistor tolerance, and PCB layout. These error sources must be evaluated against the minimum current variation NG PULSE needs to distinguish.

For NG PULSE, resistive shunt sensing is a strong preliminary candidate because of its circuit simplicity, functionality, and accuracy when combined with an op-amp. The next design step is to measure the ESP32 target's actual current range and transient behavior, then select the shunt resistance and amplifier gain together so that the ADC input range is used effectively without excessive rail voltage loss.

3.1.1.2 Hall-Effect Current Sensing

Hall-effect current sensors infer current from the magnetic field generated by a conductor. When a current flows through a sensing conductor, the resulting magnetic field interacts with a biased Hall element, producing a voltage related to the field strength. A sensor's internal conditioning circuitry typically converts this small signal into an analog or digital current measurement. Unlike a current transformer, a Hall-effect sensor can measure steady DC current as well as time-varying current.

For an analog Hall-effect sensor, the transfer characteristic is often approximated as:

$$V_{out} = V_{zero} + S \times I_{load} \quad (3.4)$$

where V_{zero} is the output at zero current and S is the sensor sensitivity in volts per ampere. The actual relationship, offset, bandwidth, and current direction conventions depend on the selected device. The measured current can be estimated by rearranging this relationship:

$$I_{load} = (V_{out} - V_{zero}) / S \quad (3.5)$$

A key advantage is that some Hall-effect sensing architectures provide galvanic isolation between the current-carrying conductor and the measurement electronics. This can be valuable when the monitored rail has a different electrical reference or higher voltage. Certain devices also offer low insertion resistance because they do not require a deliberately large series sense resistor. Isolation, however, is not guaranteed by the term "Hall-effect sensor"; it must be confirmed from the selected component's construction and ratings.

Potential disadvantages include offset drift, sensitivity to external magnetic fields, temperature-dependent gain changes, and limited precision for small current variations. Some integrated Hall-effect devices are optimized for currents much larger than those of a small microcontroller board. In that situation, the useful workload-dependent variations may occupy only a small fraction of the sensor's full-scale output, making them difficult to distinguish after ADC quantization and

noise. Bandwidth also varies significantly between devices and must be checked against the desired sampling rate.

Hall-effect sensing remains a reasonable alternative for NG PULSE if isolation or reduced insertion loss becomes a dominant requirement. Before selecting it, the team would need to verify that a candidate device has adequate low-current sensitivity, noise performance, bandwidth, package size, supply compatibility, and cost.

3.1.1.3 Current-Transformer Current Sensing

A current transformer (CT) uses electromagnetic induction to sense a changing current. The monitored conductor acts as a primary winding, while a secondary winding produces a current proportional to changes in magnetic flux. With an appropriate burden resistor, this secondary current is converted into a measurable voltage. For an ideal transformer operating within its intended frequency range, the approximate current relationship is:

$$I_{secondary} / I_{primary} \approx N_{primary} / N_{secondary} \quad (3.6)$$

where $N_{primary}$ and $N_{secondary}$ are the respective winding counts. The secondary voltage depends on the secondary current and the burden resistance. Practical CT behavior is also affected by core properties, frequency, magnetizing current, saturation, and loading.

Current transformers are widely used for alternating-current measurement because they can provide electrical isolation and introduce little direct series resistance in the primary conductor. They can be useful for measuring AC ripple or switching-current components when appropriately designed for the relevant frequency band. However, a conventional CT cannot directly measure a constant DC current: after the transient associated with a current change, a steady current does not produce the changing magnetic flux required to sustain a secondary signal.

This limitation is significant for NG PULSE. The project needs to compare current signatures across operating modes, including steady or slowly varying consumption. A CT alone would omit the DC component of the supply current and could distort low-frequency workload behavior. It would therefore not provide a complete current waveform for golden-profile generation. A CT might serve as a supplementary sensor for specific high-frequency features, but adding it would increase circuit complexity and would not eliminate the need for a DC-capable measurement method.

Current-transformer outputs also require careful termination and frequency-response analysis. Depending on the device and operating conditions, an open secondary can generate hazardous voltages; manufacturer instructions for burden resistance and safe handling must be followed. Given NG PULSE's low-voltage DC supply-rail application, a conventional CT is not best suited as the primary sensing technology.

3.1.1.4 Technology Comparison and Preliminary Selections

Table 3.1: Technology Comparisons

Criterion	Resistive shunt	Hall effect	Current transformer
Measures steady DC	Yes	Yes	No
Measures changing current	Yes, with suitable front end	Yes, bandwidth dependent	Yes, within frequency range
Isolation	No inherent isolation	Available in some devices	Typically provided
Supply-rail disturbance	Series voltage drop	Device dependent; often low	Little primary series resistance
Small-current sensitivity	Can be optimized through R and gain	Device dependent; offset may limit	Not suitable for DC baseline
Key limitations	Power loss, noise, offset, layout	Offset, drift, external fields, bandwidth	Cannot recover steady DC; frequency limits
NG PULSE suitability	Strong preliminary candidate	Viable alternative	Not suitable as sole sensor

For the current NG PULSE architecture, a low-value shunt resistor with an appropriate current-sense amplifier is the preliminary preferred approach. It supports both DC and transient measurements, fits the proposed analog-to-digital signal chain, and can be tuned for the target's measured current range. Hall-effect sensing should remain a documented alternative if isolation or insertion loss becomes more important than the achievable small-signal precision. A conventional current transformer should not be selected as the sole sensor because it cannot capture the DC component required for complete supply-rail profiling.

This recommendation is provisional. The final choice should be supported by measured minimum and maximum target current, the smallest anomaly amplitude of interest, sensor and amplifier bandwidth, input-referred noise, total rail voltage drop, power dissipation, cost, and experimental detection performance. Any specific numerical limits should be reconciled with the final engineering specifications before parts are ordered.

3.1.2 Microcontroller

3.1.2.1 Role of the Microcontroller in NG PULSE

In NG PULSE the microcontroller (MCU) is the stateful, connected half of the monitor. The FPGA decides quickly and deterministically whether the current signature has departed from the golden profile. The MCU does everything around that decision. It digitizes the conditioned current-sense voltage, stores the golden profiles in non-volatile memory, loads detection parameters into the FPGA at power-up, holds the ESP32-C6 target in reset until the monitor is ready, and logs the FPGA's error flag (basic goal). Two further functions are stretch goals in the Divide and Conquer (D&C) document: it can drive the protective outputs that automatically shut down the target (target reset and, in Rev 2, the power-FET kill

line), and it can report status over Bluetooth Low Energy (BLE). These responsibilities set the selection criteria:

• Integrated analog-to-digital conversion. Using an ADC built into the MCU removes a discrete ADC, its reference, and its digital interface from the board. The ADC needs at least 12-bit resolution (Table 2.1) and a sample rate high enough to resolve workload-dependent current changes. In practice that means tens to hundreds of kilosamples per second.

• Wireless connectivity. A stretch goal calls for status reporting to a mobile device, which favours an MCU with an on-chip BLE radio over adding a second radio IC.

• Peripheral and interconnect resources. At least one I²C master (FPGA configuration), one SPI master (sample transfer, see 7.2), edge-triggered GPIO interrupts for ALERT# and CRITICAL#, and push-pull outputs for target reset and FET control. Hardware event routing that can trigger ADC conversions from a timer without CPU involvement is strongly preferred, because it gives deterministic sample timing.

• Non-volatile storage for at least four golden profiles plus thresholds. The data is a few hundred bytes, so any MCU with internal flash qualifies.

• Hand-assembly. The team cannot reliably solder BGA or very fine-pitch parts (Section 3.1.3). The MCU must come in a leaded package, or as a castellated module with edge pads.

• Independence from the target. The monitor should not share silicon, firmware or a supply-regulation path with the processor it protects. A compromise of the target must not compromise the monitor.

• Team familiarity and available hardware. A team member already owns a Nordic nRF52832 development kit with an on-board debug probe, which lowers bring-up risk.

3.1.2.2 Microcontroller Technology Comparison

Four classes of processing device could fill the MCU role. Each is described below and compared in Table 3.2.

8-bit microcontrollers (e.g., Microchip AVR and PIC families) are inexpensive, come in hand-solderable DIP, SOIC and TQFP packages, and are easy to program. Their ADCs are typically 10-bit with sample rates in the low tens of kilosamples per second. That falls short of the 12-bit requirement, and averaging only partly makes up the difference. Their single 8-bit datapath and small RAM also make it difficult to buffer sample bursts or run a BLE stack. A separate BLE IC and an external 12-bit ADC would both be needed.

32-bit general-purpose microcontrollers without a radio (e.g., ST STM32, Raspberry Pi RP2040) offer fast 12-bit ADCs, DMA, many serial peripherals, and LQFP packages. The STM32G4 family also integrates operational amplifiers and comparators that could absorb parts of the analog front end. Their limitation for NG PULSE is the lack of an integrated radio. The BLE stretch goal would need a

second wireless IC or module, with its own antenna, firmware stack and interface.

32-bit wireless system-on-chip (SoC) devices (e.g., Nordic nRF52 series, Espressif ESP32 series, TI CC26xx) combine a 32-bit core, a 12-bit ADC, DMA, serial peripherals and a 2.4 GHz radio in one device. Because no leaded BLE SoC exists, these are used either as bare QFN parts or as pre-certified castellated modules that include the antenna, matching network and crystals. Leaded packages are avoided at 2.4 GHz because long leads add parasitic inductance and radiate. Wireless SoCs cover every functional requirement in one part.

Application processors and single-board computers (e.g., a Raspberry Pi running Linux) provide the most compute and the easiest networking. However, they bring a non-deterministic operating system, boot times measured in seconds, hundreds of milliamps of supply current, and no precision ADC. They are poorly matched to a small, always-on, deterministic monitor that must start before the target does.

Table 3.2: Microcontroller Technology Comparison

Criterion	8-bit MCU	32-bit MCU (no radio)	32-bit wireless SoC	Application processor / SBC
Integrated 12-bit ADC	Rare (usually 10-bit)	Yes, often > 1 MSPS	Yes, typically ≤ 200 kSPS	Usually none
On-chip BLE	No	No	Yes	Via add-on or SoC Wi-Fi/BT
Timer-triggered ADC with DMA	Limited	Yes	Yes	Not applicable
Deterministic real-time behaviour	Yes	Yes	Yes (radio stack must be scheduled)	No (OS scheduling)
Boot time	ms	ms	ms	seconds
Typical active current	mA	mA–tens of mA	mA (radio peaks ~5–15 mA)	hundreds of mA
Hand-solderable options	DIP/SOIC/TQFP	LQFP	Castellated module	Board-level only
Parts needed for all NG PULSE functions	MCU + ADC + BLE IC	MCU + BLE IC	One module	SBC + ADC + real-time co-processor
NG PULSE suitability	Poor	Good (no BLE)	Best fit	Poor

Based on Table 3.2, a 32-bit wireless SoC was selected as the MCU technology. It is the only class that meets the ADC, peripheral, deterministic-timing and BLE requirements with a single component. A radio-less 32-bit MCU remains the fallback if the BLE stretch goal is dropped. Specific parts are compared in Section 3.2.1.

3.1.2.3 Integrated ADC Considerations

Choosing an MCU with a built-in ADC moves part of the analog design into the MCU, so the ADC's input behaviour had to be researched as well as its headline resolution and rate. The ADCs in all three 32-bit candidate classes are successive-approximation (SAR) converters. A SAR ADC does not measure its input continuously. It connects a small sampling capacitor to the pin for a fixed acquisition time, then disconnects it and converts the stored charge. Three consequences follow for NG PULSE:

• Source impedance and acquisition time are linked. The sampling capacitor must charge to within a fraction of one code during the acquisition time. A high-impedance source, such as a resistor divider or a slow amplifier output, needs a longer acquisition time and therefore a lower sample rate. To reach its maximum rate, an MCU ADC needs a low-impedance driver: an op-amp or current-sense amplifier output, optionally followed by a small RC filter whose capacitor supplies the charge for the sampling capacitor.

• Anti-aliasing must be done in the analog front end. The ADC samples at a fixed rate, so any current-signature content above half the sample rate folds back into the band of interest and distorts the averaged signature. The current-sense amplifier's bandwidth, or a simple RC low-pass between it and the ADC pin, must roll off below the Nyquist frequency of the chosen sample rate.

• Resolution can be traded for rate. Many MCU ADCs, including the nRF52 family's, can average 2^n conversions in hardware (oversampling). This lowers random noise at the cost of a proportionally lower output rate. For NG PULSE the FPGA already averages over a window (Section 7.1), so oversampling in the ADC is kept as a tuning option rather than a requirement.

These considerations do not change the technology choice in Table 3.2, but they set requirements on the current-sense board's output stage. They are carried into the part-level ADC configuration in Section 3.2.1.2.

3.1.2.4 Clock Accuracy and Sample Timing

The FPGA compares averages taken over a fixed number of samples, so the time between samples must be constant. MCU timers are clocked either from an on-chip RC oscillator or from an external crystal. The RC oscillator starts instantly, but it is far less accurate and drifts with temperature, which would change the effective sampling rate between characterization and monitoring. A crystal oscillator is accurate to tens of parts per million. Wireless SoCs need a crystal for the radio anyway, and castellated modules include one, so crystal-referenced sample timing costs nothing extra on this class of device. The

firmware must start the crystal oscillator before sampling begins and keep it running while monitoring (Section 7.2.3).

3.1.2.5 Measurement Independence and Supply

The monitor must measure only the target's current. If the MCU, FPGA or regulators drew their supply through the target's shunt, their own current, which changes with BLE activity and FPGA switching, would be added to the measurement and would corrupt the golden profile. The monitor's supply therefore branches from the USB-C input before the shunt (Section 3.1.6). The MCU's own current draw then affects only the monitor's power budget, not the measurement. This favours a low-power MCU, but it does not make low power a hard requirement.

A related effect is digital noise. The MCU's radio and DC/DC converter switch at high frequencies and can couple into the analog input through the ground and supply. The MCU's analog input and reference should therefore be routed away from the radio and switching nodes. The firmware can also avoid radio activity during characterization runs if that coupling turns out to be measurable.

3.1.2.6 Security of the Monitor Itself

A monitor that can be reprogrammed or read out by an attacker provides little assurance. The MCU therefore needs:

• a way to lock the debug port for deployment. On the nRF52 family this is the access-port protection (APPROTECT) setting, which blocks SWD access to the CPU and memory until the chip is fully erased;

• internal flash for firmware and golden profiles, so that neither has to travel over an external bus that could be probed or replaced;

• optionally, signed firmware updates through a bootloader, so that only team-signed images can run (Section 3.1.7.7).

During development the debug port stays open, because the team needs SWD to flash and debug the firmware. Locking it is a deployment step, and the procedure will be documented with the test plan. Fault-injection attacks against the monitor's own MCU, such as voltage glitching, are outside the project's scope. They are noted here because NG PULSE is itself designed to detect that class of physical attack on its target.

3.1.3 FPGA

3.1.3.1 Role of the FPGA

The FPGA serves as the primary digital processing platform for the NG PULSE system. Its purpose is to receive, organize, and process digital samples representing the current consumption of a monitored host processor. The FPGA implements the data patch necessary through sample buffering, feature extraction, comparison against the established golden profile, and generating alerts when abnormal behavior occurs.

The digital logic for the datapath will be developed using Verilog, a hardware description language (HDL) that describes the structure and behavior of digital circuits. Verilog describes hardware functionality that can operate through interconnected logic, unlike conventional software which executes instructions sequentially. This allows NG PULSE's functions to be implemented as hardware modules with defined inputs, outputs, and control signals.

The FPGA is intended to support the physical prototype while also providing a hardware implementation that can be carried forward into the project's application-specific integrated circuit (ASIC) development. The same Verilog RTL will be used for the ASIC design in the Cadence Genus environment.

The programmable logic device must be able to accommodate the primary detection functionality while providing sufficient resources for potential design extensions. The detection datapath includes sample buffering, statistical calculations, comparison to the golden profile, and generating a flag. The statistical calculations include the mean, variance, and peak-to-peak values. A two-tiered ALERT/CRITICAL notification scheme is representative of an additional design extension intended to distinguish between different levels of deviations. The advanced design goal includes a four-profile reference bank which allows the system to evaluate the processor's behavior against multiple established operating profiles rather than a single reference.

The selected FPGA must satisfy several hardware and development requirements to support these functions. First, it must provide sufficient embedded memory to accommodate two alternating sample buffers, each containing 1,024 samples containing 12 bits. The total memory requirement is then:

$$M_T = 2 \times 1024 \times 12 = 24{,}576\ bits$$

$$M_T = \frac{24{,}576\ bits}{8\frac{bits}{byte}} = 3{,}072\ bytes$$

This shows that the two sampling windows require a total of 3,072 bytes. The buffers will operate using a ping-pong architecture in which one buffer receives incoming ADC samples while the other retains the most recently completed sampling window for feature extraction. This allows sample acquisition and statistical processing to be managed independently. By managing these independently the likelihood that incoming samples will be lost while a completed window is being processed is minimized.

The FPGA must also support the processing requirements necessary for the system's desired sampling rate. The targeted sampling rate for the project is 200 kSPS. A crystal referenced clock operating in the tens of megahertz range will provide the timing reference for the FPGA's internal operations. The selected clock frequency must provide sufficient processing time for sample reception, buffer management, statistical calculations, and communication with the remaining components.

The FPGA must support compatible I/O voltage levels for communication with the system's external components. The intended architecture uses 3.3 V digital I/O to interface with the Nordic nRF52832 and Espressif ESP32-C6 microcontrollers. Establishing compatible voltage levels is important because the FPGA must exchange data and control signals with these devices without requiring additional level shifting circuitry. The selected I/O supply voltage and electrical characteristics will be verified against the FPGA and microcontroller datasheets during hardware integration.

The physical package of the selected FPGA is another requirement that was considered. The project requires a leaded package that can be assembled using practical soldering methods available for the prototype. Ball-grid-array (BGA) packages are excluded from consideration because the solder connections are located underneath the integrated circuit. This makes inspection and manual assembly more difficult. A leaded quad flat package (LQFP/TQFP) is then the preferred for the custom PCB.

Lastly, the FPGA needs to support a development and synthesis toolchain capable of accepting portable Verilog code. The RTL must be suitable for preliminary simulation in AMD Vivado and synthesis within the Cadence Genus ASIC development flow. To maintain compatibility across these environments, the design will avoid unnecessary dependence on device-specific FPGA primitives. Where possible, standard synthesizable Verilog constructs and inferred hardware resources will be used instead of any vendor-specific options.

These requirements establish the criteria used to evaluate the available programmable logic technologies and selection of the FPGA used for the NG PULSE.

3.1.3.2 Programmable Logic Technology Comparison

Several implementation approaches were considered for the digital detection datapath; software processing on a microcontroller, complex programmable logic devices (CPLDs), SRAM-based FPGAs, flash-based FPGAs, and custom ASIC fabrication. Each of these technologies offers different advantage in processing capability, memory availability, configuration, physical integration, and development cost. The selection process took the ability to implement the architecture, while also considering the potential for future modifications, extensions, and the reuse of the developed Verilog RTL.

3.1.3.2.1 Microcontroller Based Software Processing

One possible implementation would perform the detection calculations entirely through software executing on the Nordic nRF52832 microcontroller. The nRF52832 incorporates an ARM Cortex-M4F processor that operates at up to 64 MHz. This would provide sufficient computational capacity for statistical calculations and threshold comparisons.

At the project's target sampling rate of 200 kSPS, a 64 MHz processor would have approximately 320 clock cycles available per incoming sample.

$$C = \frac{64{,}000{,}000}{200{,}000} = 320$$

The calculation shows the theoretical processing interval, but does not account for cycles consumed by any communication, interrupts, or other processor responsibilities. Regardless, it shows that basic operations such as basic statistical calculations and threshold comparisons could be implemented using microcontroller software.

This approach does come with several disadvantages. The nRF52832 would also be responsible for wireless communication and other system-level operations. The possibility of a competition being created for the processor's resources can be created between the continuous sampling when combined with Bluetooth Low Energy (BLE) communication and interrupt handling. The time available for detection calculations would depend on the software scheduling and execution requirements of other tasks.

Using a microcontroller-based software approach also introduces a scenario where detection and response is performed on the same microcontroller. This would reduce the architectural separation between the sensing and responding components of the NG PULSE. The FPGA based approach allows the digital detection datapath to operate independently of the microcontroller responsible for higher-level system functions. The separation in the architecture is an important aspect to consider as it reduces the dependence on the responder's execution schedule.

Finally, a software only implementation would not provide the dedicated hardware datapath needed for the project's future ASIC development. During a project meeting with the sponsor, it was requested that the team maintain an ASIC oriented implementation path. With this, a microcontroller software approach was retained as a possible approach, but not the preferred implementation.

3.1.3.2.2 Complex Programmable Logic Devices (CPLDs)

Complex programmable logic devices are another possible option for implementing the digital processing architecture. CPLDs provide configurable logic resources and are commonly available in nonvolatile configurations. This allows them to retain their programming information without requiring an external configuration device at startup. The nonvolatile configuration and availability in leaded packages make them an attractive option, but they provided less logic and embedded memory than FPGAs with comparable package sizes.

This limitation is particularly important for the NG PULSE because the system requires 24,576 bits of memory for the two 1,024-sample buffers alone. Implementing this storage entirely with ordinary logic resources would use a significant amount of the available programmable logic in a smaller CPLD. The additional requirements associated with feature extraction, sample management, comparison logic, and alert generation would further increase resource utilization.

Using a CPLD was therefore deemed to be unsuitable. The limited embedded memory and processing resources make them less appropriate than other options that were explored.

3.1.3.2.3 SRAM-Based FPGAs

SRAM-based FPGAs provide configurable logic, embedded memory, and programmable interconnections within a single integrated circuit. These devices have greater processing capacity compared to CPLDs and are available in multiple package configurations.

The first consideration of the SRAM-based FPGA is that the programmable logic is stored in volatile SRAM and must be loaded to the device every time it powers on. This can be accomplished through an external serial peripheral interface (SPI) flash memory or another source. This configuration would be manageable because the system already incorporates a microcontroller responsible for the higher-level system coordination. This can be done by having the nRF52832 hold everything until the FPGA indicates that the configuration has been completed. With this setup, the monitored system would be begin normal operation before the monitoring system is ready.

SRAM-based FPGAs also provide do have an advantage when considering the available development tools. The Lattice iCE40 family is supported by both vendor development software and open-source tools. These tools, like Yosys, nextpnr, and Project IceStorm, provide a development path to synthesize Verilog designs and implement them onto the iCE40 devices.

This would also provide the opportunity for us to reuse the same portable RTL across the FPGA and ASIC development process. As mentioned previously, the project intends to use the FPGA as a hardware platform while creating the corresponding ASIC-oriented implementation through Cadence Genus.

For the reasons listed above, the SRAM-based FPGA technology was selected as the preferred approach.

3.1.3.2.4 Flash-Based Field-Programmable Gate Arrays

Flash-based FPGAs were another possibility for implementation. These FPGAs retain their configuration information in nonvolatile memory. This would allow for the device to retain its configuration when the power is removed and would eliminate the need for a separate external flash device. The immediate availability of the programmed logic following the device powering on would be beneficial for embedded monitoring applications. Some of the flash-based FPGA families are also available in leaded packages, which is a compatibility requirement of the physical portion of the project.

However, flash-based devices differ in available logic and embedded memory resources. Certain devices will have limited available RAM when compared to the SRAM-based FPGAs. The development toolchains may also be more dependent on proprietary software, with fewer open-source options.

The Lattice MachXO2/MachXO3 devices and Microchip IGLOO FPGAs were potential candidates for implementation. Although these devices provide viable programmable logic alternatives, the resource availability and development environment would need to be evaluated against the memory requirements and RTL objectives of the project.

Flash-based FPGAs were retained as a possible choice, but not selected for the primary implementation.

3.1.3.2.5 Application-Specific Integrated Circuits (ASICs)

An application-specific integrated circuit is a custom integrated circuit designed to perform a particular set of functions. Unlike and FPGA, which contains programmable logic resources, an ASIC is manufactured with a fixed hardware configuration that is based on the completed design.

A custom ASIC provides several advantages for the NG PULSE. Mainly, this provides the ability to optimize the digital detection datapath specifically for the intended monitoring application. Since this would be designed for the required functionality, unnecessary resources could be eliminated. As a result, an ASIC could also have a reduced physical size and lower power consumption.

The fabrication processs introduces significant cost, development time, and manufacturing requirements though. The project schedule and available budget do not support the fabrication of a custom ASIC within the time allotted.

As a result, the ASIC implementation will be investigated through the DRACO Laboratory's Cadence design flow. This will allow for the same portable Verilog RTL developed for the FPGA to be used as the starting point for the ASIC-oriented synthesis process. Cadence Genus will be used to synthesize the digital design into an ASIC-oriented netlist with the technology libraries that are available.

This will allow the team to investigate the feasibility of transferring the FPGA-developed logic into an ASIC design without requiring the entire fabrication process. Given the constraints, the ASIC implementation is considered a simulation and synthesis objective rather than a hardware deliverable.

Table 3.3: Programmable Logic Technology Comparison

Criterion	MCU Software	CPLD	SRAM FPGA	Flash FPGA	ASIC
Logic Capacity for datapath	Limited by CPU resources	Too small	Ample	Device dependent	Ample
Memory for two 1,024 x 12 buffers	Uses MCU RAM	Insufficient	Available	Device dependent	Custom memory macro

Processing determinism	Depends on software scheduling	Hardware based	Hardware based	Hardware based	Hardware based
Configuration	Firmware	Nonvolatile	Typically external configuration source	Nonvolatile	Fixed after fabrication
Leaded Package Availability	Not applicable	Available	Available in select families	Available in select families	Depends on fabrication
Reuse of Verilog RTL for ASIC	Limited	Possible	Yes	Yes	Native implementation
Development Cost and Schedule	Lowest Initial cost	Low	Low to moderate	Low to Moderate	High
NG PULSE suitability	Fallback alternative	Unsuitable for selected architecture	Selected	Viable alternative	Simulation only

Based on the comparison, SRAM-based FPGA technology was selected for the NG PULSE programmable logic implementation. This option provides sufficient logic and memory resources for the current detection architecture while allowing additional capacity for future extensions. The support that is available through both vendors and open-source tools also provides the best flexibility throughout the process. This includes the ability to reuse the portable Verilog RTL from the FPGA for the ASIC design using Cadence.

3.1.4 Target RISC-V Processor/Platform

The Seeed Studio XIAO ESP32C6 development board was selected as the target device for the NG PULSE system. The board incorporates the Espressif ESP32-C6 system-on-chip (SoC), which features a 32-bit RISC-V high-performance processor operating at frequencies up to 160 MHz and an additional low-power processor operating at up to 20 MHz. It also includes 512 KB of SRAM, 4 MB of flash memory, and support for multiple communication interfaces, including UART, SPI, and I²C.

The XIAO ESP32C6 features a compact footprint of approximately 21 × 17.8 mm and supports a nominal 5 V input through its USB-C connector. Its small dimensions and accessible pin headers make it suitable for integration into NG PULSE's modular PCB architecture, where the target board will be mounted above the main PCB.

The primary purpose of the ESP32-C6 is to provide a programmable electrical workload whose current consumption can be monitored for deviations from expected operating behavior. The target will execute four predefined operating

modes, in addition to an inactive state. Each operating mode will produce a characteristic supply-current signature that NG PULSE can capture and use to establish a reference profile.

The target will receive power through the USB-C connection on the current capture board. A high-side shunt resistor and current-sense amplifier will measure variations in the target's supply current without requiring modifications to the target's internal circuitry.

To identify the active operating mode, the ESP32-C6 will transmit mode information to the Nordic microcontroller using a unidirectional UART connection. The board's D6 pin (GPIO16) supports UART transmission and can be connected to the Nordic MCU's UART receive input. A common ground reference will also be maintained between the two devices. Additional header connections may be used to improve mechanical stability when mounting the target above the main PCB.

The ESP32-C6 was selected primarily for its programmable operating modes, USB-C power interface, compact form factor, and straightforward integration with the monitoring system. Although the device also supports Wi-Fi 6, Bluetooth Low Energy, Zigbee, and Thread, these wireless capabilities are not required for the basic NG PULSE monitoring architecture. The board provides a representative embedded computing platform for developing, capturing, and evaluating current-signature-based anomaly detection.

3.1.5 Protection/Shutdown Circuitry

[TO DO: Technology comparison, then part comparison table (MOSFET).]

3.1.6 Power Supply

3.1.6.1 Power Supply Architecture and Requirements

The NG PULSE system is designed to operate as an integrated monitoring subsystem within a larger electronic platform rather than as an independent, battery-powered device. Its primary purpose is to observe the electrical current signature of a host microcontroller, identify deviations from expected operating conditions, and generate an error signal when an anomaly is detected. Consequently, NG PULSE does not require a dedicated external power source. Instead, it obtains its operating power from the same laptop USB-C connection used to supply the ESP32-C target board.

The USB-C connection provides a nominal 5 V input to the monitoring hardware. This input must be distributed to multiple subsystems with different voltage requirements, including the Nordic nRF52832 microcontroller, Lattice iCE40 FPGA, analog current-sensing circuitry, and peripheral components. Since these devices cannot all operate directly from the 5 V supply, voltage regulation is required to generate the appropriate operating rails.

The proposed power architecture includes a 3.3 V regulated rail for the Nordic microcontroller, FPGA input/output banks, and compatible peripheral circuitry. A separate lower-voltage rail is required for the FPGA core. The iCE40HX4K

datasheet specifies a nominal 1.2 V core supply, with a recommended operating range of 1.14–1.26 V. Therefore, the lower-voltage regulator should be designed around 1.2 V, assuming the iCE40HX4K remains the selected FPGA.

The Nordic nRF52832 supports an operating supply range of 1.7–3.6 V, making 3.3 V compatible with its electrical requirements.

Table 3.4: Power Distribution Requirements

Component	Voltage
USB-C Input	5V
ESP32-C6 Target Board	5V
Nordic nRF52832 MCU	3.3V
Lattice iCE40HX4K FPGA Core	1.2V
FPGA I/O Banks	3.3V
Current-Limiting Circuitry	5V

3.1.6.2 USB-C Power Source Selection

Several power-source technologies are available for embedded electronic systems, including batteries, dedicated AC-to-DC adapters, laboratory power supplies, and USB-powered connections. Each provides different advantages regarding portability, available power, system complexity, and integration.

Battery-powered systems are useful when portability or operation without external equipment is required. However, batteries introduce additional design requirements such as charging circuitry, battery protection, energy-capacity calculations, and maintenance. Since NG PULSE is intended to operate only while the host system is powered, an independent battery would increase complexity without directly supporting its primary monitoring function.

A dedicated external power adapter could provide an independent and potentially higher-capacity power supply. However, this would require an additional cable and power connection, reducing the convenience of integrating NG PULSE into the host platform.

A USB-powered architecture offers a more appropriate solution. The laptop already provides the electrical connection required to power the ESP32-C board, allowing NG PULSE to obtain power without introducing an additional external supply. This simplifies integration, reduces the number of required connections, and allows the monitoring subsystem to operate whenever the host platform is connected. As an added benefit, powering our device via a USB-C connector in a

laptop allows us to easily test our product, as power delivery USB-C ports can be pulled much higher than 5V on command.

3.1.6.3 Power Distribution and Voltage Regulation

The NG PULSE system receives a nominal 5 V input from the laptop's USB-C connection. However, the individual components within the system operate at different supply voltages, requiring additional voltage regulation. The proposed power distribution architecture incorporates two voltage regulators to convert the 5 V input into the 3.3 V and 1.2 V supplies required by the monitoring hardware.

The 3.3 V rail supplies the Nordic nRF52832 microcontroller and the applicable input/output (I/O) banks of the Lattice iCE40HX4K FPGA. The FPGA core requires a separate nominal 1.2 V supply to operate correctly. The remaining 5 V input is distributed to the ESP32-C target board. The current-sensing amplifier and main NG PULSE circuitry will receive power through a current-limiting device, which provides protection for our device to prevent a surge from destroying that circuitry as well, rendering the product useless.

The selection of voltage regulators is important because the monitoring subsystem must operate efficiently without introducing excessive electrical noise or unnecessarily increasing the power consumption of the overall system. Two primary voltage regulation technologies were considered: linear regulators and switching regulators.

3.1.6.3.1 Linear Voltage Regulators

Linear voltage regulators reduce an input voltage to a lower, regulated output voltage by controlling the conduction of an internal pass element. Low-dropout (LDO) regulators are a common type of linear regulator used in embedded electronic systems because of their simple implementation, relatively low component count, and generally low output noise.

One advantage of linear regulators is that they do not require the high-frequency switching operation associated with switching converters. This characteristic makes them particularly useful for analog circuitry, where electrical noise can interfere with measurement accuracy. For NG PULSE, minimizing supply noise is important because the system relies on detecting variations in the current consumption of the ESP32-C target.

However, linear regulators dissipate the difference between their input and output power primarily as heat. The approximate power dissipation of a linear regulator can be calculated using this equation:

$$P_{loss} = (V_{in} - V_{out}) * I_{out}$$

Where:

- P_{loss} = power dissipated by the regulator (W)
- V_{in} = regulator input voltage (V)
- V_{out} = regulator output voltage (V)
- I_{out} = regulator output current (A)

For a regulator converting the 5 V USB input to 3.3 V, the power dissipation becomes:

$P_{loss} = (5 - 3.3) \times I_{out}$

$P_{loss} = 1.7 \times I_{out}$

The approximate efficiency of an ideal linear regulator, neglecting quiescent current, can be expressed as:

$\eta = (V_{out} / V_{in}) \times 100\%$

For the 3.3 V rail:

$\eta = (3.3 / 5) \times 100\% = 66\%$

Therefore, approximately 34% of the input power would be dissipated as heat under these simplified conditions.

For a direct conversion from 5 V to 1.2 V, the approximate efficiency would decrease to:

$\eta = (1.2 / 5) \times 100\% = 24\%$

This relatively low efficiency makes a linear regulator less attractive for supplying the FPGA core when its current demand is significant. Nevertheless, linear regulators remain a potential option for lower-current circuitry where simplicity and low noise are more important than conversion efficiency.

3.1.6.3.2 Switching Voltage Regulators

Switching voltage regulators convert electrical energy by rapidly switching semiconductor devices and transferring energy through components such as inductors and capacitors. Unlike linear regulators, switching converters can reduce voltage without continuously dissipating the entire voltage difference across a pass element.

A buck converter is a type of switching regulator specifically designed to reduce a higher DC input voltage to a lower DC output voltage. Since NG PULSE receives a 5 V input and requires 3.3 V and 1.2 V outputs, buck converters are suitable candidates for both regulated supply rails.

The efficiency of a switching regulator is defined as the ratio of output power to input power:

$\eta = P_{out} / P_{in}$

The required input power can therefore be calculated as:

$P_{in} = (V_{out} \times I_{out}) / \eta$

Switching regulators can achieve substantially higher efficiencies than linear regulators when large voltage reductions are required. This can reduce heat generation and minimize the additional current drawn from the laptop's USB connection.

However, switching regulators introduce additional design considerations. Their high-frequency switching operation generates voltage ripple and electromagnetic interference, which may affect nearby analog circuitry. Since NG PULSE measures variations in supply current, unwanted electrical noise could interfere with the accuracy of the current-sensing system.

Based on these characteristics, switching regulators are strong candidates for the NG PULSE power distribution system, and they were used for our preliminary design.

3.1.6.4 Current Limiting and Power Protection

In addition to voltage regulation, NG PULSE incorporates current-limiting circuitry to provide electrical protection for its components. The system is designed to monitor incoming power, so if a surge knocks out the monitoring hardware, it loses its purpose.

A current limiter regulates or restricts the current delivered to a connected load. Depending on the selected component, a current-limiting integrated circuit may also provide features such as overcurrent protection, thermal shutdown, controlled startup, and fault reporting.

The proposed NG PULSE architecture uses a current-limiting device to gate the supply delivered to the current-sensing amplifier and peripheral hardware. This allows the protected circuitry to receive power under normal operating conditions while limiting current if an electrical fault occurs.

The current-limiting threshold must be selected based on the expected operating current of the protected components. A threshold that is too low could interrupt normal system operation, while an excessively high threshold could reduce the effectiveness of the protection.

The current limiter must also be positioned appropriately within the power distribution network. Since NG PULSE is designed to monitor the current signature of the ESP32-C target, the protection circuitry should not unnecessarily alter the electrical characteristics of the monitored supply path.

The current-limiting device is intended to protect the monitoring circuitry and the current-capture amplifier. A current limiter primarily restricts or controls the current supplied to a load. If an attack on the target fried the monitoring hardware, it would be rendered useless, thus necessitating a current limiter for that portion of the board.

The Texas Instruments TPS2553DBVR-1 was selected for NG PULSE due to its adjustable current-limit threshold, integrated protection features, and compatibility with the system's 5 V power architecture. The device supports an operating input voltage range of 2.5 V to 6.5 V and an adjustable current-limit range of approximately 75 mA to 1.7 A, providing flexibility to accommodate the power requirements of the system's supporting electronics.

A primary consideration in selecting this component was its externally programmable current-limit threshold. Rather than relying on a fixed current limit,

the TPS2553DBVR-1 allows the threshold to be configured using a single external resistor. This flexibility is advantageous during prototype development, as the current-limit setting can be adjusted to accommodate changes in the combined power consumption of the Nordic microcontroller, FPGA, and voltage regulation circuitry. A 19.1 kΩ programming resistor was selected for the preliminary design.

The TPS2553DBVR-1 also provides several integrated protection features, including thermal shutdown, reverse-voltage blocking, and fault reporting. The selected latch-off variant disables its output following a sustained overcurrent fault, providing an additional level of protection for downstream components. Its typical on-state resistance of approximately 85 mΩ also helps minimize conduction losses during normal operation.

Another factor influencing component selection was its compact six-pin SOT-23 package. This package occupies minimal PCB space and features externally accessible leads, simplifying component placement, soldering, and inspection during prototype assembly.

Overall, the TPS2553DBVR-1 was selected for its adjustable current-limit capability, integrated fault protection, low on-state resistance, and compact package. These characteristics make it suitable for NG PULSE's modular power distribution architecture while allowing flexibility during hardware development and integration.

3.1.6.5 Power Budget and Efficiency Considerations

The total power consumption of NG PULSE must be evaluated to ensure that the laptop's USB connection can adequately supply both the monitoring subsystem and the ESP32-C target board.

The total input current consists of the current required by the ESP32-C target, the input currents of the voltage regulators, and any additional components operating directly from the 5 V supply.

The input current required by each voltage regulator depends on its output voltage, output current, and conversion efficiency. The FPGA power estimate must account for its core and I/O supplies, while the microcontroller power estimate must account for its expected operating conditions. Additional power losses occur within the voltage regulators, current-limiting circuitry, and current-sensing components.

Because the laptop supplies power to both the ESP32-C target and NG PULSE, the combined current demand must remain within the available USB power budget. The maximum available current depends on the laptop's USB port and the applicable USB power configuration.

The proposed design must also minimize interference with the normal operation of the ESP32-C target. Since the monitoring hardware is intended to operate without significantly affecting the host, the voltage drop introduced by the current-sensing resistor and other series components must be considered. The

sensing resistor should be selected to produce a measurable voltage while minimizing the voltage reduction delivered to the target board.

The final power budget will be completed once the operating currents of the selected components have been established through datasheet analysis and prototype testing.

3.1.6.6 Preliminary Power Supply Technology Selection

Based on the operating requirements of NG PULSE, a USB-powered architecture has been selected over batteries and dedicated external power supplies. This approach allows the monitoring subsystem to function as part of the larger host platform without requiring an independent source of electrical power.

The laptop's nominal 5 V USB-C supply will serve as the primary input. Two voltage regulators will provide the required 3.3 V and 1.2 V supply rails for the Nordic microcontroller and Lattice FPGA. The 5 V supply will also be distributed to the ESP32-C target board and compatible peripheral circuitry.

Both linear and switching regulator technologies were evaluated. Linear regulators offer simpler implementation and relatively low output noise, while switching regulators generally provide greater efficiency when converting the 5 V input to substantially lower voltages. A buck converter is therefore a strong candidate for the 1.2 V FPGA core supply, while the 3.3 V regulator topology will be selected based on the final current requirements and noise considerations.

A current-limiting device will also be incorporated to control and protect the power supplied to the current-sensing amplifier and peripheral hardware.

The final power supply design will prioritize reliable voltage regulation, efficient power conversion, electrical protection, and minimal interference with the current-signature measurements. Specific regulator and current-limiter components will be selected following a comparison of manufacturer specifications, including input voltage range, output current capability, efficiency, noise characteristics, package size, and cost.

3.1.7 Microcontroller Firmware

The nRF52832 runs C firmware written by the team. That firmware:

• drives the monitor's output pins (target reset, power-FET kill, status LED);

• configures and reads the ADC input that digitizes the current-sense signal;

• runs the control loop that boots the system, moves samples to the FPGA and responds to its flags;

• stores the golden profiles in non-volatile memory;

• manages the Bluetooth connection.

The research for the firmware covered six questions. The answers are selected in Section 3.2.3 and the resulting design is described in Section 7.2.

1. Which programming language?

2. Which development platform (SDK)?

3. How should the program be structured?

4. How should ADC samples be acquired?

5. How should the golden profiles be stored?

6. How should Bluetooth be implemented?

3.1.7.1 Programming Language

Four languages can target the nRF52832 (compared in Table 3.5):

• C is the native language of every Nordic SDK and of the vendor drivers and examples. It compiles to small, predictable code and is the language the team's computer engineering members have used most in coursework.

• C++ can use the same SDKs and adds classes and templates. It also adds hidden costs (constructors, exceptions, dynamic allocation) that need discipline to avoid on a small MCU.

• Rust offers compile-time memory safety, but its nRF52 BLE support comes from community crates, and nobody on the team has used it.

• MicroPython/CircuitPython allows fast experimentation, but it is interpreted: it is too slow and too non-deterministic to manage sampling, and it uses much of the 64 KB RAM itself.

Table 3.5: Firmware Language Comparison

Criterion	C	C++	Rust	MicroPython / CircuitPython
Native vendor SDK support	Full	Full (C APIs)	Community crates	Limited port
Determinism and overhead	Excellent	Good, with discipline	Excellent	Poor (interpreted, garbage-collected)
Memory footprint	Smallest	Small	Small	Large
Team experience	High	Moderate	None	Low
Example code for nRF52832 BLE and ADC	Extensive	Some	Limited	Limited

3.1.7.2 Development Platform

The firmware platform (SDK, build system and debug tooling) determines how quickly the MCU functions in Section 7.2 can be brought up and how much low-level control remains available for timing-critical paths. The criteria were:

• Low-level peripheral access. The ADC must be triggered by hardware timer events and drained by DMA with no CPU involvement in the sample timing. The platform must expose the nRF52832 SAADC, PPI, TIMER, SPIM, TWIM and GPIOTE peripherals directly.

• A maintained BLE stack, for the stretch goal.

• Non-volatile storage APIs with wear levelling, for golden profiles.

• Long-term support. The project runs through April 2027, and the platform should not be deprecated during it.

• Debugging through SWD with breakpoints and live variable inspection, using the J-Link probe on the team's nRF52 development kit.

• Learning curve for a team whose firmware experience is mainly in C.

Five options were investigated:

1. Nordic nRF5 SDK (bare-metal C with the SoftDevice BLE stack). It is mature and very well documented, with many examples for the nRF52832. Nordic placed it in maintenance mode and recommends the nRF Connect SDK for new designs; the final release was v17.1.1 [13].

2. Nordic nRF Connect SDK (built on the Zephyr RTOS). This is Nordic's actively developed SDK [13]. It includes the Zephyr BLE host and SoftDevice Controller, a devicetree hardware description, settings and NVS storage subsystems, and the low-level nrfx drivers for direct peripheral access. VS Code integration is through the nRF Connect extension. The learning curve is steeper (Kconfig, devicetree, CMake/west).

3. Arduino (e.g., the Adafruit nRF52 core). It is the quickest way to blink an LED or advertise over BLE, but it hides the PPI and DMA configuration that deterministic sampling needs, and its debug support is limited.

4. Bare-metal register-level C (CMSIS headers and nrfx drivers only). It gives full control and the smallest footprint, but every BLE, storage and scheduling service has to be written or integrated by hand.

5. Embedded Rust (e.g., the Embassy framework with embassy-nrf). It offers memory safety and modern async I/O. It is newer, has fewer nRF52832 BLE examples, and no team member has used it.

The options are compared in Section 3.2.3 (Table 3.14).

3.1.7.3 Program Structure

Embedded firmware is usually organized in one of three ways (compared in Table 3.6):

• Super-loop (polling). One endless loop checks each input in turn: ADC ready, flag pins, Bluetooth events. It is the simplest to write and understand. Its response time depends on how long one pass of the loop takes, so a slow task (for example, a flash write) delays everything else, including the response to an attack flag.

• Interrupt-driven foreground/background. Hardware interrupts handle urgent events (flag edges, ADC buffer complete) immediately, and set flags that a background loop processes. Response is fast, but as features are added it gets harder to keep interrupt handlers short and shared data consistent.

• Real-time operating system (RTOS) threads. Each function (sampling bookkeeping, attack policy, supervision, telemetry) runs as a separate thread with a priority, and the scheduler guarantees the highest-priority ready thread runs first. Interrupts stay short and hand work to threads through queues. This adds some RAM and learning overhead. In exchange, the attack-response path can be guaranteed to pre-empt Bluetooth and logging work.

Table 3.6: Program Structure Comparison

Criterion	Super-loop	Interrupt-driven	RTOS threads
Worst-case flag response	One full loop pass (unbounded as features grow)	Interrupt latency (µs)	Interrupt latency + highest-priority thread (µs)
Coexistence with BLE stack	Poor	Fair	Good (designed for it)
Complexity to start	Lowest	Moderate	Moderate–high
Scalability as features are added	Poor	Fair	Good
RAM overhead	None	None	A few KB (thread stacks)

3.1.7.4 ADC Input Acquisition Method

The ADC can be triggered and read in three ways. The method determines how evenly the samples are spaced in time (jitter). Uneven spacing distorts the averaged current signature the FPGA compares against the golden profile. The three methods are compared in Table 3.7.

• Software polling. The main loop starts a conversion, waits for it to finish and reads the result. The sample spacing depends on everything else the loop does, so jitter is large and unpredictable.

• Timer interrupt. A hardware timer fires an interrupt, and the handler starts each conversion. Spacing is much better, but any higher-priority interrupt delays the sample. On the nRF52832 the Bluetooth radio's interrupts can pre-empt

application interrupts, so jitter grows when Bluetooth is active. It also costs one interrupt per sample, which is 200,000 per second at full rate.

• Hardware-triggered with DMA. A hardware timer event is routed through the nRF52832's Programmable Peripheral Interconnect (PPI) directly to the ADC's start-conversion task, and EasyDMA writes each result into RAM. No software runs per sample: spacing is set by the crystal-driven timer alone, and the CPU is interrupted only once per buffer of hundreds of samples.

Table 3.7: ADC Acquisition Method Comparison

Criterion	Software polling	Timer interrupt	Hardware trigger (TIMER → PPI → SAADC) + EasyDMA
Sample-timing jitter	Large, depends on loop	Interrupt latency; worse with BLE	Set by the timer only (lowest)
CPU load at 200 kSPS	100%	Very high (one interrupt per sample)	One interrupt per buffer
Maximum practical rate	Low	Moderate	Full 200 kSPS
Complexity	Lowest	Low	Moderate (PPI and DMA setup)

3.1.7.5 Golden Profile Storage

The golden profiles (a mean, an allowed deviation and thresholds for each operating mode) are only a few hundred bytes. They must survive power loss and must not be silently corrupted. The four options are compared in Table 3.8.

• Hard-coded in the firmware image. The simplest and the hardest to tamper with, but every re-characterization needs a rebuild and reflash. This is the approach used on the ASIC track, where the values are fixed in silicon.

• Raw internal flash writes. Through the flash controller (NVMC). Fast, no extra parts. The firmware must handle page erasure, wear and power-loss during a write itself.

• Wear-levelled key-value storage in internal flash (Zephyr NVS/settings, or FDS in the nRF5 SDK). It handles erasure, wear levelling and interrupted writes, at the cost of a few kilobytes of flash.

• External EEPROM or flash over I²C or SPI. Separates profile storage from the firmware, but adds a part, board area and a bus transaction to every read.

Table 3.8: Golden-Profile Storage Comparison

Criterion	Hard-coded	Raw internal flash	Wear-levelled storage (NVS/settings)	External EEPROM

Update without reflashing	No	Yes	Yes	Yes
Power-loss safety	n/a	Must be written by the team	Built in	Device dependent
Extra hardware	None	None	None	Yes
Tamper resistance	Highest	Depends on update path	Depends on update path	Lower (separate bus)
Effort	Lowest	High	Low	Moderate

3.1.7.6 Bluetooth Connectivity (Stretch Goal)

Bluetooth reporting to a mobile device is a stretch goal in the D&C. It is researched now because it drove the choice of MCU. Bluetooth Low Energy (BLE) offers several ways to get monitor status to a phone (compared in Table 3.9):

• Advertising-only broadcast. Status is placed in the BLE advertising packet and any nearby phone can read it without connecting. This is simple, but anyone can see the status, nothing can be sent back, and the payload is about 31 bytes.

• Nordic UART Service (NUS). A serial port over BLE. Very easy to use with existing phone apps for logging, but the data is unstructured text, and the phone app must parse it.

• Custom GATT service. The monitor defines its own characteristics, such as status, event log and control, each with its own access permissions, and the phone is notified when a value changes. More design work, but structured, efficient, and able to require a paired and encrypted connection before any control write.

• Standard Bluetooth SIG profiles. No adopted profile describes a security monitor, so a standard profile would only fit loosely.

Table 3.9: Bluetooth Approach Comparison

Criterion	Advertising broadcast	Nordic UART Service	Custom GATT service
Two-way communication	No	Yes	Yes
Structured data	Minimal	No (text stream)	Yes
Per-item access control (e.g., pairing required to write)	No	All or nothing	Yes

Phone app needed	Generic scanner	Generic UART app	Generic GATT app (e.g., nRF Connect), custom app optional
Effort	Low	Low	Moderate

3.1.7.7 Firmware Update and Boot Integrity

How new firmware reaches the MCU affects both development speed and security. Three approaches were investigated:

• Debug-port programming only. Firmware is written through SWD with the J-Link probe. There is no bootloader to write or maintain, and nothing can be loaded without physical access and a probe. This is the simplest option, and it is how every development build will be loaded.

• A signed-image bootloader. A small bootloader runs first, checks a cryptographic signature on the application image, and starts it only if the signature matches a key built into the bootloader. The nRF Connect SDK includes MCUboot for this purpose. It adds a few tens of kilobytes of flash and a key-management step, but an image changed by anyone without the key will not run.

• Over-the-air (BLE) updates. Built on top of a bootloader, this allows updates from a phone. It is convenient, but it adds an attack surface to the monitor and is only meaningful once the BLE stretch goal exists.

The research conclusion is that SWD programming is enough for the prototype. A signed-image bootloader is the right step for a deployed monitor, and it combines with debug-port locking (Section 3.1.2.6) so that the firmware can be changed neither through the debug port nor through the bootloader. Over-the-air update is not planned.

3.1.7.8 Verification and Debugging Approach

Firmware that controls a security response must be testable before the hardware exists, and observable once it does. The options researched were:

• Host-based unit tests. Logic that does not touch hardware, such as the policy state machine, the profile record's CRC check and the register-map encoding, can be compiled and tested on a PC. Zephyr provides a test framework (ztest), a test runner (Twister) and a native simulation target for this, so the same source runs on the PC and the nRF52832.

• On-target logging. SEGGER Real-Time Transfer (RTT) carries log messages over the existing SWD connection without using a UART pin, and with little effect on timing.

• External measurement. A logic analyzer on the SPI, I²C, ALERT# and mode lines shows the real timing between the devices. Toggling a spare GPIO at key points in the code allows interrupt and response latencies to be measured directly, which is how the flag-to-reset latency in Section 7.2.8 will be verified.

• Hardware-in-the-loop tests. With the boards built, the ESP32-C6 can be scripted to change modes and to run anomalous workloads while the monitor's responses are logged. This supports the detection-rate and false-positive measurements in the test plan.

All four approaches are used. Host-based tests start in SD1, before the boards exist.

3.1.7.9 Buffering and Timing Budget

Hardware-triggered sampling (Section 3.1.7.4) writes samples into RAM buffers by DMA. The firmware must then move each full buffer to the FPGA over SPI before the next one fills. The buffer size sets a trade-off:

• small buffers give lower latency, because a sample reaches the FPGA sooner, but cause more interrupts per second;

• large buffers use more RAM and add latency, but cause fewer interrupts.

At 200 kSPS a 256-sample buffer fills every 1.28 ms, giving about 780 buffer interrupts per second, which is a light load for a 64 MHz Cortex-M4F. It adds at most about 1.3 ms of delay between a current change and its arrival at the FPGA, almost three orders of magnitude below the 1 s detection-latency specification. Double buffering, where the ADC fills one buffer while the previous one is sent, keeps the stream continuous. The 16-bit SPI frame for each sample takes 2 µs at an 8 MHz SPI clock. Sending 200,000 frames per second therefore occupies about 40% of the SPI link, which leaves margin for the sequence counter and occasional retransmission. The final buffer size will be set during bring-up, but these figures show the approach has ample margin.

3.1.8 Clock Source

The FPGA needs a clock to process samples and compare measured current against a golden profile. Clock research considers frequency stability, jitter, output voltage, startup behavior, power consumption, and supporting circuitry.

3.1.8.1 Quartz Crystal with an External Oscillator

A quartz crystal is a passive resonator that establishes the frequency of an oscillator circuit. It requires an active circuit to sustain oscillation and produce a usable clock signal. Its small package can help reduce board area, but the complete circuit also needs supporting components. Important specifications include initial frequency tolerance, temperature stability, load capacitance, and drive level. The circuit must account for board capacitance and reliable startup, which adds design and verification work.

3.1.8.2 Packaged Quartz Oscillator

A packaged quartz oscillator combines the crystal and active circuitry in one component. It provides a clock output when supplied with power, reducing the need for a separate crystal driver. The device still requires supply decoupling and the correct enable connection. Its output voltage, load rating, and startup time

must match the receiving circuit. Compared with a passive crystal, this approach introduces a supply-current requirement and may use a larger package.

3.1.8.3 MEMS Oscillator

A MEMS oscillator uses a mechanical resonator fabricated in silicon with supporting circuitry to generate a digital clock. These devices are available with different frequencies, supply voltages, package sizes, and stability grades. Evaluation includes checking jitter, startup behavior, output loading, and enable functions. Although a MEMS device may resemble a quartz oscillator in size, its pin functions and electrical characteristics still need review. The relevant tradeoff is whether it meets the timing requirements within the available power and board area.

Table 3.10: Clock Technology Comparison

Criterion	Crystal with external circuit	Packaged quartz oscillator	MEMs oscillator
Complete clock output from one device	No	Yes	Yes
External oscillator design	Required	Not Required	Not Required
Main electrical checks	Load, drive level, startup, output conditioning	Supply, output levels, load, jitter	Supply, output levels, load, jitter
Board integration	Crystal plus active circuit and support parts	Module, decoupling, enable connection	Module, decoupling, enable connection
Main design consideration	Oscillator circuit development	Output compatibility and power	Output timing and configuration

3.2 Part Selection

3.2.1 Microcontroller Part Selection

Five wireless and general-purpose MCUs were compared against the criteria in Section 3.1.2.1 (Table 3.11) [4]–[8].

Table 3.11: Microcontroller Part Comparison

Parameter	Nordic nRF52832	Nordic nRF52840	Espressif ESP32-C6	ST STM32G431	Raspberry Pi RP2040

Core / clock	Arm Cortex-M4F, 64 MHz	Arm Cortex-M4F, 64 MHz	RISC-V, 160 MHz (+ LP core)	Arm Cortex-M4F, 170 MHz	2 × Arm Cortex-M0+, 133 MHz (200 MHz at 1.15 V core)
Flash / RAM	512 KB / 64 KB	1 MB / 256 KB	External or in-package flash / 512 KB	Up to 128 KB / 32 KB	External QSPI / 264 KB
ADC	12-bit SAR, 200 kSPS, 8 ch., programmable gain	12-bit SAR, 200 kSPS, 8 ch.	12-bit SAR, 7 ch.	2 × 12-bit, 4 MSPS	12-bit, 500 kSPS, 4 ch.
Radio	BLE 5 (1M/2M PHY)	BLE 5 incl. Coded PHY, 802.15.4, USB	Wi-Fi 6, BLE 5, 802.15.4	None	None
Hardware event routing	PPI (timer → ADC, no CPU)	PPI	GDMA / ETM	DMA + timer triggers	DMA + PIO
Serial interfaces	3 × SPI, 2 × TWI (shared IDs with SPI0/1), 1 × UART	4 × SPIM / 3 × SPIS, 2 × TWI, 2 × UART, USB, QSPI	SPI, I²C, UART	3 × SPI, 3 × I²C, 4 × USART, FDCAN, USB FS	2 × SPI, 2 × I²C, PIO
Supply range	1.7–3.6 V	1.7–5.5 V	3.0–3.6 V	1.71–3.6 V	1.8–3.3 V I/O, 1.1 V core
Hand-solderable form	Castellated module (e.g., Raytac MDBT42Q)	Castellated module (e.g., Raytac MDBT50Q)	Castellated module (ESP32-C6-WROOM-1)	LQFP-32 to LQFP-100	QFN-56 only
Team hardware on hand	Dev kit with on-board J-Link	No	Yes (it is the target)	No	No
Key limitation for NG PULSE	ADC limited to 200 kSPS	More cost and capacity than needed	Same device as the monitored target	No radio	No radio, ADC linearity, QFN only

Discussion.

• nRF52840. It is a superset of the nRF52832 with more memory, USB and extra radio protocols. NG PULSE needs none of these. Its bare package is a 0.5

mm-pitch aQFN-73 that cannot be hand-soldered, so it would also be used as a module, at higher cost.

• ESP32-C6. It has the fastest core and the most connectivity, and it is cheap as a module. It was rejected because it is the same part as the monitored target. Using it would mean the monitor shares its silicon, ROM bootloader and toolchain with the device it watches. A flaw or fault-injection technique that defeats the target could then defeat the monitor too. Keeping the two architectures different is a deliberate design choice.

• STM32G431. It is the strongest technical alternative. Its 4 MSPS ADCs are twenty times faster than the nRF52832's, and its on-chip op-amps and comparators could simplify the current-sense board. It also comes in hand-solderable LQFP. It was not selected because it has no radio: the BLE stretch goal would need a second wireless module, more firmware and board area. It is the documented fallback if more ADC bandwidth turns out to be needed (Section 7.2.4).

• RP2040. It is inexpensive and well documented, with a flexible PIO block. It was rejected because it has no radio, is offered only in QFN-56, and its ADC is known to have differential non-linearity at several codes (erratum RP2040-E11), which would distort small current changes [8].

Selection: Nordic nRF52832. The nRF52832 is the only candidate that meets every requirement with one component.

• Its integrated 12-bit SAADC has a programmable-gain input stage.

• The PPI system lets a hardware timer trigger conversions and EasyDMA move results to RAM with no CPU involvement, so sample timing is deterministic.

• It has the required I²C, SPI and GPIO-interrupt peripherals.

• Its BLE 5 radio covers the stretch goal.

• It has an on-chip comparator (COMP) that could later provide a hardware trip layer independent of the FPGA path.

• A team member already owns its development kit.

Its main limitation is the 200 kSPS sample rate. That is adequate for the ≤ 1 s detection-latency requirement (Section 7.2.8) but limits how much of the high-frequency current signature can be captured. Golden-profile characterization (Section 7.2.9) will show whether that bandwidth is enough to separate the four ESP32-C6 operating modes.

3.2.1.1 MCU Package and Module Selection

The nRF52832 is offered only in a 6 × 6 mm QFN-48 with 0.5 mm pitch and in a wafer-level chip-scale package [4]. No leaded version exists, and none is offered for any comparable BLE SoC. Three ways to put it on the NG PULSE board were compared (Table 3.12).

Table 3.12: nRF52832 Mounting Options

Option	Assembly difficulty	RF design required	Certification	Approx. cost	Notes
Bare QFN-48	Hot air with flux; exposed pad hard to inspect	Yes: antenna, matching network, 32 MHz and 32.768 kHz crystals	None (board would need its own)	~$3–4 + passives	Highest risk
Castellated module (Raytac MDBT42Q)	Hand-solderable edge pads	None: antenna, matching and crystals inside	FCC/IC/CE pre-certified	~$8	Selected
Breakout or dev board on headers	Trivial	None	Board vendor	~$15–40	Bulky; header wiring adds noise near the analog path

Other castellated nRF52832 modules evaluated were the Fanstel BT832 and the u-blox NINA-B112. They are functionally equivalent. The Raytac module was chosen because it is widely stocked and documented and is used on popular hobbyist boards [9].

Correction to the Rev 1 schematic and BOM: they name the module "MDBT40". The nRF52832 module is the Raytac MDBT42Q; the MDBT40 is the older nRF51822 module (Cortex-M0, 10-bit ADC). The Rev 1 documents are to be updated to MDBT42Q before parts are ordered.

3.2.1.2 SAADC Configuration for the Current-Sense Input

With the part selected, the integrated ADC considerations in Section 3.1.2.3 become concrete settings for the nRF52832's successive-approximation ADC (SAADC) [4]:

• Resolution and mode: 12-bit, single-ended, one channel connected to the current-sense amplifier output.

• Reference and gain: the internal 0.6 V reference with a gain of 1/6, giving a 0–3.6 V input range (limited in practice by the 3.3 V supply). The internal reference was preferred over the supply-relative VDD/4 reference, because a reading tied to the supply would shift whenever the 3.3 V rail moved with load. That would put an artificial current change into the golden profile. With this range one code is about 0.88 mV.

• Acquisition time: 3 µs, the shortest setting, which is needed to reach 200 kSPS. The SAADC specification limits the source resistance to 10 kΩ at that setting, so the amplifier output must drive the pin directly, or through a small series resistor

with a filter capacitor at the pin. The current-sense board's output stage will be checked against this limit.

• Oversampling: off at the full rate. The FPGA's 16-sample window provides the averaging (Section 7.1.3). Hardware oversampling remains available if a lower rate with less noise turns out to be better for some operating modes.

• Limit events: the SAADC can raise a hardware event when a sample crosses a programmable upper or lower limit, with no CPU involvement. This gives a coarse over-current trip that is independent of the FPGA path. It is noted as a possible extra protection layer for the shutdown stretch goal.

These settings are proposals for bring-up. Each will be confirmed against the final amplifier gain and output range, which are still open on the current-sense board.

3.2.1.3 Selection Risks and Fallback

The main risk of this selection is bandwidth. At 200 kSPS the SAADC captures current content only up to about 100 kHz. Golden-profile characterization (Section 7.2.9) will show whether that is enough to separate the ESP32-C6's operating modes. If it is not, there are two fallbacks:

• the STM32G431, whose 4 MSPS ADCs give twenty times the bandwidth. It would cost the integrated BLE radio, so the Bluetooth stretch goal would need a separate module;

• an external high-speed SPI ADC read directly by the FPGA. The nRF52832 would then keep its configuration, policy and BLE roles, but would no longer sit in the sample path.

The second fallback changes only the FPGA's input stage and the current-sense board, and leaves the firmware architecture of Chapter 7 largely intact. It is therefore preferred. A second, smaller risk is supply: nRF52832 modules are widely stocked, but the team will order spares, because a castellated module that is damaged during hand-soldering cannot easily be reworked.

3.2.1.4 Relationship to the ASIC Track

Northrop Grumman's ASIC request excludes the ADC, I²C and BLE from the ASIC's required scope (Section 7.4). Those are exactly the functions this selection places in the nRF52832. The FPGA and ASIC therefore share the same purely digital detection logic, while the MCU keeps everything analog, wireless or stateful. That division is what allows one RTL to serve both tracks.

3.2.2 FPGA Part Selection

3.2.2.1 FPGA Package Constraint and Physical Implementation

Selecting the FPGA is an important consideration because the NG PULSE project intends to develop a custom PCB for the physical prototype. The selected FPGA must be compatible with the team’s capability for assembly and provide sufficient connections for the subsystem. Leaded packages were favored over BGA packages due to the BGA connections not being directly visible after

assembly, and that installation of these requires certain aspects that are not available to us.

Leaded quad flat packages, including low-profile flat packages (LQFP), provide exposed leads around the perimeter of the integrated circuit. These leads allow for visual inspection of the solder joints and are generally more accessible. This is an important consideration for development of the PCB.

This requirement influenced the selection of the Lattice iCE40HX4K-TQ144. The selected device is available in a 144-pin thin quad flat package with a nominal package outline of 20mm x 20mm and a 0.5 mm lead pitch [DATASHEET]. The package provides 107 general-purpose programmable I/O pins for the FPGA to connect to the external components of the overall system [DATASHEET]

The package dimensions also allow for the use of existing PCB library resources as a basis for the board layout. The team identified an existing Xilinx TQG144 footprint from Ultra Librarian that may provide a useful reference for the physical layout. However, the footprint cannot be assumed to be electrically or mechanically interchangeable just because both devices use a 144-pin quad flat package.

Before PCB fabrication, the footprint must be verified against the Lattice mechanical package drawing. This includes confirming dimensions, lead spacing, pad dimensions, pin numbers, orientation, and recommended PCB land pattern. The electrical pin assignments must also be recreated according to the Lattice iCE40HX4K-TQ144 pinout. The Xilinx footprint may serve as a starting point, but it must be validated before use.

The project also considered the Xilinx Artix-7 FPGA used on the Digilent BASYS 3 development boards. While the BASYS3 provides a convenient environment for digital logic development and simulation, the Artix-7 device is not the selected FPGA for the NG PULSE PCB. The BGA configuration does not satisfy the team's leaded-package requirement. Additionally, since the BASYS 3 uses an AMD/Xilinx FPGA, it cannot directly implement or program anything to the Lattice iCE40 selection.

With this, the iCE40HX4K-TQ144 was selected as it satisfies the project's package requirements. It also provides ample resources to handle the logic, memory, and I/Os for the planned architecture.

3.2.2.2 Selected FPGA: Lattice iCE40HX4K-TQ144

The Lattice iCE40HX4K-TQ144 belongs to the iCE40 LP/HX family and provides configurable logic resources, embedded RAM, programmable I/O, and integrated clock management capabilities. It also contains 3,520 logic cells and 20 embedded RAM4K blocks, providing a total of 80 Kbits of memory. There are also 107 programmable I/O pins and two phase-locked loops (PLLs) for clock management [DATASHEET].

Table 3.13: Selected FPGA Technical Specifications

Specification	iCE40HX4K-TQ144
Manufacturer	Lattice Semiconductor
FPGA Family	iCE40HX
Device	iCE40HX4K
Package	TQFP-144
Dimensions	20mm x 20mm
Lead pitch	0.5mm
Logic Cells	3,520
Embedded RAM Blocks	20
Total Embedded RAM	80 Kbits
Phase-locked loops	2
General-Purpose programmable I/O	107
Configuration architecture	SRAM-based, with supported configuration options

Source: Lattice Semiconductor iCE40 LP/HX Family Datasheet [DATASHEET]

The device's embedded memory is important to the NG PULSE because the two ping-pong buffers require 24,576 bits of memory. This is equivalent to roughly 31% of the available memory. So additional memory for potential storage requirements, like feature values and reference information, is available.

The 3,520 logic cells will provide the resources for the buffer controller, sample management, calculations, feature extraction, comparison logic and alert generation. The advanced four-profile reference bank and two-tier notification system will also be considered when evaluating the available logic resources.

The 107 programmable I/O pins provide flexibility regarding connecting the FPGA to external components. These will support the sample interface, communication signals, clock and reset inputs, configuration signals, and status outputs. The final pin assignments will be determined after the PCB and interface requirements have been established.

Overall, the iCE40HX4K-TQ144 provides a suitable balance of programmable logic, embedded memory, package accessibility, and development flexibility for the proposed NG PULSE architecture.

3.2.2.3 FPGA Development Environment and Toolchain

The FPGA development environment must support Verilog design, RTL simulation, synthesis, implementation, and verification. Because the selected FPGA is manufactured by Lattice Semiconductor, the environment must be compatible with the iCE40HX4K architecture and the TQ144 package.

Currently, AMD Vivado Design Suite 2024.2 is the available environment being used for preliminary Verilog development. Vivado provides HDL editing, RTL simulation, synthesis, and implementation so it can be used to write and simulate the project's Verilog modules during the initial development phase. However,

Vivado does not provide all of this for the Lattice iCE40HX4K so its use is strictly limited to preliminary RTL development and simulation.

A compatible development environment will be required when the project advances toward physical implementation. Potential options include Yosys, nextpnr, and Project Ice Storm. Yosys provides Verilog synthesis capabilities, while nextpnr performs FPGA placement and routing. Project IceStorm provides device support and tools for the iCE40 FPGA family. Together these tools support a Verilog-to-bitstream development process for supported iCE40 devices [Yosys] [Yosys2] [IceSTorm].Open-source software provides an alternative to relying on proprietary software. It also helps in maintaining compatibility with the ASIC-oriented development by supporting portable Verilog.

The project will use a staged development approach. Vivado will be used for preliminary RTL simulation, while the final synthesis and implementation will be handled with Lattice compatible software. The same RTL will also be used with Cadence Genus for the ASIC-oriented portion.

3.2.3 Microcontroller Firmware Selection

Language: C. It is native to every Nordic SDK, deterministic and compact, and it is the language the team knows best (Table 3.5).

Program structure: RTOS threads, with sampling done in hardware. Attack response must never wait behind Bluetooth or logging work, which rules out a super-loop. An RTOS gives the policy logic guaranteed priority and coexists with the BLE stack by design (Table 3.6).

ADC input: TIMER → PPI → SAADC with EasyDMA. Only hardware triggering gives evenly spaced samples at the full 200 kSPS without loading the CPU, and it is unaffected by Bluetooth activity (Table 3.7).

Storage: wear-levelled NVS/settings storage, holding one versioned record with a CRC. Profiles can be updated after re-characterization without reflashing, and power-loss protection is built in. Writes are allowed only in a protected characterization mode, so the profile stays immutable during monitoring (Table 3.8).

Bluetooth (stretch goal): a custom GATT service (status, event log, protected control). Writes require an encrypted, bonded connection. The Nordic UART Service may be added during development for bench logging (Table 3.9).

Development platform: compared in Table 3.14.

Table 3.14: MCU Firmware Platform Comparison

Criterion	nRF5 SDK	nRF Connect SDK (Zephyr)	Arduino (Adafruit core)	Bare-metal C (nrfx)	Embedded Rust (Embassy)
Vendor status	Maintenance mode	Actively developed	Community	Driver layer maintained	Community

Direct SAADC/PPI/ TIMER/SPIM access	Yes	Yes (nrfx)	Hidden	Yes	Yes (HAL)
BLE stack	SoftDevice S132	Zephyr host + SoftDevice Controller	Bluefruit wrapper	None (integrate yourself)	nrf-softdevice crate
Non-volatile storage	FDS	Settings / NVS	Limited	Write it yourself	Crates
Debugging with J-Link	Yes	Yes (VS Code)	Limited	Yes	Yes (probe-rs)
Learning curve	Moderate	Steep at first	Low	Moderate	Steep
Support through April 2027	Bug fixes only	Yes	Uncertain	Yes	Uncertain

Selection: nRF Connect SDK (Zephyr RTOS), written in C. It is Nordic's supported path for new nRF52832 designs, so it carries no deprecation risk during SD2.

• Its nrfx drivers give the register-level control needed to chain TIMER → PPI → SAADC → EasyDMA for jitter-free sampling.

• Zephyr's threads, BLE host and settings/NVS subsystems cover the policy logic, telemetry and golden-profile storage without custom infrastructure.

• Development uses VS Code with the nRF Connect extension.

• Flashing and debugging use the J-Link on the team's nRF52 development kit, connected to the board's SWD header (J5) through the kit's debug-out connector.

• nRF Connect for Desktop (already installed by a team member) provides the Toolchain Manager used to install the SDK, the Programmer app for flashing, and the Bluetooth Low Energy app for testing the GATT service when the BLE stretch goal is built.

The extra up-front cost of Kconfig and devicetree configuration is accepted in exchange for long-term support. The nRF5 SDK, which has more nRF52832 examples, is the fallback if bring-up of the nRF Connect SDK stalls.

3.2.3.1 Update, Security and Verification Selections

The firmware research in Sections 3.1.7.7 to 3.1.7.9 leads to the following further selections:

• Programming: SWD only for the prototype. Development builds are flashed with the J-Link probe. MCUboot with signed images is planned for a deployed configuration and will be evaluated in SD2. Over-the-air updates are not planned.

• Debug lock: documented, not enabled during development. APPROTECT stays open while the firmware changes. Enabling it, together with a signed-image bootloader, is the deployment step, and it will be carried out and documented once the firmware is frozen.

• Verification: host-based tests, RTT logging and logic-analyzer timing. The policy state machine and the profile-record handling are tested on a PC with Zephyr's ztest and Twister from SD1 onward. On-target behaviour is observed through RTT, and timing is measured on GPIO test points with a logic analyzer.

• Sample buffering: double-buffered DMA. Buffers of about 256 samples are used, giving roughly 1.3 ms of added latency at 200 kSPS. The final size will be tuned during bring-up.

3.2.3.2 Configuration Management

Builds must be reproducible, because the golden profiles are tied to a firmware and FPGA configuration. The firmware is pinned to one nRF Connect SDK release through the west manifest, and that release is recorded with every characterization run. The firmware, the RTL and the testbenches are kept in the team's Git repository. Each golden-profile record carries a version number (Section 7.2.9). A profile measured with one firmware and RTL build can then always be matched to the code that produced it, and a mismatch between a stored profile and the running build can be detected at boot.

3.2.3.3 Summary of the Firmware Selection by Goal Tier

The selected platform covers all three goal tiers without a change of toolchain:

• basic goal: deterministic acquisition (TIMER → PPI → SAADC → EasyDMA), SPI transfer to the FPGA, I²C configuration, flag handling and protected profile storage;

• advanced goal: mode-line inputs and per-mode profile selection;

• stretch goal: the BLE GATT service and the reset-and-power-kill shutdown response.

All of these are written in C on the nRF Connect SDK and use only its supported subsystems. This keeps the stretch features from forcing a platform change late in SD2.

3.2.4 Clock Source Selection

Three clock components were compared for the Lattice iCE40HX4K-TQ144: the Abracon ABM8-12.000MHZ-B2-T crystal and the Abracon ASV-12.000MHZ-EJ-T and ASE-12.000MHZ-L-C-T oscillators. All three support a nominal 12 MHz reference, but they require different supporting circuitry. The ABM8 is a passive resonator, while the ASV and ASE include active circuitry and provide digital clock outputs.

Table 3.15: Clock Component Comparison

Parameter	Abracon ABM8-12.000MHZ-B2-T	Abracon ASV-12.000MHZ-EJ-T	Abracon ASE-12.000MHZ-L-C-T
Nominal frequency	12 MHz	12 MHz	12 MHz
Supply	Passive crystal; external circuit required	3.3 V nominal	3.3 V nominal
Output	Resonator; no digital clock output	HCMOS	CMOS
Frequency Specification	±20 ppm initial tolerance; ±50 ppm temperature stability	±20 ppm stability	±50 ppm stability
Operating Temperature	−20 to +70 °C	−20 to +70 °C	−40 to +85 °C
Package Dimensions	3.2 × 2.5 × 0.8 mm	7.0 × 5.08 × 1.8 mm	3.2 × 2.5 × 1.2 mm
Main Tradeoff	Needs active oscillator circuitry	Larger package; powered module	Powered module; different pin functions

The ABM8 has a 3.2 × 2.5 × 0.8 mm package and a standard load capacitance of 18 pF. In the B2 ordering code, B specifies an operating range of −20 to +70 °C and 2 specifies ±20 ppm initial frequency tolerance at 25 °C. The standard temperature stability is a separate ±50 ppm specification. These values should not be combined into a claim of ±20 ppm accuracy under all conditions. At 12 MHz, the initial ±20 ppm tolerance corresponds to ±240 Hz.

The ASV-12.000MHZ-EJ-T provides a complete 12 MHz clock from a nominal 3.3 V supply. Its HCMOS output avoids the need to design a separate crystal driver. The 7.0 × 5.08 × 1.8 mm package is larger than the ABM8, and the supply budget would need to include the oscillator current.

The ASE-12.000MHZ-L-C-T is a smaller complete oscillator with a 3.2 × 2.5 × 1.2 mm package, a 3.3 V supply, and a CMOS output. Its specified stability is ±50 ppm. Although its footprint dimensions are close to those of the ABM8, the parts have different pin functions and are not interchangeable. Selecting the ASE would require revising the clock circuit and checking the footprint.

The ABM8-12.000MHZ-B2-T was selected because the team wanted an accurate reference for the FPGA clock circuit. Its initial frequency tolerance is ±20 ppm at 25 °C, equivalent to ±240 Hz at 12 MHz [C3]. This supports predictable timing for operations controlled by the FPGA clock. Its compact package also fits the board layout. The crystal's initial tolerance is separate from

its temperature stability, and the completed circuit will still require appropriate oscillator circuitry, load capacitance, and startup checks.

[TO DO: Technology comparison (I²C, PMBus, SPI, UART, GPIO, BLE), then selection.]

3.3 Software

3.3.1 PCB Design Software

PCB design tools support schematic capture, component libraries, board routing, and manufacturing outputs. For NG PULSE, research also considers how each workflow supports physical-fit checks and keeps electrical and mechanical information consistent.

3.3.1.1 PCB Design with Separate Mechanical CAD

A dedicated PCB application handles the schematic, footprints, routing, and fabrication files, while a separate mechanical tool handles the enclosure or mounting arrangement. A 3D viewer or exported board model can support checks of connector access and component clearance. The main limitation is keeping the models consistent after changes. Board outlines, mounting holes, and component positions must be transferred and reviewed when either design is updated.

3.3.1.2 Integrated Electronics and Mechanical Tools

An integrated environment connects the schematic, PCB layout, and mechanical model within one platform. This can simplify checks of connector placement, component height, and mounting locations because the electrical and physical views are related. Component libraries still need accurate symbols, footprints, and 3D models. Integration reduces the work involved in exchanging geometry, but it does not replace checking pin assignments and package dimensions against component documentation.

3.3.1.3 Connected Electrical and Mechanical Tools

This workflow uses separate electrical and mechanical applications with tools for exchanging board geometry and design changes. It allows each designer to work in the application suited to their task while sharing information needed for physical-fit checks. The main consideration is how changes are transferred and approved. Board dimensions, mounting positions, and component models must remain consistent, and the workflow may require additional software access or exchange setup.

Table 3.16: PCB Design Workflow Comparison

Criterion	Separate PCB and mechanical tools	Integrated Environment	Connected Electrical and mechanical tools
Electrical design work	Dedicated PCB application	Integrated electronics workspace	Dedicated PCB application
Mechanical coordination	Export and review between programs	Related models in one platform	Exchange changes between programs
Main project consideration	Keep exported models current	Verify linked libraries and models	Maintain exchange setup and access
Workflow consideration	Geometry export and model updates	Shared electrical and mechanical model	Exchange of changes between tools

3.3.2 PCB Software Selection

Autodesk Fusion 360, KiCad, and Altium Designer were compared for schematic capture, PCB layout, component libraries, and manufacturing outputs. Each can support the monitoring board. The comparison considers these capabilities alongside the work needed to maintain the team's existing design files and review the physical arrangement of the components.

3.3.2.1 Autodesk Fusion 360

Fusion combines schematic capture, PCB layout, component libraries, and a related 3D board model. Its manufacturing tools produce files such as Gerbers, while the 3D view helps check connector access and component fit. The team already uses Fusion, so continuing with it preserves the existing design files and libraries. Footprints and pin mappings still need datasheet checks. Available electronics features depend on the license, so the team must confirm that its account supports the intended board size, layer count, and shared access.

3.3.2.2 KiCad

KiCad is an open-source suite with schematic capture, PCB layout, manufacturing outputs, and a 3D viewer. It offers a practical alternative without a commercial software subscription. However, moving the existing Fusion design would require checking imported symbols, footprints, and net connections before fabrication.

3.3.2.3 Altium Designer

Altium Designer supports schematic capture, PCB layout, design-rule management, and manufacturing outputs. Its features could support the monitoring board, but the team would need suitable software access and would have to transfer or rebuild the existing design and libraries.

Table 3.17: PCB Software Comparison

Criterion	Autodesk Fusion	KiCad	Altium Designer
Schematic and PCB layout	Yes	Yes	Yes
3D board review	Integrated with mechanical design	Integrated viewer and model export	Native 3D and mechanical collaboration
Custom component libraries	Yes	Yes	Yes
Electrical and layout checks	ERC and DRC	ERC and DRC	Electrical checks and PCB rules
Manufacturing output	Gerber and other CAM outputs	Gerber and other fabrication outputs	Gerber and other fabrication outputs
Access consideration	License with UCF	Open source	Verify institutional or commercial access

We decided to go with Fusion because we are most comfortable using this software and have most if not all of the features alternatives have. We can maintain a shared set of verified footprints, including the planned 0805 general passives, while checking the larger packages separately. The 3D model will help us review physical fit, and the manufacturing outputs will provide the files needed for fabrication. Electrical-rule and design-rule checks will be used alongside datasheet review; they cannot determine whether a selected shunt, amplifier, or regulator meets the circuit's performance requirements.

3.3.3 Digital ASIC Design Platform

Digital design platforms provide different ways to implement and evaluate the detector logic. The research considers RTL reuse, technology-library requirements, verification effort, and the results each approach can provide. Table 3.C3 compares standard-cell, full-custom, and FPGA-only implementation.

3.3.3.1 RTL to Standard Cells

A standard-cell flow converts RTL into a netlist of gates and registers from a technology library. Physical-design tools then place the cells and route their connections. This approach supports evaluation of circuit area and timing using the library models and design constraints. The detector arithmetic can be described in RTL, but device-specific clock and memory functions require separate handling. Compatible technology files and verification tools are needed throughout the flow.

3.3.3.2 Full Custom Transistor Design

Full-custom design develops the circuit at the transistor level instead of relying mainly on existing standard cells. This provides direct control over individual circuits and can support specialized analog or digital functions. It also requires transistor models, circuit simulation, layout, and physical verification. For a current-profile detector, that work would extend beyond the arithmetic and comparison logic, increasing the design effort needed to produce and verify an implementation.

3.3.3.3 FPGA Only Implementation

An FPGA-only approach maps the detector logic into a programmable device and allows testing with samples from physical hardware. It supports repeated changes during development and provides resource-use and timing reports for that device. These reports do not establish the area or timing of the same logic in an ASIC technology. An FPGA can demonstrate detector behavior, but a separate ASIC flow is needed to evaluate a standard-cell implementation.

Table 3.18: Digital Design Platform Comparison

Criterion	RTL to Standard cells	Full custom transistor design	FPGA only
Starting description	Synthesizable RTL	Transistor circuit	Synthesizable RTL
Reuse of detector RTL	Direct, with technology-specific changes	Requires substantial redesign	Direct, with FPGA-specific support
Implementation result	Mapped cells and optional physical layout	Custom transistor layout	FPGA configuration
ASIC library required	Yes	Device models and process rules	No
Evaluation focus	Mapped logic, area, and timing	Transistor behavior and physical layout	Operation on programmable hardware

3.3.4 ASIC Software Selection

Cadence is the selected environment because the DRACO lab provides the software and resources for our Northrop-sponsored project. This avoids arranging a separate commercial license and gives the team an available environment for the ASIC work. Genus will synthesize the detector RTL into a technology-mapped netlist, while Innovus supports placement, clock-tree synthesis, and routing if physical implementation is completed within the project scope.

The NGRAD repository provides a useful example from another Northrop-sponsored UCF Senior Design project. It documents a Cadence RTL-to-GDSII flow and includes synthesis scripts, timing constraints, physical-design scripts, and a gate-level simulation environment. These files show how the stages connect: synthesis produces a mapped netlist and reports, simulation checks the netlist with cell models, and physical-design scripts organize floorplanning, power planning, clock-tree synthesis, and routing. This gives us a concrete reference for organizing our work.

3.3.4.1 Synopsys Design Compiler and IC Compiler II

Design Compiler supports RTL synthesis and optimization for timing, area, and power. IC Compiler II provides physical implementation functions such as placement, clock-tree synthesis, and routing. Together, they provide a technically suitable alternative for the same type of standard-cell design. Synopsys provides a comparable commercial toolchain, but access would require a suitable license.

3.3.4.2 Yosys and OpenROAD

OpenROAD Flow Scripts combines tools for an RTL-to-GDSII flow, including Yosys for synthesis and OpenROAD for physical implementation. The flow requires a supported platform, RTL files, and timing constraints .It offers a way to explore a digital ASIC design without depending on the same commercial tool licenses. It still needs a compatible process design kit and standard-cell library, and its setup would have to be checked for the intended ASIC work.

Table 3.19: ASIC Design Flow Comparison

Criterion	Cadence flow	Synopsys flow	Open-source flow
RTL synthesis	Genus	Design Compiler	Yosys
Physical implementation	Innovus	IC Compiler II	OpenROAD
Technology requirement	Compatible library and process files	Compatible library and process files	Supported platform and libraries
Timing constraints	Required	Required	Required
Access	Provided through DRACO lab	Separate commercial license access needed	Open source; no commercial license fee
Project fit	DRACO resources and documented student workflow	Separate access and setup	Separate setup and technology checks

Cadence Genus and Innovus are selected because DRACO provides the resources for the Northrop-sponsored project . The NGRAD example adds a documented student workflow that can guide how we organize synthesis, verification, and physical-design files. Synopsys would require separate access, while an open-source flow would require its own compatible setup.

[TO DO: Comparison tables for the HDL and simulation tools (e.g., Verilog/SystemVerilog, Vivado/XSim, Icarus Verilog, Yosys/nextpnr/Project IceStorm), the ASIC flow (Cadence Genus/Innovus), MCU firmware toolchain, and PCB CAD (Fusion 360). At least 3 options each.]

Chapter 4 – Standards and Design Constraints

4.1 Standards

4.1.1 Printed Circuit Board Design and Current Capacity

The printed circuit board provides the electrical connections and mechanical support for the NG PULSE hardware. Its layout affects how current reaches the target, how much heat is generated in the conductors, and how accurately the sensing circuit measures the shunt voltage. IPC-2221C, Generic Standard on Printed Board Design, provides a general reference for board design. IPC-2152 provides more specific guidance on the relationship between conductor size, current, and acceptable temperature rise [16], [17].

These standards are relevant because the NG PULSE board contains both a power path and a sensitive measurement path. The power path carries the current supplied to the ESP32-C6. The measurement path carries the much smaller voltage developed across the shunt resistor. Although these circuits connect at the shunt, their routing requirements are different. The power conductors need enough cross-sectional area for the operating current, while the sensing connections need to avoid including unrelated voltage drops in the measurement.

For PCB conductors, current capacity depends on more than trace width. Copper thickness, nearby planes, board construction, and the surrounding temperature all affect how much heat a trace can dissipate. IPC-2152 is useful when selecting conductor dimensions for a specified current and temperature rise [17]. For our board, those dimensions will depend on the final operating range and the construction offered by the PCB manufacturer. A minimum trace width allowed by the manufacturer only indicates what can be fabricated. It does not establish the current rating of that trace.

The same consideration applies to the rest of the current path. A wide copper region can still connect to a narrow pad, an undersized connector, or a component with a lower current rating. The usable current range is therefore limited by the complete path through the assembly. This is particularly important if the proposed 20 A upper measurement limit is retained. The board, shunt,

connectors, and test wiring would all need to support the conditions used to demonstrate that range.

The shunt connections also affect measurement accuracy. Kelvin sensing uses separate connections to measure the voltage at the shunt terminals so that voltage drops in the load-current connections contribute less error. Texas Instruments discusses this effect in its current-sensing layout guidance [18]. As an example, an additional 1 mΩ of resistance included in the sensing path would represent 10% of a 10 mΩ shunt. The resulting error could change the stored golden profile even when the target's actual current remains the same.

For NG PULSE, the proposed layout will place the amplifier near the shunt and keep its input connections short. Clock lines, SPI signals, and switching-regulator nodes need adequate separation from the analog input. A continuous ground reference and short decoupling connections provide a starting point for controlling the return currents. These layout choices support the measurement circuit, but their effectiveness will still depend on the noise observed during hardware testing.

The location of the shunt determines which current is being measured. If the monitor's own supply current passes through the same shunt as the ESP32-C6, FPGA activity or MCU communication could appear in the target's profile. The schematic will therefore identify the loads downstream of the shunt. The same measurement arrangement is needed during both golden-profile capture and normal monitoring so that the two sets of data can be compared meaningfully.

Fusion 360 will be used to check the PCB against the selected manufacturer's fabrication rules. These checks help identify spacing violations and incomplete routing. Footprints, component orientation, and pin assignments also need to be compared with the selected parts before the layout is released. The PCB standards provide the design reference, while the manufacturer rules and later electrical measurements establish whether the proposed layout is suitable for this particular board.

4.1.2 I²C Configuration and Status Communication

NG PULSE requires a communication interface that allows the MCU to configure the FPGA and read its status. The current firmware design assigns this function to I²C. NXP's UM10204, I²C-Bus Specification and User Manual, defines the signaling, timing, and electrical behavior of the bus. I²C uses a serial data line, SDA, and a serial clock line, SCL, allowing multiple addressed devices to share the same connection [19].

In our system, the nRF52832 acts as the controller and the FPGA acts as the target. The MCU will use this connection to load golden-profile parameters, thresholds, and other settings before monitoring begins. It will also read status information from the FPGA. This arrangement gives the firmware access to the detector configuration without requiring a separate group of parallel data lines for each parameter.

I²C devices normally pull the bus low and release it to allow pull-up resistors to establish the high level. The resistor values therefore affect the signal rise time. A large resistance reduces the current drawn while a line is low, but it also takes longer to charge the capacitance of the connected devices and wiring. A smaller resistance produces a faster rising edge while increasing the current that a device must sink [19], [20].

The proposed interface operates in Fast-mode at up to 400 kbit/s. For this mode, the allowed rise time is 300 ns. The approximate relationship between pull-up resistance and bus capacitance is shown in Equation 4.1, where Rp is the pull-up resistance and Cb is the total bus capacitance [19], [20].

$$t_r = 0.8473 R_p C_b \quad (4.1)$$

For example, a 4.7 kΩ resistor and an assumed bus capacitance of 100 pF produce a calculated rise time of approximately 398 ns. Under those assumptions, the rise time would exceed the Fast-mode limit. Reducing the resistance to 3.3 kΩ gives approximately 280 ns. These values illustrate the design tradeoff; the final resistance will depend on the assembled bus capacitance, resistor tolerance, and the devices' low-level current capability. The proposed 4.7 kΩ value therefore needs to be checked on the actual connection.

The bandwidth of the interface also determines which information it can carry. At 200,000 samples per second, a stream of 12-bit ADC values contains 2.4 Mbit/s of sample data before any framing is added. This exceeds the capacity of a 400 kbit/s I²C connection. The current architecture consequently uses SPI to transfer samples and reserves I²C for configuration and status. Keeping these functions separate allows frequent sampling without requiring the configuration interface to carry the full measurement stream.

Reliable communication also depends on the way configuration changes are applied. The FPGA needs a complete set of valid parameters before it begins comparing samples. Reading the settings back after a write provides a way to check that the intended values were received. If communication stops during an update, the detector should retain a defined state rather than use a mixture of old and new parameters. The detailed register behavior belongs in the MCU and FPGA design sections, but the need for consistent configuration affects both implementations.

During integration, the bus will be checked for correct addressing, acknowledgements, and recovery after either device resets. Oscilloscope measurements can establish whether the electrical edges meet the timing requirements, while a logic analyzer can show the transferred addresses and data. The early project concept also mentions PMBus. The present proposal uses project-specific I²C registers, so PMBus compatibility is not assumed from the use of the same two-wire connection.

4.1.3 System Verilog and Portable Digital Design

The digital portion of NG PULSE uses the FPGA to evaluate current measurements and compare them with the stored golden profile. This requires logic for collecting samples, calculating values over an analysis window, and determining when an abnormal condition should produce an error flag. IEEE 1800-2023 defines the SystemVerilog language used for hardware description and verification. The standard includes behavioral, register-transfer, and gate-level modeling, along with testbench features such as assertions and coverage [21]. These capabilities are relevant because the detection logic will be simulated, implemented on an FPGA, and adapted to the digital ASIC flow.

Using a common hardware-description language helps maintain consistent behavior across these stages. However, the selected tools may support different language features, particularly during synthesis. The detector therefore needs to use constructs supported by the simulation tools, FPGA synthesis tools, and Cadence flow. For our design, the arithmetic and comparison logic will be kept separate from functions that depend on a particular device, such as clock generation, configuration, and physical input/output connections. This allows the same detection logic to be reused while those surrounding functions are adapted to each implementation.

Each implementation will still require its own checks. A simulation in the Xilinx development environment can help evaluate the intended behavior, but the design must also be built and checked for the selected iCE40 FPGA. The Cadence flow introduces another set of synthesis and implementation requirements. Keeping the common logic separate from device-specific functions makes these transitions easier to manage and helps identify whether a difference comes from the detector itself or its connection to the selected hardware.

The number of bits used in the calculations is one practical concern. A 12-bit unsigned ADC sample can represent values from 0 to 4,095. If the detector adds 1,024 samples to calculate a mean, the largest possible sum is 4,193,280. Representing this value requires 22 unsigned bits. An accumulator limited to 12 bits would overflow before the calculation was complete, producing an incorrect mean that could affect the detection result. The running-sum calculation also needs enough intermediate range to add a new sample and remove the oldest sample without losing information. Changes to the window length will require corresponding checks of the accumulator, sample index, and averaging operation.

The comparison with the golden profile requires similar attention. The measured mean may be either above or below the reference value. Subtracting a larger reference from a smaller unsigned value can produce a wrapped result instead of the intended negative difference. The logic therefore needs a defined way to calculate the magnitude of the deviation. The threshold comparison must also specify what happens when the deviation is exactly equal to the threshold. This decision affects the expected result in both simulation and hardware. Counters

used to identify repeated abnormal measurements must also avoid wrapping into a value that incorrectly clears an active condition.

Startup and operating-mode changes introduce additional requirements. The detector needs a valid golden profile and enough samples to fill its analysis window before normal comparisons begin. When the target changes workloads, samples from the previous mode may no longer represent the current operating condition. The window therefore needs a defined reset or reinitialization process. The same behavior will be checked in the ASIC testbench, even though that testbench supplies digital samples directly rather than receiving measurements from the physical ADC.

Verification will include normal inputs, maximum sample values, threshold crossings, missing samples, mode changes, and reset during operation. Prolonged abnormal inputs will also help check the persistence counters and error-flag behavior. RTL simulation evaluates these conditions using modeled inputs, while FPGA timing analysis checks whether the implemented logic meets its clock and interface requirements. The ASIC implementation adds checks associated with its technology libraries and physical design.

Recorded hardware samples will provide another way to evaluate the detector using signals produced by the actual measurement circuit. Applying the same sample data and expected results across the implementations will help confirm consistent digital behavior. Analog measurement accuracy will still need to be evaluated separately. The source revision, parameter values, test data, and tool settings used for each result will be recorded so that the team can repeat the test and investigate any differences.

4.1.4 USB Power and Connection Requirements

The prototype is planned to power NG PULSE from the same laptop USB-C connection used by the ESP32-C6. This keeps the normal demonstration setup simple and avoids adding another power adapter. Since our board adds a load to that connection, the USB Type-C Cable and Connector Specification applies to the design, [21].

The USB-C configuration-channel connections handle attachment detection, cable orientation, and the current capability advertised by the source [22]. The schematic needs to account for the connection to the laptop and the connection supplying the target. Our power budget must include the ESP32-C6, sensing circuit, MCU, FPGA, and regulator losses. Checking only the target’s current consumption would leave out the power needed to run the monitor.

The planned input is 5 V, which supplies the regulators producing the 3.3 V and 1.2 V rails. Higher voltages require compatible USB Power Delivery negotiation and hardware rated for the resulting voltage [22]. Any test that changes the target supply will also need to protect the monitoring circuit.

The proposed 20 A sensing range cannot be tested through the normal USB connection. Even a properly configured 5 A USB-C power path falls below that target. Testing the full range would require a separate supply, load, wiring, and protection rated for the higher current [22].

During integration, we will check both cable orientations and measure the supply voltage under load. If USB data passes through our board, the ESP32-C6 must remain accessible for programming and debugging. External power connections used during testing must also prevent one supply from feeding back into another.

4.1.5 Soldered Assembly and Electrostatic Discharge

The prototype will be assembled and handled repeatedly during development. IPC J-STD-001J provides requirements for producing soldered electrical and electronic assemblies, while IPC-A-610J provides acceptance criteria for inspecting those assemblies [24], [25]. These references are useful for NG PULSE because the board combines surface-mount parts, power connections, and measurement circuits whose performance can be affected by an incomplete or damaged joint.

A solder bridge at the FPGA could prevent configuration or connect two signals unintentionally. A poor connection near the shunt could produce an intermittent voltage change that resembles a change in target current. Assembly inspection therefore supports both basic functionality and measurement repeatability. Component orientation, joint condition, and possible shorts need to be checked before power is applied. Continuity checks and initial supply measurements provide additional information that visual inspection alone cannot establish.

ANSI/ESD S20.20-2021 addresses the development of an electrostatic-discharge control program [26]. ESD precautions are relevant while handling the MCU, FPGA, and analog components. The laboratory's approved grounding practices, work surfaces, and protective storage will be used during assembly and transport. Components can be damaged before an obvious functional failure appears, which would make later debugging more difficult.

The preference for larger passive packages and a leaded FPGA supports manual placement and inspection. It does not remove the need to examine fine-pitch joints or verify the complete power path. The project will document the assembly methods and checks actually performed. Referring to these standards provides a basis for workmanship decisions without implying that the prototype has received an IPC class certification or that the laboratory has been audited as an ESD control program.

4.1.6 Laboratory Safety and Electromagnetic Compatibility

IEC 61010-1 covers general safety requirements for electrical measurement, control, and laboratory equipment. IEC 61326-1 addresses electromagnetic emissions and immunity for equipment in these categories [27], [28]. Their scopes provide relevant design references for a laboratory current monitor.

Application to a future product would require a complete evaluation of its intended use, construction, and applicable particular requirements.

The immediate safety concern is the energy available in the power path and disturbance-test fixture. Low nominal voltage does not prevent hot conductors, damaged components, or high fault current. The prototype should use a current-limited supply, an accessible means of disconnecting power, protected test connections, and component ratings appropriate to the approved operating conditions. Anomaly detection must not be the only protection against a wiring short or a failed power component. The exact disturbance method and safe limits remain items for the team and laboratory supervisor to establish before testing.

Electromagnetic compatibility also affects the validity of the measurements. FPGA clocks, serial transfers, regulator activity, and optional Bluetooth transmissions can couple into the current-sense output. If these disturbances exceed the detector's noise margin, a false alarm may describe the monitor's own interference rather than a change in the ESP32 workload. The design should control return paths and cable placement, provide local supply decoupling, and compare measurements with optional radio activity enabled and disabled.

Initial verification should observe supply ripple and the analog output during worst-case digital activity. Cable movement, connection changes, and power cycling should produce either continued valid operation or a recognizable fault state. These engineering checks help locate weaknesses, but they do not substitute for formal IEC safety or EMC testing. The completed report should describe the tests performed and their limits without claiming qualification that was outside the project scope.

4.1.7 PCB Land Pattern Design

IPC-7352, Generic Guideline for Land Pattern Design, provides guidance for designing the pads used to attach components to a printed circuit board. It addresses pad dimensions, component tolerances, and spacing needed for soldering, inspection, and rework. This applies to NG PULSE because the footprints selected in Fusion 360 need to match the physical components we plan to assemble.

For our board, we will check each footprint against the package drawing and recommended layout in the component datasheet. This includes pad spacing, lead dimensions, and pin numbering. Our planned use of 0805 resistors and capacitors helps with handling during assembly, but the ICs and connectors require their own package-specific footprints. Selecting a part with the correct electrical value does not guarantee that it will fit the pads on the board.

We will also leave enough space around components to inspect solder joints and replace parts during testing. IPC-7352 includes guidance on component courtyards, which define the space reserved around a component, along with solder-mask clearance and land pattern tolerances. These considerations will guide our footprint review before the PCB is ordered. Checking them early should

reduce assembly problems and avoid a board revision caused by an incorrect footprint.

4.2 Design Constraints

4.2.1 Current Measurement and Power Constraints

The current-sensing circuit must produce enough signal for the ADC while keeping its effect on the target supply small. A shunt resistor converts current into a voltage that the amplifier can measure. Increasing the resistance increases this voltage, but it also increases the voltage lost before power reaches the target. The shunt additionally dissipates heat. These relationships are shown in Equation 4.2, where I is current and Rs is shunt resistance.

$V_s = IR_s$ and $P_s = I^2R_s$ (4.2)

The preliminary design information includes both 10 mΩ and 50 mΩ shunt values. Table 4.1 compares them at the lower and upper ends of the documented measurement range. The values are calculated examples that show how strongly the final shunt choice affects the circuit.

Shunt	At 0.5 A	At 20 A	Effect at 5 V input
10 mΩ	5 mV; 2.5 mW	0.20 V; 4 W	4% voltage drop at 20 A, before other losses.
50 mΩ	25 mV; 12.5 mW	1.00 V; 20 W	20% voltage drop at 20 A, before other losses.

Table 4.1: Calculated Shunt Voltage Drop and Power Dissipation

At low current, the larger shunt produces a signal that is easier to amplify. At 20 A, however, its calculated voltage drop and dissipation become substantial. The resistor's power rating, mounting arrangement, and surrounding copper would need to support that condition. The preference for 0805 general passive components cannot be applied to the shunt without checking its power requirements. A component that is convenient to assemble may still be unsuitable for the intended current.

Amplifier gain introduces a related tradeoff. For an approximately zero-offset circuit, the output voltage is the product of current, shunt resistance, and gain. With a 10 mΩ shunt, an illustrative 3.0 V output limit at 20 A allows a gain of about 15. At a gain of 150, the same output would be reached at only 2 A. A high

gain improves sensitivity to small changes but reduces the current that can be measured before the amplifier or ADC saturates. Separate measurement ranges may therefore be needed if both sensitive workload monitoring and a broad current range are retained.

The amplifier's input range is also important. In a high-side arrangement on a 5 V rail, both shunt terminals remain close to 5 V even though the voltage difference between them is small. The selected amplifier must accept this common-mode voltage at its actual supply voltage. The shunt, amplifier gain, and input limits need to be treated as one circuit when the final component values are selected.

The nRF52832 ADC supports 12-bit conversion. Its nominal measurement span depends on the selected reference and gain. With a 0.6 V internal reference and gain of 1/6, the nominal span is 3.6 V, although the analog input must remain within the device supply rails [29]. The calculated code width is approximately 0.879 mV. With the example 10 mΩ shunt and external gain of 15, this corresponds to about 5.86 mA per code. Small workload changes can therefore occupy only a few codes before noise and component error are considered.

This sensitivity matters for the Not-in-Use state, which may draw less than the documented 500 mA lower limit. Characterizing the actual ESP32-C6 currents will establish the useful measurement range. The 20 dB SNR target also needs a clear definition of the measured signal and noise. A large average current does not necessarily make a small change in workload easy to distinguish.

The supply rails must preserve these measurement conditions. The proposed 5 V input is converted to 3.3 V for the MCU and I/O and nominally 1.2 V for the FPGA core. Regulator heating and switching noise can affect nearby analog circuitry. Power measurements during operation will therefore be needed alongside the current-sense measurements, particularly when deciding the final gain, filtering, and layout.

4.2.2 Sampling Throughput and Timing Constraints

We plan to sample the target current at 200 kSPS, or once every 5 µs. This is the maximum rate supported by the nRF52832 ADC, but reaching it depends on the acquisition time and the circuit connected to the ADC input [29]. Our amplifier and filter need enough time to settle between measurements. We will test them together with the MCU to determine whether 200 kSPS is practical for the finished board.

At this rate, the Nyquist frequency is 100 kHz. Signals above that frequency can alias into the measured data, so the analog circuit needs filtering before the ADC. Choosing the filter cutoff will involve a tradeoff. We want to reduce noise without removing the changes in current that distinguish the ESP32-C6 workloads. Too much filtering could make different workloads look similar to the detector.

The MCU also needs to move the samples to the FPGA while continuing to collect new ones. We currently plan to use an 8 Mbit/s SPI connection and send each 12-bit sample in a 16-bit frame. At 200 kSPS, this produces 3.2 Mbit/s of

sample data, which uses 40% of the available SPI bandwidth before accounting for gaps between transfers [30].

A 256-sample buffer would hold 512 bytes and fill in 1.28 ms. Sending those bytes at 8 Mbit/s takes 0.512 ms without any transfer overhead. These numbers suggest that SPI can keep up, but we still need to check continuous sampling in firmware. Using two buffers would let the MCU fill one while transmitting the other. If transmission takes too long, the next samples could overwrite data that has not been sent.

The transfer size needs attention as well. The nRF52832 SPIM peripheral has an eight-bit transfer-count field, limiting each programmed transfer to 255 bytes [30]. We therefore cannot send the entire 512-byte buffer in one transfer. One option is to divide it into 254-byte, 254-byte, and 4-byte transfers. These even-sized segments would keep each two-byte sample together. The FPGA interface must handle the gaps between segments without treating them as missing or additional samples.

The time needed to detect an abnormal workload depends on the analysis window and the number of abnormal comparisons required to raise a flag. For a sliding window checked after every accepted sample, the time to collect a full window of new data and complete the required comparisons can be estimated as

$$T = (N + P - 1) / fs$$

Here, (N) is the number of samples in the window, (P) is the number of consecutive abnormal comparisons, and (fs) is the sampling rate. A 1,024-sample window with four required comparisons at 200 kSPS gives 5.135 ms. This assumes that the disturbance lasts long enough and produces a window average outside the allowed threshold.

The 5.135 ms estimate covers only this part of the detection process. Buffering, SPI transfers, mode changes, and output timing add to the total delay. Our test must measure the time from a defined detectable change in the target current to the output flag to check the one-second requirement. We also need to choose the live window size based on testing. The 1,024 samples collected for a golden profile do not automatically set the window size used during detection.

Another timing consideration is the FPGA clock. We plan to use a 12 MHz reference, which must come from a suitable oscillator circuit or oscillator module. A passive crystal and load capacitors alone will not provide a clock at an ordinary FPGA input. The final circuit needs to use the clocking features available on the iCE40HX4K.

Our preliminary PLL settings also need to be corrected. With SIMPLE feedback, a 12 MHz input and settings of DIVR = 0, DIVF = 7, and DIVQ = 1 calculate to 48 MHz, not 96 MHz. These settings must also satisfy the PLL’s internal operating limits before they can be used [31]. We will check the configuration with the device tools and include synchronization and buffering between the SPI clock domain and the FPGA processing clock domain.

4.2.3 Package Selection

The components selected for NG PULSE need to work together electrically and fit into a board that our team can assemble and test. A part may meet the circuit requirements but still create problems if its package is difficult to solder or does not match the footprint used in Fusion 360. Package selection therefore needs to happen alongside electrical selection, before we finish the PCB layout.

We plan to use 0805 packages for general resistors and capacitors because they are easier to handle during assembly. However, this size will not be suitable for every component. The current-sensing shunt must be selected for its power dissipation and current rating, which may require a larger package. Inductors and capacitors in the regulator circuits also need to meet their current, voltage, and thermal requirements. We cannot choose these parts based on package size alone.

The MCU, FPGA, amplifier, and regulator packages need separate consideration. Pin spacing and exposed pads affect whether we can solder and inspect them with the equipment available to us. Packages with connections underneath the device can make assembly and troubleshooting more difficult. Before selecting a part, we need to check both its electrical specifications and how we will mount it, inspect the connections, and replace it if necessary.

Each selected part also needs a matching footprint in Fusion 360. We will check the exact package variant, pad dimensions, and pin numbering against the manufacturer's documentation. Two versions of the same device may have different packages or pin arrangements, so the schematic symbol, PCB footprint, and part number in the bill of materials must agree. Finding a similar-looking component in the library is not enough to confirm that the ordered part will fit.

The placement of these components must support the rest of the design. The sensing circuit needs short, carefully routed measurement connections, while the regulators need room for their supporting components and power routing. Connectors must remain accessible once the board is mounted, and nearby parts need enough clearance for soldering and rework. We also need access to programming connections and test points so the MCU, FPGA, and sensing circuit can be checked during integration.

These requirements limit how tightly we can pack the board and which component packages we can use. Our goal is a layout that brings the sensing, processing, and power circuits together while remaining practical for the team to assemble and troubleshoot.

4.2.4 Detection Validity and Security

NG PULSE compares the measured current against a golden profile collected during normal operation. A difference between the two does not necessarily mean unauthorized activity occurred. A normal workload change or a disturbance in the power supply could also cause the current to move outside the threshold. There is also a chance that two different workloads draw similar average

currents, making them difficult to distinguish using our comparison method. Our testing needs to establish which changes the detector can identify and under what operating conditions.

We plan to keep the golden profiles fixed while monitoring the target. This prevents abnormal samples collected during operation from changing the reference used for comparison. The proposed CRC provides a way to check for accidental corruption, but it does not prevent someone from replacing a profile or verifying that a change was authorized. We therefore need a defined procedure for collecting and loading new profiles. Adding remote profile updates would require a separate method for controlling who can make those changes.

The target, MCU, and FPGA must also agree on which operating mode is active. The advanced design includes four work modes and a Not-in-Use state, giving us five states to represent. Two bits only provide four combinations, so this version needs at least three mode bits or another encoding that identifies all five states. During a mode change, the detector may need to pause comparisons while the workload settles. That pause needs a set limit, and the logic must resume monitoring afterward.

We will use separate trials to select the thresholds and evaluate detection performance. Testing only with the data used to tune the detector would not show how well it handles new measurements. The results should include the number of normal and abnormal trials, along with the detection rate, false-positive rate, and F1 score. We also need to resolve the difference between the stated 0% computing-efficiency loss requirement and the earlier allowance of up to 5%. Once that requirement is settled, we can compare the same target workload with and without the monitor under consistent test conditions.

Chapter 5 – Application of ChatGPT and Other Similar Platforms

5.1 Platforms Used and How

The team used large language model (LLM) assistants throughout Senior Design I, mainly Anthropic's Claude and OpenAI's ChatGPT. [Add which members used ChatGPT and for what.] Uses included:

• explaining unfamiliar concepts;

• surveying parts and narrowing candidates;

• drafting and reviewing schematics, BOM tables and Verilog;

• interpreting simulation and synthesis errors;

• organizing project documentation, including a structured project knowledge base and parts of this report.

Every use described in this chapter is listed with its prompt and outcome in Appendix E. Wherever AI-generated text or tables were used in this report, they were reviewed, checked against primary sources (datasheets, simulation output, sponsor guidance) and edited by the responsible team member.

5.2 Advantages

• Speed of orientation. An assistant could explain, in minutes, background that would otherwise take hours across datasheets and application notes. Examples include why BLE SoCs are never sold in leaded packages, what a successive-approximation ADC needs at transistor level, and how iCE40 configuration works.

• Breadth of the first pass. Asking for "every hand-solderable FPGA with a maintained toolchain" produced a candidate list across vendors, which the team then checked against vendor documentation.

• Second reviewer. Pasting a schematic, BOM or testbench and asking for problems found real issues a tired reviewer might miss. Examples are the current-sense amplifier gain discrepancy and the SWD/I²C pin overlap.

• Documentation discipline. Keeping a dated decision log with reasons and rejected alternatives, maintained with AI help, directly fed Chapter 3 of this report.

5.3 Disadvantages and Limitations

• Confident errors about part availability and specifications. Assistants stated package options and part numbers that do not exist (Section 5.4.2). Every number in this report was re-checked against manufacturer documentation.

• Plausible but wrong debugging hypotheses. An assistant explains a symptom fluently even when the explanation is wrong (Section 5.4.3).

• Generated detail that looks finished but is not. A generated schematic had pin numbers that looked authoritative but were placeholders (Section 5.4.4).

• No access to the lab. An assistant cannot measure the board. Golden-profile values, thresholds and timing margins must come from measurement.

• Learning risk. If accepted without understanding, AI output can replace learning instead of supporting it. The team's rule was that the responsible member must be able to explain, without the assistant, any AI-assisted design decision they present.

• Restricted information. Comparable defense integrity monitors are proprietary or export-controlled. The assistant could only draw on open literature, which matches the team's own constraint.

5.4 Examples

5.4.1 FPGA Selection Under a Package Constraint (helped, with a correction)

On 10/8/2026 the team learned it could not hand-solder BGA packages and asked an assistant for an LQFP alternative to the BASYS 3's Artix-7. The assistant correctly explained that no Artix-7 is offered in a leaded package. It first proposed the Xilinx Spartan-6 XC6SLX9 in TQG144, which met the package and logic requirements. Further research, including a community thread on programming Spartan-6, showed that its only toolchain is ISE 14.7 (2013), which runs on modern Windows only as a virtual machine and has weak SystemVerilog support. When asked again with toolchain maintenance as an explicit criterion, the assistant identified the Lattice iCE40HX4K-TQ144, which the team selected.

Lesson: the assistant optimized for the constraints it was given. The toolchain criterion was obvious to experienced FPGA users but was not stated until the team found it. Stating all selection criteria up front, and checking recommendations against practitioners, mattered as much as the assistant's knowledge.

5.4.2 A Part That Does Not Exist (harmed, caught by the team)

During the same discussion the assistant described the iCE40HX8K-TQ144 as a drop-in upgrade path. Jacob checked Lattice's ordering information and found that the HX8K is offered only in BGA packages; the TQ144 part does not exist. The team re-verified the headroom estimates against the HX4K alone. A related slip surfaced in review: the Rev 1 schematic and BOM named the nRF52832 module "MDBT40", which is the older nRF51822 module; the correct part is the MDBT42Q (Section 3.2.1.1).

Lesson: LLMs produce part numbers that follow a family's naming pattern even when the part was never made. Availability and package claims must be confirmed from the manufacturer's ordering guide before they reach a schematic.

5.4.3 RTL Debugging: the Bug Was in the Testbench (mixed)

On 10/22/2026, scenario S3 (sustained anomaly should raise alert) failed while the other four scenarios passed.

1. The assistant first suggested the anomaly was too short. Lengthening it did not help.

2. It then suggested the comparison logic was clearing its persistence counter while the window filled, and proposed restructuring that logic. The change was cleaner but did not fix the failure.

3. The root cause was found only after the design was rerun in Icarus Verilog and internal signals were probed. The blanking counter was still at 120 in the middle of S3, because the mode change in S2 had started a 256-cycle blanking window that S2b did not wait out. The design was correctly suppressing alerts, and the bug was in the testbench timing.

4. After S2b was extended to BLANK_CYCLES + N + 32 cycles, all scenarios passed in both Vivado XSim and Icarus.

Lesson: the assistant was useful for setting up the second simulator and the probes, but its first two explanations were guesses. Evidence from the waveform settled it. One unnecessary design change also came out of the guessing, which shows that AI-suggested "fixes" should be reverted when they don't address the confirmed cause.

5.4.4 Concept Clarification and Review (helped)

• Immutable golden profile. Jacob initially proposed recalculating the golden profile from each measurement. In discussion, the assistant explained that an adaptive baseline can be walked by a slow-drift attack, where the attacker shifts the rail gradually and the reference follows. That led to the team's decision to keep the golden profile immutable and reject false alarms with windowed averaging, a persistence counter and mode-change blanking.

• ASIC scope. The assistant explained why a 12-bit SAR ADC is analog design work (capacitor DAC, comparator, bootstrapped sampling switch) that the DRACO digital flow does not cover. This supported keeping the ADC out of the ASIC scope, as the sponsor later confirmed.

• Schematic review. Reviewing the current-sense schematic, the assistant flagged that the feedback resistors set a gain of 10,000 while the amplifier was labelled 1,000×, and that about 150× would be appropriate for the expected shunt voltage. That issue is now an open item for the current-sense design.

• Placeholder pin numbers. The same exercise produced a cautionary example. The generated Rev 1 schematic listed iCE40 pin numbers that looked authoritative but had not been checked against the datasheet, including a 12-bit bus assigned to a single pin. The schematic was marked "verify against DS1040 before layout".

Lesson: AI was most valuable as an explainer and reviewer whose claims could be checked, and most dangerous as a source of finished-looking detail.

5.5 Summary

AI assistants shortened research and review cycles and helped the team document its reasoning. They also introduced errors that were caught only because the team checked datasheets and simulation results. The team's working rules for SD2 are:

1. State every selection criterion explicitly.

2. Verify every part number and specification against the manufacturer's documentation.

3. Treat AI debugging suggestions as hypotheses that waveforms must confirm.

4. Present only AI-assisted work the responsible member can explain unaided.

Chapter 6 – Hardware Design

6.1 Current Sense Board

The Current Sense Board serves as the primary data acquisition interface for the NG PULSE system. Its purpose is to monitor the electrical current consumed by the target device and convert variations in current consumption into an analog voltage signal suitable for digitization by the Nordic microcontroller. This measurement provides the input data necessary for generating reference current signatures and identifying deviations from expected operating behavior. The board operates independently of the target’s internal software and processing functions.

The Current Sense Board incorporates a high-side shunt resistor, a dedicated current-sense amplifier, USB-C input and output interfaces, and a current-limiting circuit for protecting NG PULSE’s supporting electronics. The circuit uses a nominal 5 V USB supply. The amplifier measures the voltage difference across the shunt and produces an amplified, ground-referenced signal for the Nordic microcontroller’s analog-to-digital converter (ADC). All discussion in this section will refer to this circuit diagram:

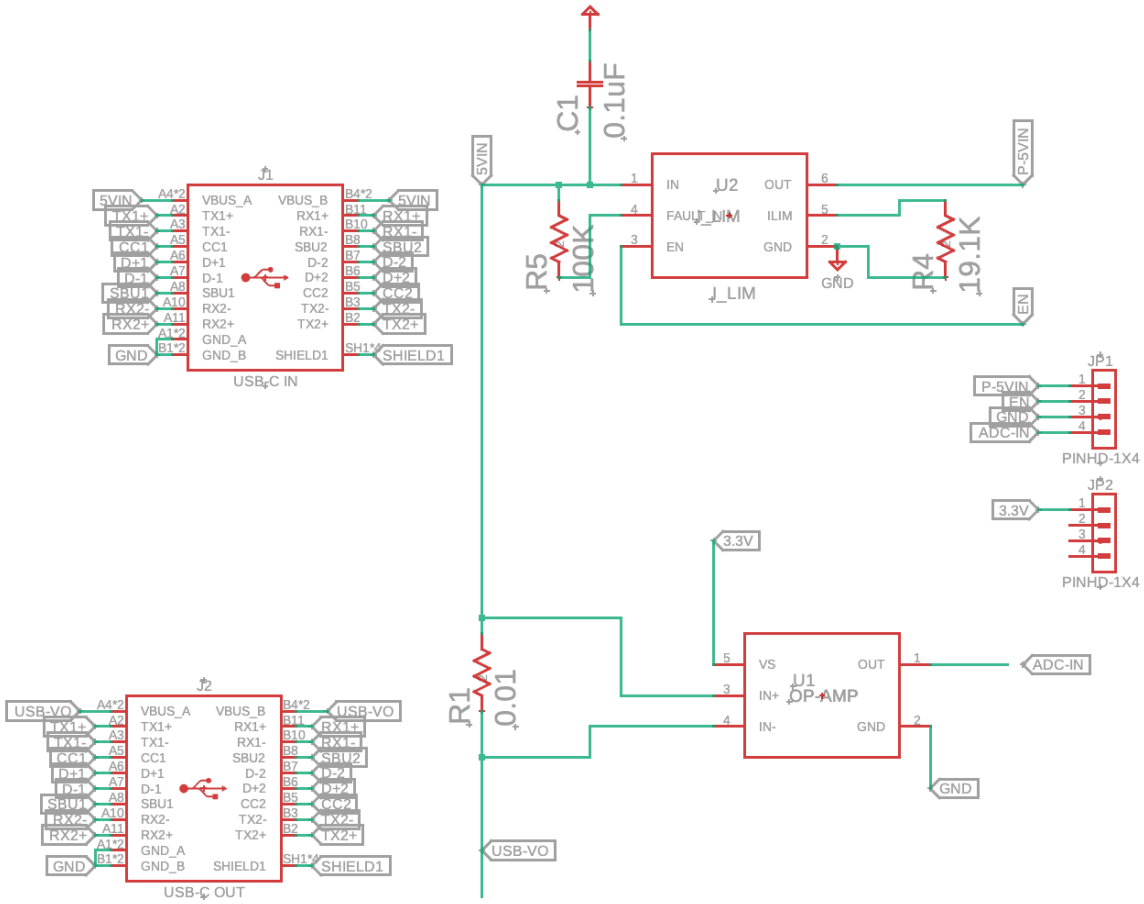


Figure 6.1: Current Sense Board

6.1.1 Current Sense Board Architecture

The Current Sense Board is organized into two electrical paths: the target power path and the NG PULSE protection and measurement path. Both originate from the incoming USB-C supply but perform different functions.

The target power path connects the incoming 5 V supply to the target's USB-C input through a series shunt resistor. The shunt introduces a small voltage drop proportional to target current; it is used for measurement rather than regulation or control. NG PULSE does not actively interrupt or limit target supply current during normal operation.

A separate branch supplies NG PULSE's supporting electronics through a current-limiting integrated circuit. This protects the monitoring hardware without placing the limiter in series with the target. The high-side current-sense amplifier monitors the shunt voltage and sends its analog output to the Nordic microcontroller for digitization.

6.1.2 High-Side Current Measurement

Current measurement uses a low-resistance shunt resistor, R1, connected in series with the positive supply between the incoming USB-C connector and the target's USB-C power input. The accepted schematic specifies R1 = 0.1 Ω.

$V_{shunt} = I_{target} \times R_{shunt}$

Here V_{shunt} is the voltage drop across the shunt in volts, I_{target} is the target current in amperes, and R_{shunt} is the shunt resistance in ohms.

For a nominal target current of 18 mA:

$V_{shunt} = 0.018\ A \times 0.01\ \Omega = 0.00018\ V = 180\ \mu V$

The low resistance minimizes target supply reduction. Because the resistor is on the positive supply rail, this is high-side sensing. The upstream shunt terminal connects to the amplifier's non-inverting input (IN+), and the downstream terminal connects to its inverting input (IN−), producing a positive output for current flowing toward the target.

6.1.3 Current-Sense Amplifier

U1 is the Texas Instruments INA180A4 current-sense amplifier. Unlike a conventional general-purpose operational amplifier, it is designed to detect small differential voltages in the presence of a larger common-mode voltage. Both shunt terminals operate near the nominal 5 V rail, although their difference may be only a fraction of a millivolt.

The INA180A4 provides a fixed nominal gain of 200 V/V:

$V_{out} = A_v \times (V_{IN+} - V_{IN-}) = A_v \times I_{target} \times R_{shunt}$

At the nominal operating current:

$V_{out} = 200 \times 0.018\ A \times 0.01\ \Omega = 0.036\ V$

Thus, the ideal output is approximately 360 mV. The amplifier VS pin receives the 3.3 V supply; GND connects to system ground; IN+ and IN− sense opposite shunt terminals; and OUT is routed to ADC-IN at the Nordic interface header.

The measured output represents the target's current variation subject to the amplifier's bandwidth, offset, noise, and output-swing limits. These characteristics are especially important because the nominal signal is small.

6.1.4 Current Limiting and Protection

U2 is a Texas Instruments TPS2553DBVR-1 current-limiting power-distribution switch. Its role is to protect NG PULSE's supporting electronics, not to regulate or interrupt power to the target.

The TPS2553 receives the incoming 5 V supply at IN and provides the protected P-5VIN output. R4 = 19.1 kΩ connects ILIM to ground to establish the programmed current-limit setting; the actual threshold depends on the device characteristics. EN is routed to the external enable interface. The active-low,

open-drain FAULT indication has R5 = 100 kΩ pulling it up to 5VIN. C1 = 0.1 µF provides input bypassing.

The switch is not in series with the target USB supply. Its limiting action therefore applies only to the NG PULSE protected branch. It does not independently guarantee that the Nordic ADC is protected against every transient originating on the target rail; that depends on the amplifier operating limits and ADC input interface.

6.1.5 USB-C Power Interface

J1 is the USB-C input from the external power source and J2 is the outgoing USB-C connection to the target. The 5VIN net supplies the upstream shunt terminal and the TPS2553 input. The downstream shunt terminal is labeled USB-VO and connects to the outgoing target power interface. The difference between 5VIN and USB-VO is measured by the INA180A4.

This inline configuration supports target current monitoring without modifying the target's internal power circuitry. The accepted schematic establishes the power-sensing connections; USB-C configuration-channel termination and any required data pass-through connections remain implementation details for final integration verification.

6.1.6 Interface to the Nordic Microcontroller

The INA180A4 output is routed as ADC-IN to JP1, the four-pin interface header. The Nordic microcontroller samples this analog signal using its ADC and converts it to digital values for current-signature processing.

Table 6.1: Primary Pin-Header Descriptions

Pin	Signal	Function
1	P-5VIN	Protected 5 V supply for NG PULSE circuitry
2	EN	Current-limiter enable control
3	GND	Common ground reference
4	ADC-IN	Amplified current measurement

An additional header, JP2, provides access to the 3.3 V rail, and has 3 extra pin-headers to allow for completion of stretch goals or for potential troubleshooting during testing. The Current Sense Board therefore performs analog acquisition and conditioning, while the Nordic microcontroller performs digitization and subsequent data handling.

6.1.7 Design Considerations and Limitations

The design aims to minimize interference with the target's power supply while producing a measurable analog representation of its current consumption. The 0.1 Ω shunt produces a small drop at 18 mA, preserving supply voltage but yielding a relatively small amplifier output. Overall sensitivity depends on the shunt, amplifier, and Nordic ADC.

The fixed 200 V/V gain removes the need for an external gain-setting resistor network. Higher target currents produce proportionally larger outputs until the amplifier approaches its output-swing limits. Input offset, gain error, noise, and bandwidth may affect detection of small differences among operating modes.

A shared ground reference, appropriate supply bypassing, and careful PCB routing are needed to preserve signal integrity. The accepted schematic establishes the intended architecture; final hardware verification will determine measurement resolution, signal quality, and operating margin.

6.1.8 Section Summary

The Current Sense Board converts target supply-current variations into a voltage for the Nordic ADC using a high-side shunt and INA180A4 amplifier. The independent TPS2553 branch protects NG PULSE supporting electronics without limiting the target current. Together, the USB-C interfaces, shunt, amplifier, and headers provide the analog acquisition foundation for current-signature anomaly detection.

6.2 FPGA/MCU Board

[TO DO: Subsystem block diagram and schematic.]

6.3 Power Distribution

The NG PULSE power distribution subsystem receives a nominal 5 V supply through a USB-C connection and distributes power to the monitored target and the monitoring electronics through separate branches. The target supply passes through the current-sensing shunt, whereas the NG PULSE electronics are supplied from the output of a dedicated current-limiting switch. Downstream switching regulators provide the voltages required by the monitoring hardware. This architecture maintains a measurable target current path while applying independent current limiting to the monitoring subsystem.

6.3.1 USB-C Power Input

The system uses an incoming USB-C connector to receive nominal 5 V power from an external source. The incoming VBUS connection forms the common upstream supply node, and the ground connection provides the electrical return reference. The incoming supply is divided into two functional paths: a target path and an NG PULSE electronics path.

In the target path, the 5 V supply passes through the high-side current-sense shunt resistor before reaching the outgoing USB-C connection to the ESP32 target. The small voltage difference across this resistor is measured by the INA180A4 current-sense amplifier, as described in Section 6.1. The TPS2553 current limiter is not placed in series with this target path.

In the monitoring path, the same incoming 5 V supply feeds the TPS2553 current-limiting switch. The switch output then supplies the downstream NG PULSE regulators. The separate schematic for the 3.3 V regulator labels its input 5VIN; in the integrated system, this label refers to the protected 5 V output of the current limiter, not a direct bypass around the limiter.

The final PCB implementation must preserve the intended USB-C VBUS, ground, configuration-channel, and any required USB data connections. Connector role configuration and available USB source current should be verified during integration; the presence of a USB-C connector alone does not establish USB-C negotiation or data pass-through functionality.

6.3.2 Current Limiting and Protection

The Texas Instruments TPS2553DBVR-1 power-distribution switch provides overcurrent protection for NG PULSE electronics. Its IN terminal receives the upstream 5 V supply, while its OUT terminal generates the protected supply used by downstream regulation stages. The target-device supply is intentionally excluded from this protected branch so that the monitoring electronics do not directly limit the target current being observed.

The accepted current-sense-board schematic uses a 19.1 kΩ resistor (R4) between the ILIM terminal and ground to program the current-limit behavior. The resulting threshold must be established using the device-specific datasheet relationship and tolerance limits; it should not be inferred from the resistor value alone. The enable input is routed to an EN control connection, and the active-low FAULT output uses a 100 kΩ pull-up resistor (R5) to the 5 V input rail. The fault signal can indicate an abnormal condition in the protected branch if incorporated into the system monitoring interface.

A 0.1 µF capacitor is shown at the input supply node in the current-limiter schematic. Appropriate local input bypassing and PCB placement are necessary to manage supply transients. The TPS2553 provides protection to its downstream power path; it does not independently clamp the analog output of the INA180A4 or guarantee protection against all target-side transients. Those interfaces are evaluated separately.

The current-limit threshold must be coordinated with the combined startup and steady-state input demand of the 3.3 V and future 1.2 V regulators. Otherwise, normal startup inrush or simultaneous rail loading could trigger unintended current limiting.

6.3.3 3.3 V Buck Regulator

The 3.3 V rail is generated by the Texas Instruments TLV62569ADRLR, an adjustable synchronous step-down (buck) converter in a six-pin SOT-563 package. The converter receives the protected nominal 5 V supply from the TPS2553 and produces the lower-voltage rail used by compatible NG PULSE electronics, including the Nordic microcontroller and INA180A4 current-sense amplifier. The TLV62569A family supports a 2.5 V to 5.5 V input range and has a rated output-current capability of up to 2 A, subject to thermal and application conditions. Its typical switching frequency is 1.5 MHz and it operates in forced PWM mode.

The current schematic uses a 4.7 µF input capacitor (C1), a 1 µH switching inductor (L1), and a 22 µF output capacitor (C2). The input capacitor provides local energy storage and reduces switching-related disturbance at VIN. The inductor and output capacitor smooth the switching waveform into a regulated DC output. The enable pin is tied to the regulator input so the converter is enabled when the protected 5 V rail is present. The power-good (PG) output is not connected in the current schematic. See the schematic below:

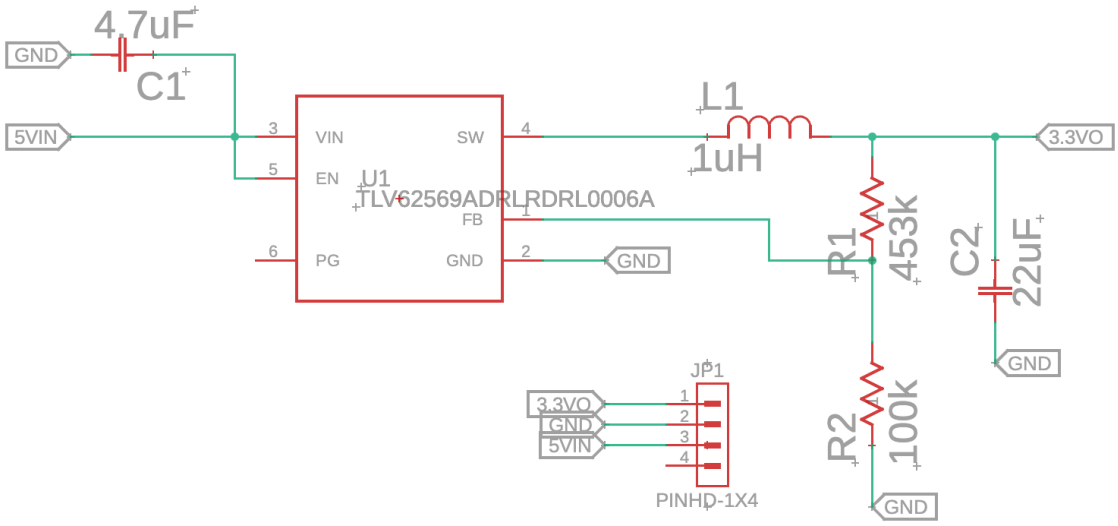


Figure 6.2: 3.3 V Buck Regulator (TLV62569)

The output voltage is established by the resistor divider connected between the output, FB input, and ground. The upper resistor R1 is 453 kΩ, and the lower resistor R2 is 100 kΩ. Using the nominal 0.6 V feedback reference:

VOUT = VFB × (1 + R1 / R2)

VOUT = 0.6 V × (1 + 453 kΩ / 100 kΩ) = 3.318 V

The calculated nominal output of 3.318 V is close to the intended 3.3 V rail. The actual output varies with feedback-reference tolerance, resistor tolerance, loading, and operating conditions. The regulator is selected to reduce the power loss associated with stepping 5 V down to 3.3 V compared with a conventional

linear regulator; actual conversion efficiency must be evaluated at the expected load current.

Table 6.2: Component Explanations

Schematic designator	Selected component / purpose
U1	TLV62569ADRLR; synchronous buck regulator
C1	4.7 µF; input bypass capacitor
L1	1 µH; switching inductor
C2	22 µF; output capacitor
R1	453 kΩ; upper feedback resistor
R2	100 kΩ; lower feedback resistor
JP1	Four-pin header: 3.3VO, GND, 5VIN, unused fourth pin

The selected 1 µH / 22 µF output filter and 4.7 µF input capacitor are consistent with values appearing in the manufacturer's typical application guidance, but final capacitor dielectric, DC-bias derating, inductor saturation-current rating, thermal behavior, and PCB layout must be verified for the actual load. The FB trace should be routed away from the switching node, and the input bypass capacitor should be placed close to the regulator power pins.

6.3.4 1.2V Buck Regulator

The 1.2 V power rail is intended to supply the FPGA core circuitry and is generated using the Texas Instruments TPS62160DGKT synchronous step-down regulator (U1). The regulator will receive its input from the protected 5 V distribution branch downstream of the TPS2553 current limiter. In the standalone regulator schematic, this protected input is labeled VIN. The 1.2 V output is routed to the main PCB through a pin-header interface, allowing the regulator to be implemented as a separate daughterboard.

The proposed circuit includes a 10 µF input capacitor (C2), a 2.2 µH inductor (L1) between the SW pin and the output, and a 22 µF output capacitor (C1). The input capacitor provides local energy storage and limits input voltage disturbances caused by switching. The inductor and output capacitor form the output filter that smooths the regulator's switching waveform into the DC supply required by the FPGA. Both analog ground (AGND) and power ground (PGND) connect to the common ground net; their PCB routing and return paths will be considered during layout to reduce switching noise. See the schematic below:

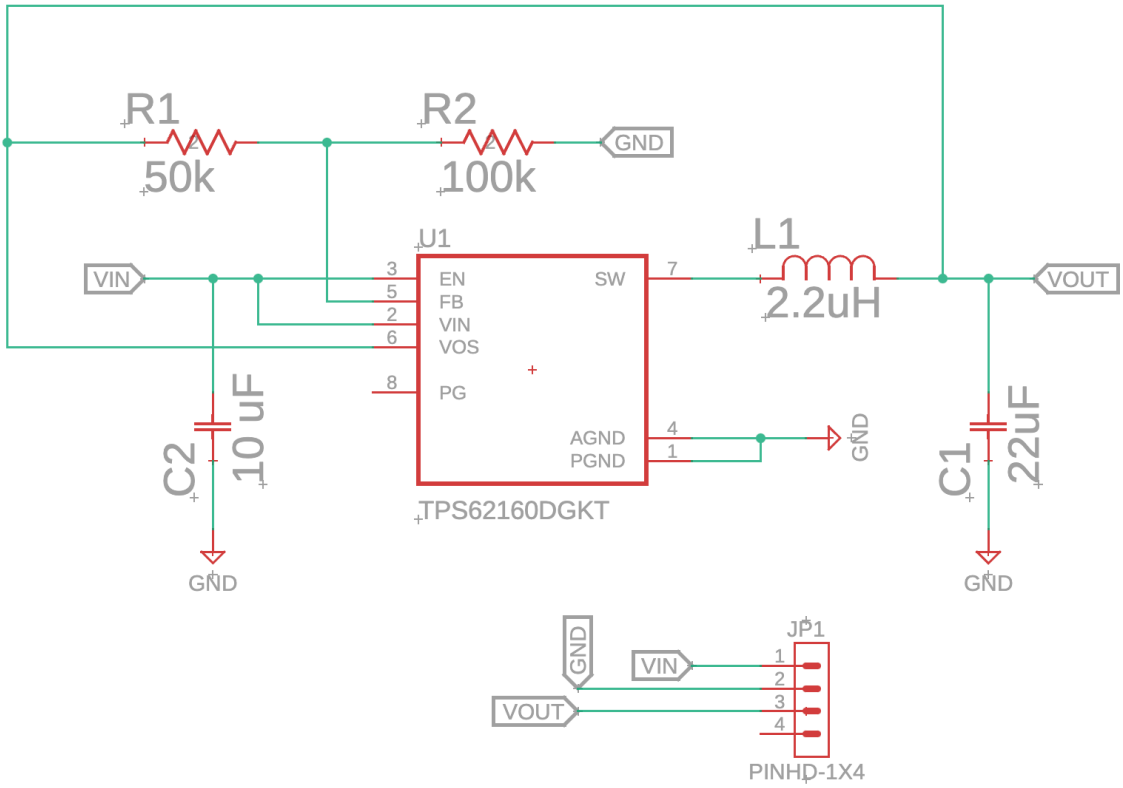


Figure 6.3: 1.2 V Buck Regulator (TPS62160)

The regulator uses an external feedback divider to establish the output voltage. The schematic connects a 50 kΩ upper resistor (R1) from VOUT to FB and a 100 kΩ lower resistor (R2) from FB to ground. Using the nominal 0.8 V feedback reference, the output voltage is calculated as follows:

VOUT = VFB × (1 + R1 / R2)

VOUT = 0.8 V × (1 + 50 kΩ / 100 kΩ) = 1.20 V

The VOS sense input is connected directly to the output node, providing output-voltage sensing at the regulator. The EN pin is tied to VIN, so the converter is intended to enable when input power is applied. The PG (power-good) pin is not connected in the present schematic, because the proposed module does not require a separate power-good indication. The four-position interface header (JP1) exposes VIN, GND, and VOUT; the fourth position is unused in the current design.

Table 6.3: Component Descriptions (1.2V)

Reference	Component/value	Purpose
U1	TPS62160DGKT	Synchronous buck regulator
C2	10 µF	Input bypass capacitor
L1	2.2 µH	Switching/output filter inductor
C1	22 µF	Output filter capacitor

R1	50 kΩ	Upper feedback resistor
R2	100 kΩ	Lower feedback resistor
JP1	4-pin header	VIN, GND, VOUT interface; one spare pin

Before fabrication, the regulator's current capacity, passive-component ratings, output-voltage tolerance, and startup behavior will be checked against the selected FPGA's power requirements. The layout will prioritize short switching-current loops, local capacitor placement, and appropriate separation of the switching node from the feedback trace. The 1.2 V rail will be verified independently before connection to the FPGA.

6.3.5 Overall Power Distribution Architecture

The system-level power tree separates the monitored target from the monitoring electronics at the USB-C input. In the first branch, USB VBUS passes through the current-sensing shunt and continues to the ESP32 target. In the second branch, the TPS2553 creates a protected nominal 5 V rail for NG PULSE. The TLV62569A converts this protected rail to approximately 3.3 V. A separate 1.2 V regulator will be incorporated into the same protected distribution architecture once its design is finalized.

Power path summary: USB-C 5 V input → (a) shunt resistor → ESP32 target; (b) TPS2553 current limiter → protected 5 V → 3.3 V regulator and future 1.2 V regulator.

The two branches share an electrical ground reference but serve different functions. The target branch prioritizes current observation with minimal additional series impedance, while the monitoring branch prioritizes regulated, protected power for data acquisition and comparison circuitry. Final system validation should include steady-state current measurements, startup behavior, output ripple, rail regulation under load, and verification that current limiting does not unintentionally interrupt normal NG PULSE operation.

Chapter 7 – Software Design

7.1 RTL Design

The detection datapath is written once in synthesizable Verilog and used on two platforms: the iCE40HX4K FPGA on the physical prototype, and the simulated digital ASIC that is carried through the DRACO Laboratory's Cadence flow (Section 7.4). This section describes that shared RTL as it stands at the midterm: what each module does, how it maps onto the goal tiers of the Divide and Conquer (D&C), how it was verified, and what still has to change. The complete source listings and testbenches are in Appendix D (D.2); none of the code is reproduced in this chapter.

7.1.1 RTL Scope and Goal Tiers

The D&C defines three goal tiers. The RTL is organized so that each tier adds to the one below it without rewriting it:

• Basic goal. Monitor the ESP32-C6 supply rail in one work mode and in the Not-in-Use state against a stored golden profile, and raise an error flag when the rail departs from it. On the ASIC track, the basic goal is the Cadence digital design of exactly this comparison logic and flag. In the RTL this is the windowed mean, the absolute-deviation comparison, the persistence filter and the alert output.

• Advanced goal. Extend monitoring to four work modes plus Not-in-Use, select the right golden profile automatically from the mode the ESP32-C6 reports, and keep monitoring continuously through mode transitions. In the RTL this is the golden-profile store indexed by op_mode, and the blanking logic driven by mode_changing and a post-change hold-off counter.

• Stretch goal (ASIC). Add ADC functionality, memory storage and current capture to the ASIC (a 4-bit digital ADC first, then 12-bit). None of this is in the RTL yet. The hard-coded golden store is the first step toward the memory-storage item, because it already places the golden values inside the synthesized design rather than outside it.

Table 7.1 maps each RTL feature to its tier and gives its status at the midterm.

Table 7.1: RTL Features by D&C Goal Tier

Feature	Module	Goal tier	Status (11/6/2026)
16-sample running mean of 12-bit samples	windowed_mean	Basic	Coded, simulated
Absolute deviation from the golden value, compared with THRESHOLD	comparison_logic	Basic	Coded, simulated
Persistence filter (PERSIST_COUNT consecutive anomalous comparisons) and alert flag	comparison_logic	Basic	Coded, simulated
Golden value for Not-in-Use plus one work mode	golden_store	Basic	Coded; values are placeholders
Four-entry golden	golden_store	Advanced	Coded for four codes;

store selected by op_mode			a fifth state needs a 3-bit op_mode (Section 7.1.4)
Blanking during and after mode transitions (mode_changing and BLANK_CYCLES)	comparison_logic	Advanced	Coded, simulated
Cadence synthesis of the comparison logic and flag	all	Basic (ASIC)	Planned (Section 7.4)
Golden values held inside the ASIC	golden_store	Stretch (ASIC memory storage, first step)	Coded as a constant ROM
On-die ADC (4-bit, then 12-bit) and current capture	none yet	Stretch (ASIC)	Not started
SPI sample receiver in place of the parallel adc_in port	FPGA wrapper	Prototype integration	Planned (Section 7.2.5)
Input synchronizers for op_mode and mode_changing	FPGA wrapper	Prototype integration	Planned (Section 7.1.6)

7.1.2 Datapath Overview

The top-level module, ng_pulse_top, instantiates three sub-modules in a straight pipeline. A 12-bit current sample enters on adc_in. windowed_mean turns the stream of samples into a running average over the last N samples. golden_store supplies the expected average for the operating mode reported on op_mode. comparison_logic subtracts the two, decides whether the difference is large enough and persistent enough to be an anomaly, and drives the alert output and the deviation value. Table 7.2 summarizes the modules and their interfaces.

Table 7.2: RTL Module Summary

Module	Function	Inputs	Outputs	State held
ng_pulse_top	Top level; wires the three sub-modules and exposes the parameters	clk, rst_n, adc_in[11:0], op_mode[1:0], mode_changing	alert, deviation[11:0]	None (structural)

windowed_mean	Running mean of the last N samples	clk, rst_n, sample_in[11:0]	mean_out[11:0], mean_valid	N × 12-bit sample window, 16-bit running sum, fill counter
golden_store	Constant ROM of golden means, one per op_mode code	op_mode[1:0]	golden_out[11:0]	None (combinational)
comparison_logic	Absolute deviation, threshold, persistence filter, blanking	clk, rst_n, live_mean, mean_valid, golden, op_mode, mode_changing	alert, deviation[11:0]	Blanking counter, previous op_mode, alarm counter, alert and deviation registers

The design uses one clock and an active-low asynchronous reset. Every module is written in portable Verilog with no vendor primitives, so the same source compiles in Vivado XSim, Icarus Verilog, the iCE40 open-source flow and Cadence Genus. All parameters are set once in ng_pulse_top and passed down; Table 7.3 lists them with their current values.

Table 7.3: RTL Parameters

Parameter	Value	Meaning	Effect at 200 kSPS (one window step per sample)
WIDTH	12	Sample, mean, golden and deviation width in bits	Matches the nRF52832 SAADC's 12-bit result
N / N_LOG2	16 / 4	Samples in the averaging window; N must be a power of two	80 µs window
THRESHOLD	50	Allowed absolute deviation in ADC codes (strictly greater raises a count)	About 1.2% of full scale
PERSIST_COUNT	4	Consecutive anomalous comparisons needed to assert alert	20 µs of sustained deviation
BLANK_CYCLES	256	Hold-off after every	1.28 ms

		op_mode change	

7.1.3 Windowed Mean (Basic Goal)

windowed_mean keeps the last N samples in a shift register, with index 0 holding the newest sample. Instead of adding all N samples every cycle, it keeps a running sum and updates it incrementally: each cycle it subtracts the sample leaving the window and adds the one entering. The sum is WIDTH + N_LOG2 = 16 bits wide, which holds the worst case of 16 × 4,095 = 65,520 without overflow. Dividing by N is a right shift by N_LOG2, so no divider is synthesized. The mean is computed from the updated sum in the same clock as the new sample is registered, so mean_out always includes the most recent sample.

A fill counter holds mean_valid low until the window contains N real samples after reset. This keeps the comparison from acting on a window that is still partly filled with the reset value of zero, which would look like a large drop in current.

The mean is a deliberate choice for the basic goal. It is cheap (one adder, one subtractor and a shift), it suppresses uncorrelated ADC noise by roughly $\sqrt{N}$ (a factor of four for N = 16), and it is easy to reproduce in a golden model. Variance and peak-to-peak features are design extensions beyond the D&C (Section 3.1.3.1) and would be added beside the mean, not in place of it.

7.1.4 Golden-Profile Store (Basic and Advanced Goals)

golden_store is a small combinational ROM. It returns one 12-bit golden mean for each value of op_mode. The four entries are named IDLE (00), COMPUTE (01), CRYPTO (10) and RADIO (11) in the code. At the midterm all four hold the placeholder value 0x800 (mid-scale), because the real values can only come from the golden-profile characterization run on the physical prototype (Section 7.2.9).

Mapping to the goal tiers. We interpret code 00 as the Not-in-Use state. The reasoning is that when the ESP32-C6 is held in reset or is not running a workload, it cannot drive its mode lines, and pull-down resistors on those lines (to be confirmed on the Rev 2 schematic) leave op_mode at 00. The default state of the lines is then the state in which the target is doing nothing, which is what Not-in-Use means. With that mapping:

• the basic goal (Not-in-Use plus one work mode) uses codes 00 and one other code, and is fully covered by the current 2-bit store;

• the advanced goal (four work modes plus Not-in-Use) needs five codes. Two bits cannot hold five states, so op_mode will be widened to 3 bits. The change is local: the op_mode ports and the ROM case statement grow by one bit, code 000 stays Not-in-Use, and codes 001 to 100 become the four work modes. The ESP32-C6 firmware and the nRF52832 pin assignment (Table 7.11) need one more mode line. This is listed as an open item with the ESP32-C6 firmware (Section 7.3).

Why the golden store is immutable. The store is fixed at synthesis and has no write path. An adaptive baseline that slowly follows the measured current would be easier to tune, but it can be "walked": an attacker who raises the current in small steps, each below THRESHOLD, would drag the baseline along and never trigger an alert. Fixing the golden values removes that attack. On the prototype the values are loaded once at boot from the nRF52832's protected profile record (Section 7.2.9); on the ASIC they are constants in the netlist.

7.1.5 Comparison, Persistence and Blanking (Basic and Advanced Goals)

comparison_logic does three things every clock.

Absolute deviation (basic). When the window is full and the comparison is not blanked, it computes |live_mean − golden| with a compare-and-subtract, so no signed arithmetic is needed. The result is registered on the deviation output, which the nRF52832 can read for logging and telemetry through the FPGA register map (Section 7.2.6).

Persistence filter (basic). A deviation larger than THRESHOLD increments a 3-bit alarm counter, which saturates at PERSIST_COUNT; any comparison at or below THRESHOLD clears it. alert is asserted on the comparison that brings the run of consecutive anomalous comparisons to PERSIST_COUNT, and it stays high for as long as the deviation stays above THRESHOLD. It clears on the first in-range comparison. alert is therefore a level that tracks the anomaly, not a latched event. Counting and escalation (SUSPECT, reset, power kill) are left to the nRF52832 policy state machine (Section 7.2.8), which keeps the FPGA stateless and fast. The prototype's ALERT# pin is the active-low inverse of alert.

The filter counts consecutive comparisons, one per window step. It does not count separate, non-overlapping windows of N samples. Because successive windows overlap in N − 1 samples, four consecutive comparisons correspond to 4 new samples, not 64. The testbench comments describe the S3 stimulus in terms of whole windows, but the stimulus is longer than either interpretation needs, so the result is unaffected. The behaviour described here is the behaviour of the RTL.

Blanking (advanced). A legitimate change of workload moves the rail current, and a comparison made during the move would raise a false alert. Comparisons are blanked whenever either condition holds:

• mode_changing is high. The ESP32-C6 raises it before it touches the workload and lowers it after the new workload has settled (Section 7.2.7). mode_changing replaces the earlier active-high mode_valid strobe (mode_valid = ~mode_changing). The team agreed the change, and it is made in the RTL listed in Appendix D.

• The blanking counter is running. Whenever op_mode differs from its value in the previous clock, the counter is loaded with BLANK_CYCLES and counts down to zero. This is a second line of defence: it covers a mode change that arrives without mode_changing, and gives the analog front end time to settle after one that does.

While blanked, the alarm counter, deviation and alert are cleared, so no partial run of anomalous comparisons carries across a mode change. While the window is still filling after reset, the comparison state is held rather than cleared.

Continuous monitoring across mode transitions, which is the advanced-goal requirement, follows from the two mechanisms together. The monitor is blind only while the target says it is changing modes and for a bounded hold-off afterwards, and it compares against the new mode's golden value as soon as both have ended. The fixed hold-off is also a predictable blind spot, which an attacker could exploit by timing a burst to coincide with a mode change. Whether to keep a fixed BLANK_CYCLES, shorten it, or end blanking adaptively once the mean has settled is an open design decision.

7.1.6 Timing and Integration on the Prototype

As written, the RTL takes a new sample on every clock. That matches the ASIC testbench, which drives adc_in directly, but not the prototype, where samples arrive from the nRF52832 over SPI at 200 kSPS and the FPGA runs from the 12 MHz reference (Section 3.2.4), multiplied by the iCE40 PLL. Feeding the datapath once per clock would fill the window with repeated copies of the same sample. The FPGA wrapper will therefore:

• add a sample-valid strobe from the SPI receiver (Section 7.2.5) that enables the window shift, the persistence counter and the blanking counter, so that all three advance once per sample instead of once per clock. The parameter timings in Table 7.3 assume this;

• pass op_mode and mode_changing through two-flip-flop synchronizers, because the ESP32-C6 drives them asynchronously to the FPGA clock. The two op_mode bits can be captured on different clocks during a change, so a transient wrong code can appear for one clock. Raising mode_changing before op_mode changes, as the firmware rule requires, keeps that transient inside the blanking window;

• synchronize the de-assertion of rst_n, so that every register leaves reset on the same clock edge;

• drive ALERT# as the active-low inverse of alert, and report the alert state and op_mode through the STATUS register of the I²C register map (Table 7.8).

At 200 kSPS with one window step per sample, the measured latencies in Section 7.1.7 become 30 µs for a large step (6 samples) and 90 µs for a deviation just above THRESHOLD (18 samples). Even with the window fill time added, both are more than four orders of magnitude inside the 1 s detection-latency specification. The logic itself is shallow: the longest path is the 16-bit add-and-subtract in windowed_mean followed by the shift, which is not expected to limit timing at 12 MHz or at the PLL frequency. This will be confirmed by the iCE40 timing report and by Genus (Section 7.4).

The current RTL uses about 250 flip-flops: 192 for the 16-sample window, 16 for the running sum and the rest for counters and output registers. That is well under

a tenth of the iCE40HX4K's 3,520 logic cells. The ping-pong sample buffer planned for the feature extractor would use block RAM, not logic (Section 3.2.2). These are estimates; the design has not yet been synthesized for the iCE40.

7.1.7 Verification

The top-level testbench, tb_ng_pulse_top (Appendix D), runs the complete design with a 100 MHz simulation clock, a mid-scale golden value and ±5-code random noise on the clean rail. It checks five pass/fail conditions in four scenarios, listed in Table 7.4. The original RTL passed all of them in Vivado XSim and Icarus Verilog on 10/22/2026. After the change from mode_valid to mode_changing, the revised RTL and testbench were run again in Icarus Verilog 12.0 on 11/6/2026, and all five checks pass.

Table 7.4: RTL Verification Scenarios and Results

Scenario	Stimulus	Expected	Result (11/6/2026)
S1 Clean rail	48 samples at the golden value ± 5 codes	No alert	Pass
S2 Mode transition	mode_changing high, op_mode 01 → 10, 64 samples at 0x500 (768 codes low)	No alert while blanked	Pass
S2b Settle in new mode	mode_changing low, 304 clean samples (longer than BLANK_CYCLES + N)	No alert	Pass
S3 Injected anomaly	138 samples at 0x500	Alert asserted; deviation = 768	Pass (deviation = 768)
S4 Recovery	138 clean samples	Alert cleared	Pass

A second, directed testbench, tb_latency (Appendix D), was added on 11/6/2026 to measure the response rather than only check it. Its results, in clock cycles from the first anomalous sample, are:

• a step of 768 codes below the golden value raises alert after 6 cycles, and a step of 100 codes after 13;

• a step of 60 codes, just above THRESHOLD, raises alert after 18 cycles, because the mean moves only 60/16 = 3.75 codes per sample and needs 14 samples to cross the threshold before the 4 consecutive comparisons can complete;

• a step of exactly 50 codes never raises alert. The threshold test is strictly greater-than, and this boundary will be documented with the golden-profile tolerances;

• after a sustained anomaly, alert clears 16 cycles after the rail returns to the golden value;

• if op_mode changes without mode_changing and the rail is anomalous at once, alert is held off for the 256-cycle blanking window and asserts 261 cycles after the change. This confirms the counter works as a backstop on its own.

Coverage gaps and next tests. Because all four golden values are the same placeholder, the tests do not yet show that the correct golden value is selected for each mode. The next testbench revision will use distinct values per mode. It will also add a persistence break (three anomalous comparisons, then one in range, which must not alert), a reset during an anomaly, and a regression against a golden model in Python that replays captured prototype samples. Unit testbenches for each sub-module will be added before the ASIC synthesis run, so that a gate-level mismatch can be traced to one module.

7.1.8 Known RTL Issues and Planned Changes

A review of the RTL for this report found the following items. None affects the simulation results above, and each is scheduled before the Verilog ≥ 80% milestone (11/27/2026):

• Blocking assignment in a clocked block. abs_diff is assigned with = inside the clocked always block of comparison_logic. It simulates and synthesizes correctly, but it mixes blocking and non-blocking assignments, which lint tools (including Genus) flag. It will become a separate combinational signal.

• One-clock exposure at a mode edge. The blanking counter is loaded on the clock edge at which op_mode changes, so it takes effect on the next clock. If mode_changing is low on that edge, one comparison runs against the new golden value before blanking starts. With the firmware rule (mode_changing raised before op_mode changes) this cannot happen, but adding the op_mode change itself to the blanking condition closes it in hardware.

• Hard-coded counter widths. fill_count is 5 bits (enough for N ≤ 16) and the blanking counter is 9 bits (BLANK_CYCLES ≤ 511), independent of the parameters. Both will be sized from the parameters, so that changing N or BLANK_CYCLES cannot silently overflow them.

• Combinational ROM at time zero. golden_store uses a plain always @(*) block. In simulation such a block does not run until one of its inputs changes, so a testbench that sets op_mode before time zero sees an unknown golden value. This was found while writing tb_latency. It does not affect hardware, and it will be rewritten as a continuous assignment (or always_comb).

• Placeholder golden values. All four entries are 0x800. Besides making mode selection untestable, identical constants let synthesis remove the ROM and the op_mode inputs entirely, which would make the ASIC area and timing results

meaningless. Distinct values, first estimated and later measured, are required before synthesis.

• Mode encoding. op_mode will be widened to 3 bits for the advanced goal's five states (Section 7.1.4).

7.2 MCU Design

7.2.1 Responsibilities

The nRF52832 firmware is the "brain and responder" of NG PULSE. The FPGA classifies; the MCU acquires, configures, decides how to respond, and reports. Its responsibilities are:

1. Boot sequencing. Hold the ESP32-C6 in reset, confirm the FPGA has configured, load detection parameters into it, then release the target. *(Basic goal.)*

2. Current acquisition. Sample the conditioned current-sense voltage on AIN0 at a fixed rate with the SAADC, and stream the samples to the FPGA. *(Basic goal.)*

3. Golden-profile management. Store golden values and thresholds in flash, and capture new characterization data on request. The basic goal stores the profile for one work mode plus the Not-in-Use state. The advanced goal extends this to all four work modes, with the MCU automatically selecting the profile that matches the mode the ESP32-C6 reports.

4. Flag handling and status. Receive the FPGA's error flag, log and count it, and show it on a status LED. *(Basic goal.)*

5. Attack response (policy FSM): automatic shutdown of the ESP32-C6. Escalate repeated or severe flags to a target reset and, in Rev 2, to removing its power. This is a stretch goal (D&C Section 2.1, "Demonstrate a protective response in which the NG PULSE error flag can initiate an automatic shutdown of the ESP32-C"). The basic and advanced designs stop at item 4.

6. Telemetry over Bluetooth Low Energy (BLE). Report the ESP32-C6's operating mode and NG PULSE's normal, abnormal or error status to a mobile device. This is a stretch goal. UART logging for bench use and the watchdog and FPGA health checks are not part of the D&C goals and are supporting functions.

Table 7.5 shows which D&C goal tier each firmware feature in this chapter serves.

Table 7.5: Firmware Features by Goal Tier

Firmware feature	Goal tier	D&C reference
Current acquisition (SAADC) and stream to the FPGA	Basic	Basic objective: MCU provides the FPGA with ADC values

One golden profile plus Not-in-Use state, stored on the MCU	Basic	Basic objectives: characterize one work mode; store the profile on the Nordic MCU
ESP32-C6 reports its mode or Not-in-Use to the MCU	Basic	Basic objective
Error flag received, logged, shown on an LED	Basic	Basic goal: assert an error flag
Four work modes plus Not-in-Use: profiles for all stored	Advanced	Advanced objectives
Automatic profile selection from the reported mode	Advanced	Advanced goal and objective
Continuous monitoring across mode transitions (mode_changing signal)	Advanced	Advanced goal
Automatic ESP32-C6 shutdown (reset, power-FET kill, escalation)	Stretch	Stretch goal and objective
BLE status and mode reporting	Stretch	Stretch goal and objective
Single custom PCB for sensing, MCU and FPGA	Stretch	Stretch goal and objective

This section describes the planned firmware structure, interfaces and timing. The firmware is in the design stage. Code will be placed in Appendix D as it is written.

7.2.2 Firmware Architecture

The firmware is built on the nRF Connect SDK (Zephyr RTOS) (Section 3.2.3). Timing-critical sampling is done entirely in hardware: TIMER → PPI → SAADC → EasyDMA, then the SPI master. Everything else runs in Zephyr threads ordered by priority (Table 7.6). Figure 7.1 shows the modules and the data that flows between them.

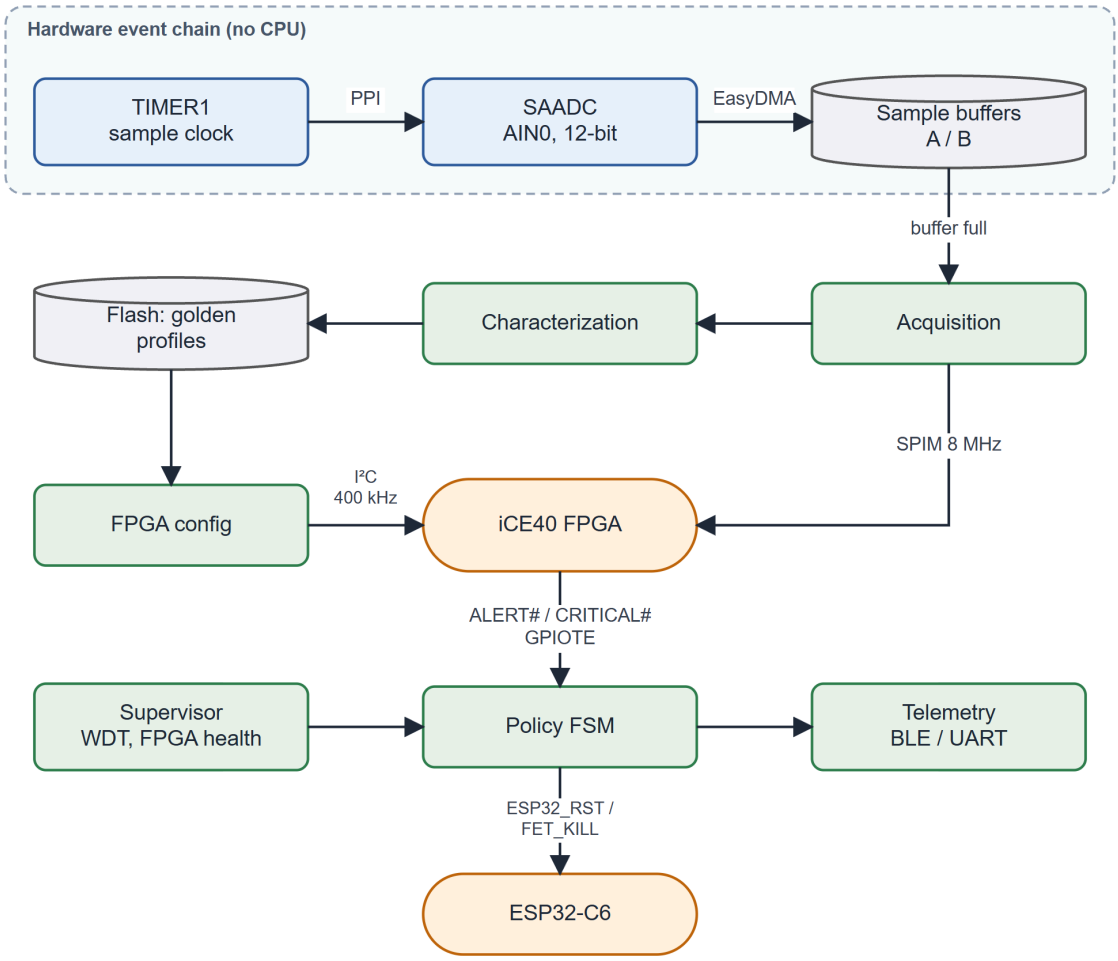


Figure 7.1: nRF52832 Firmware Module Diagram

Table 7.6: Firmware Execution Contexts

Context	Priority	Work
GPIOTE interrupt (ALERT#, CRITICAL#)	Highest	Timestamp the event and post it to the policy queue
SAADC / SPIM interrupts	High	Swap DMA buffers and start the SPI transfer of the completed buffer
Policy thread	High	Escalation state machine, target reset, FET control
Acquisition thread	Medium	Buffer bookkeeping and characterization statistics
Supervisor thread	Medium	Watchdog feed, FPGA status poll every 100 ms

Telemetry thread	Low	BLE notifications and UART log
BLE controller	Managed by the SoftDevice Controller	Radio events (may pre-empt application code for short periods)

The radio controller can pre-empt the CPU, but it cannot delay sample timing, because sampling never involves the CPU. It can delay only the buffer-swap interrupt. That delay is absorbed by double buffering (Section 7.2.4).

7.2.3 Boot and Initialization Sequence

The monitor must be watching before the target runs, so the boot sequence is ordered (Figure 7.2):

1. Power-on. ESP32_RST (P0.14) is configured as an output and driven low first, holding the ESP32-C6 in reset.

2. Clocks and watchdog. The high-frequency crystal clock starts, and the watchdog timer is enabled.

3. Load the configuration. The golden-profile record is read from flash and its CRC-32 is checked. If it is missing or corrupt, the monitor enters a fault state: the target stays in reset, the LED blinks an error code, and UART and BLE report "NO PROFILE".

4. Wait for the FPGA. The firmware pulses CRESET_B, then waits up to 500 ms for CDONE to go high (the iCE40 has loaded its bitstream from SPI flash). On a timeout it retries once, then enters the fault state.

5. Configure the FPGA over I²C. Write the golden values for every mode, the thresholds, the window size, the persistence count and the blanking length to the FPGA register map (Table 7.8). Read every register back and compare.

6. Start acquisition. Start the TIMER → SAADC chain and the SPI stream, then confirm through the FPGA STATUS register that samples are arriving.

7. Release the target. Drive ESP32_RST high. The FPGA blanks comparisons until the ESP32-C6 lowers mode_changing for its first workload.

8. Monitor. Enter the policy FSM in the MONITOR state.

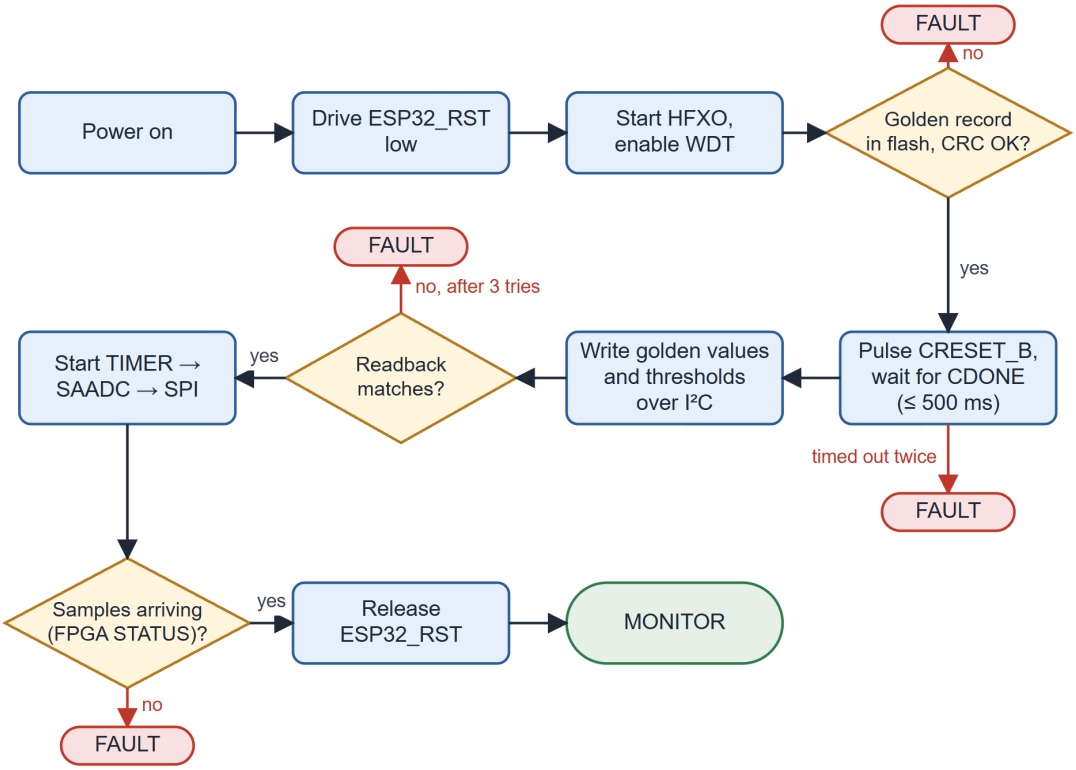


Figure 7.2: Boot Sequence Flowchart

The fault state is deliberately fail-closed: if any boot check fails, the ESP32-C6 stays in reset and is never allowed to run unmonitored. A security monitor that silently lets the target run when it cannot watch it would give a false sense of protection. The trade-off is availability: a monitor fault also stops the target. That is acceptable for the prototype and will be revisited if the sponsor's application needs it.

7.2.4 Current Acquisition (SAADC)

Configuration. The conditioned current-sense output (0–3.3 V, from the current-sense board via J3 pin 3) connects to AIN0 (P0.02). The SAADC is set up as follows [4]:

• single-ended input on AIN0;

• 12-bit resolution;

• internal 0.6 V reference with gain 1/6, for a full-scale input of 3.6 V. This leaves headroom above the 3.3 V maximum amplifier output, and gives 3.6 V / 4096 ≈ 0.88 mV per LSB;

• acquisition time 3 µs, which is allowed because AIN0 is driven by a low-impedance op-amp output. With the 2 µs conversion time this permits the full 200 kSPS;

• oversampling disabled, because the FPGA's windowed mean already averages;

• conversions triggered by a TIMER COMPARE event through a PPI channel, with no CPU involvement and therefore no software jitter.

Sample rate. The rate is a firmware constant, $f_s \leq 200$ kSPS. The nominal value will be set from the golden-profile characterization data (Section 7.2.9). If the four ESP32-C6 modes separate cleanly at lower rates, a lower f_s reduces SPI traffic and power. If more bandwidth is needed than 200 kSPS allows, the documented fallback is an external ADC or the STM32G4 alternative (Section 3.2.1).

Buffering. EasyDMA writes results into one of two RAM buffers of B samples each (B = 256 is proposed; each sample takes 16 bits in RAM). When a buffer completes, the SAADC END event fires. The interrupt handler points the SAADC at the other buffer and starts an SPI transfer of the completed one. At 200 kSPS a 256-sample buffer fills in 1.28 ms, which is far longer than any expected radio pre-emption. A missed swap is detected by checking a sequence counter and reported as an acquisition fault.

7.2.5 Sample Transfer to the FPGA (SPI)

The Rev 1 schematic leaves the nRF-to-FPGA sample transport undecided (I²C, SPI or a 12-bit parallel bus). The required data rate settles it:

• At 200 kSPS with 12-bit samples, the payload is 2.4 Mbit/s. Sent as 16-bit SPI frames, it is 3.2 Mbit/s.

• The nRF52832's I²C master supports at most 400 kbit/s. Counting address, acknowledge and framing overhead, I²C could carry only about 15 kSPS, roughly 7% of the required rate.

• A 12-bit parallel bus would cost twelve or more GPIOs on each side, plus a strobe, and the nRF would have to drive it from software or PPI tricks.

• The nRF52832 SPI master (SPIM) runs at up to 8 Mbit/s with EasyDMA. That carries the full 200 kSPS stream at 40% bus utilization using four signals.

Design decision (proposed): SPI carries samples, and I²C carries configuration and status. The FPGA implements an SPI target (slave) receiver in front of the ping-pong buffer. Table 7.7 gives the proposed frame format.

Table 7.7: Proposed SPI Sample Stream Format

Item	Value
Role	nRF52832 = SPI controller; iCE40 = SPI target
Clock / mode	8 MHz, SPI mode 0 (CPOL = 0, CPHA = 0), MSB first
Frame	16 bits: [15:14] sequence counter, [13:12] reserved (0), [11:0] ADC sample
Chip select	Asserted for one DMA buffer (B frames), deasserted between buffers
Throughput margin	3.2 Mbit/s needed at 200 kSPS of 8 Mbit/s available
Error detection	FPGA flags a sequence-counter gap in its STATUS register

Resolution is preserved end to end. Each 16-bit frame carries the complete 12-bit SAADC result in bits [11:0], so the FPGA receives exactly the same 12-bit samples the ADC produced. The SPI receiver hands them to the ping-pong buffer and feature window unchanged, and the windowed mean and comparisons remain 12-bit, as in the current RTL (WIDTH = 12). SPI changes only how samples travel between the chips, not their width.

The 2-bit sequence counter lets the FPGA detect dropped frames. Because the FPGA's clock is independent of the SPI clock, its receiver must synchronize SCK and CS into the 12 MHz/96 MHz FPGA clock domain. At the 96 MHz PLL output an 8 MHz SCK gives 12 samples per bit, which is adequate. [Coordinate with Sean: this replaces the ADC_IN[11:0] placeholder in the RTL top level with an SPI receiver.]

7.2.6 FPGA Configuration Interface (I²C)

At boot, and whenever the configuration changes, the nRF52832 (I²C controller, TWIM, 400 kHz) writes the detection parameters into a register file in the FPGA (I²C target). SDA and SCL are on P0.07 and P0.18 (moved from P0.26 and P0.27 for Rev 2, because the MDBT42Q specification limits P0.22–P0.30 to low-drive signals at 10 kHz or less) with 4.7 kΩ pull-ups (R7, R8).

Table 7.8: Proposed FPGA Register Map (7-bit target address 0x42, proposed)

Address	Name	Access	Contents
0x00	ID	R	Design ID and version (constant)
0x01	CONTROL	R/W	bit0 ENABLE, bit1 SOFT_RESET, bit2 CLEAR_FLAGS
0x02	STATUS	R	bit0 READY, bit1 SAMPLES_OK, bit2 ALERT, bit3 CRITICAL, bit4 SEQ_ERROR, bit5 MODE_CHANGING, bits[7:6] current op_mode
0x10–0x19	GOLDEN[0..4]	R/W	12-bit golden mean per mode, 2 bytes each. Basic goal: one work mode + Not-in-Use. Advanced goal: four work modes + Not-in-Use
0x20–0x21	THRESH_MINOR	R/W	12-bit minor (ALERT#) deviation threshold
0x22–0x23	THRESH_MAJOR	R/W	12-bit major (CRITICAL#) deviation threshold
0x24	WINDOW_LOG2	R/W	log2 of window size N (4–10, i.e., 16–1,024 samples)
0x25	PERSIST	R/W	Consecutive anomalous windows required (M)
0x26–0x27	BLANK	R/W	Blanking cycles after a mode change
0x30–0x31	DEVIATION	R	Latest absolute deviation, for telemetry

The firmware writes the registers in ascending order, reads each one back, and sets ENABLE only when every register matches. A NAK or mismatch is retried up to three times before the fault state. During monitoring, the supervisor thread

polls STATUS every 100 ms, which checks that the FPGA is alive and that samples are arriving.

On the ASIC track the golden values are hard-coded and I²C is excluded (Section 7.4). The register map applies only to the FPGA prototype.

7.2.7 Operating-Mode Reporting

The monitor needs to know which golden profile applies, and it must not mistake a legitimate change of workload for an attack. When the ESP32-C6 moves from one operating level to another, its supply current moves too, and a comparison made during that move would raise a false flag. The ESP32-C6 firmware therefore drives three GPIO lines to the monitor:

• op_mode[1:0]: the active work mode, encoded in 2 bits (four modes);

• mode_changing: 1 bit that is high while the ESP32-C6 is switching modes and low once the new workload is running and the rail has settled.

The FPGA uses op_mode to select the golden value and uses mode_changing to blank its comparisons, so no flag can be raised during a transition. The nRF52832 reads the same lines as inputs so it can select the matching stored profile, which is the advanced-goal behaviour (automatic profile selection and continuous monitoring across transitions). It can also log the mode with each event, and report it over BLE (stretch goal). The signals are summarized in Table 7.9.

Table 7.9: Mode Signals from the ESP32-C6

Signal	Width	Meaning
op_mode[1:0]	2 bits	Selects one of four work modes; mode names to be assigned (the current RTL placeholders are idle, compute, crypto, radio)
mode_changing	1 bit	1 = the ESP32-C6 is changing modes, so comparisons are blanked; 0 = steady state, so comparisons are enabled

The RTL now takes mode_changing directly (Section 7.1). It replaced the earlier active-high mode_valid strobe, which was its inverse (mode_valid = ~mode_changing), because it gives the ESP32-C6 firmware a clearer rule: raise it before touching the workload, lower it afterwards. The firmware should raise it before changing op_mode and lower it only after the new workload has settled, so that the FPGA never sees the new mode number alongside an unsettled rail.

7.2.8 Flag Handling and Attack Response (Policy State Machine)

ALERT# and CRITICAL# from the FPGA are active-low lines to P0.11 and P0.12. They are configured as GPIOTE inputs that trigger on the falling edge. The interrupt handler only timestamps the event and queues it; the policy thread then advances the state machine in Figure 7.3.

The response is split by goal tier. Flag handling (basic goal) is the part the D&C requires of every build:

• Tier 1 (any ALERT# or CRITICAL#): basic goal. Log the event, increment the alert counter, and show it on the status LED. This is the "assert an error flag" behaviour of the basic goal.

Automatic shutdown of the ESP32-C6 is a stretch goal (D&C Section 2.1). The escalation below is therefore designed but is a stretch feature, and the design remains valid if only Tier 1 is built:

• Tier 2 (stretch goal): N consecutive ALERT# events without a quiet period, default N = 3. Reset the ESP32-C6.

• Tier 3 (stretch goal): any CRITICAL#. Reset immediately.

• Tier 4 (stretch goal): M target resets without recovery, default M = 3. Assert FET_KILL to remove target power. Recovery needs a physical power cycle. In Rev 1, Q1 is not populated, so Tier 4 holds the target in reset instead.

BLE notifications for each tier are a separate stretch goal (Section 7.2.10).

A quiet period of T_clear (proposed 10 s) with no events clears the alert counter. A sustained period of normal operation clears the reset counter.

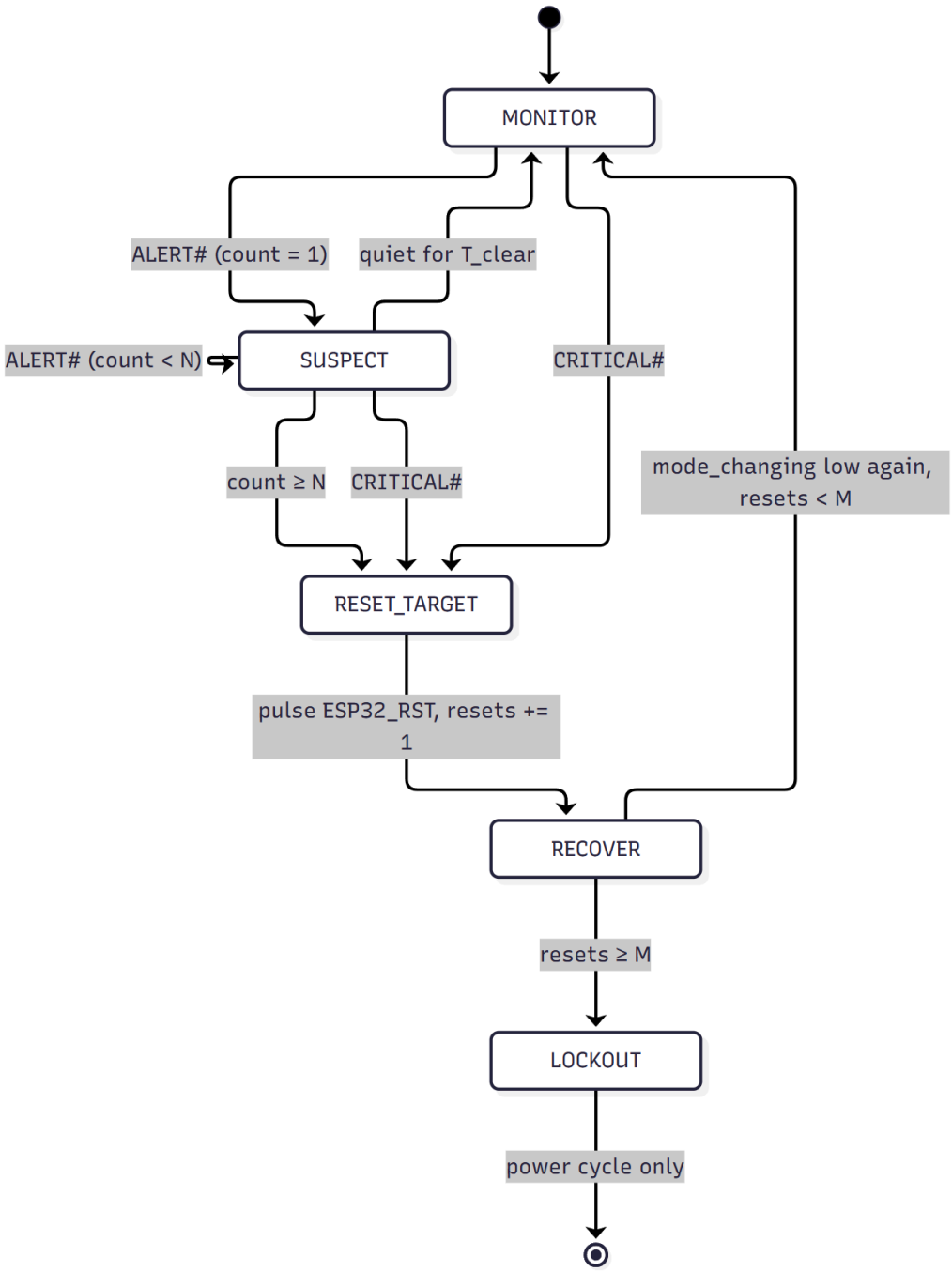


Figure 7.3: Policy State Machine (the MONITOR and SUSPECT states are the basic goal; every transition into RESET_TARGET, RECOVER and LOCKOUT is the shutdown stretch goal)

Reset timing (stretch goal). ESP32_RST drives the ESP32-C6 enable (EN/CHIP_PU) input low for 10 ms. That is well above the 50 µs minimum low time (t_RST, ESP32-C6 datasheet Table 2-15) [6]. After release, the FPGA blanks comparisons until the target lowers mode_changing again, so the reboot's current transient does not cause a new alert.

Latency against the specifications. The detection path is: window fill, then M persistent windows, then the FPGA flag, then the GPIOTE interrupt, then the reset.

• At f_s = 200 kSPS with N = 1,024 and M = 4, the worst-case time from a sustained anomaly to ALERT# is about 4 × 5.12 ms ≈ 20 ms. Firmware reaction adds less than 1 ms.

• This is well inside the ≤ 1 s detection-latency requirement and the < 0.1 s instantaneous overcurrent shutoff (Table 2.1).

• In Rev 1, removing power requires the U2 current limiter on the current-sense board, because the FET kill switch is not populated until Rev 2. The firmware can only reset the target. This matters for the overcurrent shutoff specifications in Table 2.1, which depend on the stretch-goal shutdown path.

7.2.9 Golden-Profile Characterization and Storage

Capture. In characterization mode, entered by a UART or BLE command (or a jumper), the policy FSM is disabled. For each operating mode the ESP32-C6 runs a stable workload and drives its mode lines (one mode for the basic goal, all four for the advanced goal). The nRF captures samples only while mode_changing is low, so transition transients are excluded. The 64 KB RAM can't hold long full-rate recordings, and UART at 1 Mbaud or BLE can't stream 200 kSPS continuously. So capture works two ways:

1. On-device statistics: a running sum, sum of squares, minimum and maximum over 1,024 or more samples per mode (the D&C's basic objective). These give the mean and the acceptable deviation directly.

2. Burst capture: about 16,384 samples (32 KB) are recorded at full rate and then dumped over UART or BLE, so the waveform can be analyzed offline. Repeated sessions cover temperature and supply variation.

Storage. Golden profiles are kept as one versioned record in Zephyr's settings/NVS flash storage. The record holds a version number, the number of modes, a 16-bit mean, allowed deviation and minor and major thresholds for each of up to five modes, the window size (as a power of two), the persistence count, the blanking length and a CRC-32. The record structure is listed in Appendix D.

The same values are pasted into the ASIC's hard-coded golden store, so the simulated ASIC uses golden data measured on real hardware.

Security of updates. The golden profile is deliberately immutable during monitoring, because an adaptive baseline can be walked by a slow-drift attack. For the same reason, updating it is a privileged operation. Characterization and profile writes are allowed only in characterization mode, which requires local action (jumper or UART). Over BLE they require an LE Secure Connections bonded connection. In demo configuration, BLE profile writes are disabled.

7.2.10 Telemetry (BLE) — Stretch Goal

Reporting the ESP32-C6's operating mode and NG PULSE's normal, abnormal or error status to a mobile device is a stretch goal (D&C Section 2.1 and 2.2). The basic and advanced designs do not depend on it. If built, a custom GATT service exposes:

• a read/notify Status characteristic: monitor state, current mode, alert and reset counters, last deviation;

• a notify Event characteristic: timestamped ALERT#, CRITICAL#, reset and lockout events;

• a write-protected Control characteristic: clear counters, enter characterization mode when permitted.

The same information is printed on the UART log for bench use. BLE work is scheduled at the lowest application priority and never blocks the policy thread.

7.2.11 Error and Status Handling

Table 7.10 lists each fault the firmware detects, how it is detected and the response.

Table 7.10: Fault Handling

Fault	Detection	Response
Missing or corrupt golden record	CRC check at boot	Fault state; target stays in reset
FPGA not configured	CDONE low after timeout	One reconfiguration retry, then fault state
I²C NAK or readback mismatch	TWIM error or compare failure	Three retries, then fault state
FPGA not receiving samples / sequence gap	STATUS poll every 100 ms	Restart the acquisition chain; report
Firmware hang	Watchdog (proposed 2 s)	MCU reset; target held in reset during reboot
Supply brown-out	Power-fail comparator	Log the event, hold the target in reset

A status LED shows the states: steady for monitoring, slow blink for characterization mode, fast blink for a fault.

7.2.12 MCU Pin Assignment

Table 7.11 lists the nRF52832 pin assignment.

Table 7.11: nRF52832 Pin Assignment (P0.02 and P0.11–P0.14, as in the Rev 1 schematic; I²C moved to P0.07/P0.18 for Rev 2; the rest proposed)

GPIO	Signal	Direction	Notes
P0.02 / AIN0	V_SENSE	In (analog)	From the current-sense board, J3 pin 3
P0.11	ALERT_N	In	GPIOTE, falling edge
P0.12	CRITICAL_N	In	GPIOTE, falling edge
P0.13	FET_KILL	Out	Rev 2 only (Q1 not populated in Rev 1)
P0.14	ESP32_RST	Out	Active low; driven low first at boot
P0.07	SDA	Bidirectional	4.7 kΩ pull-up
P0.18	SCL	Out	4.7 kΩ pull-up
P0.15	SPI_SCK	Out	Proposed: sample stream
P0.16	SPI_MOSI	Out	Proposed
P0.17	SPI_CS_N	Out	Proposed
P0.19	FPGA_CDONE	In	Proposed: configuration-done monitor
P0.20	FPGA_CRESET_N	Out (open drain)	Proposed: lets the MCU reconfigure the FPGA
P0.22–P0.23	OP_MODE[1:0]	In	Proposed: mirror of the ESP32 mode lines
P0.24	MODE_CHANGING	In	Proposed: high while the ESP32 switches modes
P0.06 / P0.08	UART TX / RX	Out / In	Proposed: bench log
SWDIO / SWDCLK	Debug	—	Dedicated pins on the nRF52832, not shared with any GPIO (MDBT42Q pads 36 and 37)

On the nRF52832, SWDIO and SWDCLK are dedicated pins rather than GPIOs. The MDBT42Q brings them out on their own pads (36 and 37), so the SWD/I²C overlap noted on the Rev 1 schematic does not apply. The analog-capable pins

(P0.02–P0.05, P0.28–P0.31) and the NFC pins (P0.09–P0.10) are kept free of digital signals.

7.3 ESP32-C Firmware

[TO DO: Describe workload/operating-mode signaling.]

7.4 ASIC Flow

At Northrop Grumman's request (10/21/2026), the detection logic is also implemented as a digital ASIC in simulation. The ASIC is not fabricated. It is carried through the Cadence tools provided by the DRACO Laboratory (Sections 3.3.3 and 3.3.4), using the same RTL as the FPGA (Section 7.1), so that the team can report a standard-cell implementation of the comparison logic alongside the working FPGA prototype. This section describes the planned flow, what goes into it, what comes out, and how its scope maps onto the D&C goal tiers.

7.4.1 ASIC Scope by Goal Tier

• Basic goal: the Cadence digital design of the comparison logic and the error flag. For the ASIC this means the windowed mean, the comparison, the persistence filter and the alert output, synthesized to standard cells and checked by gate-level simulation against the RTL.

• Advanced goal: the golden-profile store for four work modes plus Not-in-Use, and the mode-transition blanking, are in the same RTL and are synthesized with it. No separate ASIC work is needed beyond distinct golden values and the 3-bit op_mode (Section 7.1.8).

• Stretch goals (D&C): ADC functionality, memory storage and current capture on the ASIC (a 4-bit digital ADC first, then 12-bit). The current RTL has none of these. The ADC and the analog current capture are mixed-signal blocks that the digital standard-cell flow cannot produce on its own.

The sponsor excluded the ADC, I²C and BLE from the required ASIC scope. In the ASIC the 12-bit sample arrives on the parallel adc_in port, driven by the testbench, rather than through the SPI receiver used on the FPGA. The SPI receiver, synchronizers and register interface are FPGA wrapper logic (Section 7.1.6) and are not part of the ASIC top level.

7.4.2 Flow Overview

The flow follows the RTL-to-GDSII structure documented by the earlier Northrop-sponsored NGRAD project (Section 3.3.4): RTL simulation, logic synthesis in Genus, gate-level simulation, and then, if the schedule allows, physical implementation in Innovus. Table 7.12 lists the stages, the files each one needs and produces, and whether it is committed or an extension.

Table 7.12: ASIC Flow Stages

Stage	Tool	Inputs	Outputs	Scope
1. RTL simulation and lint	Icarus Verilog / Vivado XSim (done); Cadence Xcelium if available	RTL, tb_ng_pulse_top , tb_latency	Pass/fail log, waveforms, lint report	Committed (basic)
2. Logic synthesis	Cadence Genus	RTL, standard-cell timing library (.lib), timing constraints (SDC)	Gate-level netlist, area, timing and power reports	Committed (basic)
3. Gate-level simulation	Xcelium (or Icarus) with the library's cell models	Netlist, cell Verilog models, the same testbenches	Pass/fail log matching stage 1	Committed (basic)
4. Floorplan, placement, clock-tree synthesis and routing	Cadence Innovus	Netlist, cell and technology LEF, SDC	Routed design, post-route timing report, DEF	Extension if time allows
5. Physical verification and GDSII export	Innovus with the PDK's DRC and LVS decks	Routed design, PDK rule decks	GDSII, DRC and LVS reports	Extension if time allows
6. Post-route timing simulation	Xcelium with SDF back-annotation	Routed netlist, SDF delays	Pass/fail with real cell and wire delays	Extension if time allows

Stages 1 to 3 are the committed deliverable, because they show that the RTL maps to standard cells and that the mapped netlist behaves the same as the RTL. Stages 4 to 6 produce a physical layout. They are planned, but they depend on the DRACO setup and on the time left after the prototype work, and they are not presented as a project requirement.

7.4.3 Inputs Needed Before Synthesis

Technology library. Genus needs a standard-cell library with timing models, and Innovus needs the matching physical views and rule decks. The process design kit (PDK) has not yet been confirmed with the DRACO Laboratory. The two candidates are the educational PDK installed with the lab's Cadence tools and the open SkyWater SKY130 kit. The choice will be recorded with every result, because area and timing numbers are only meaningful for a stated library and corner.

Timing constraints. The SDC file defines the clock and the input and output timing. The constraints in Table 7.13 are proposed for the first run.

Table 7.13: Proposed ASIC Synthesis Constraints

Constraint	Proposed value	Reason
Primary clock	12 MHz (83.3 ns period)	Matches the prototype's FPGA reference clock (Section 3.2.4)
Stretch clock target	100 MHz (10 ns period)	Matches the testbench clock; shows the timing margin
Input and output delay	20% of the clock period	A conservative placeholder until a surrounding system is defined
Reset	rst_n treated as asynchronous; reset de-assertion synchronized in the wrapper	Avoids a timing arc on the asynchronous reset
Operating corner	Library typical corner first, then the slow corner	Report typical results and check worst-case slack

RTL clean-up. The items in Section 7.1.8 are fixed first. Two of them matter more for the ASIC than for the FPGA:

• the golden values must be distinct, or synthesis folds golden_store into a single constant and removes the op_mode inputs, so the reported area would leave out the advanced-goal logic;

• the blocking assignment in comparison_logic is changed to a separate combinational signal, so that Genus elaborates the design without lint warnings.

Memory. The 16-sample window is 192 flip-flops, which a standard-cell flow handles without any memory macro. A 1,024-sample ping-pong buffer, if the feature extractor is ever moved into the ASIC, would need an SRAM macro from a memory compiler for the chosen PDK. That falls under the stretch goal's memory-storage item and is not in the committed scope.

7.4.4 Verification of the Synthesized Design

The ASIC is verified with the same testbenches as the RTL, so the gate-level results can be compared line for line with Table 7.4. The pass criteria are:

• all five checks of tb_ng_pulse_top pass on the netlist;

• tb_latency reports the same latencies, in clock cycles, as the RTL (6, 13, 18 and 16 cycles, and 261 cycles for the blanking backstop);

• no unknown (X) values appear on alert or deviation after reset.

When golden profiles have been measured on the prototype, recorded sample streams from the physical board will be replayed through both the RTL and the netlist. This lets the report compare the FPGA and the ASIC on identical data, while keeping the distinction required in Section 9.7 between measured prototype results and simulated or synthesized ASIC results.

7.4.5 Expected Results and Reporting

The design is small. It has about 250 flip-flops, a 16-bit adder and subtractor, two 12-bit comparators and a few counters, so it is expected to synthesize to roughly one to two thousand standard cells and to meet a 12 MHz clock with a wide margin. These are expectations, not results. The midterm report gives no ASIC numbers because synthesis has not been run.

The final reports will include, for the stated PDK and corner:

• the cell area and cell count from Genus, broken down by module;

• the worst negative slack at 12 MHz and the highest clock frequency that meets timing;

• an estimate of dynamic and leakage power from Genus, with its switching-activity assumption stated;

• the gate-level simulation logs;

• if stages 4 to 6 are completed, the post-route timing, the DRC and LVS summaries and an image of the final layout.

Debug screenshots of the tools stay out of the body of the report, in line with the guideline.

7.4.6 Schedule

The ASIC work follows the D&C milestones. Verilog major functions are due by 11/13/2026 and at least 80% of the Verilog by 11/27/2026. Synthesis scripts and a first Genus run on the placeholder-free RTL are planned before the end of SD1, once DRACO tool access and the PDK are confirmed. The D&C places "RTL → ASIC" in SD2 (milestone 1/30/2027), together with golden-profile acquisition and the full system test. Gate-level verification with measured golden values belongs to that milestone, and physical implementation, if it is attempted, follows it.

Chapter 8 – Risk Assessment

8.1 Technical Risks and Open Decisions

- The shutdown mechanism and interface to the protected system remain a stretch design decision.
- This project and its specifications are determined in large part by the team at Northrop Grumman and so they are subject to change depending on their direction.

8.2 High-Risk Components

This section examines the three hardware and software elements that the team considers most likely to fail or to disrupt the demonstration. Each is discussed in terms of its failure modes, its impact on the project, its risk level, its mitigation strategy and a target resolution date. Likelihood and impact are rated Low, Medium or High, and the overall risk level is the team's judgment of the two together. Table 8.1 summarizes the three risks, and Sections 8.2.1 to 8.2.3 give the detail. The target dates follow the milestones in the Divide and Conquer plan: hardware selection on November 6, 2026, a PCB layout ready to order on November 27, 2026, assembly on January 16, 2027, and current-capture testing and integration later in January 2027.

Table 8.1: Summary of the Three Highest-Risk Components

Risk	Component	Likelihood / Impact	Risk level	Target resolution
R1	Golden profile	Medium / High	High	Jan 2027 (current-capture testing)
R2	USB-C connectors	Medium / Medium	Medium	Before Nov 27 order or at assembly
R3	Power path and sensing	Medium / High	High	Before first power-up, Jan 2027

8.2.1 Golden Profile Mismatch (R1)

NG PULSE decides whether the ESP32 is behaving normally by comparing live current samples against a stored golden profile. The profile is captured from known-good operation and loaded into the FPGA detector as reference values, thresholds, window length and persistence count. If the stored profile does not represent what the board sees in operation, the detector has no valid reference. In the worst case it reports Critical or Alert permanently, even when the ESP32 is running normally.

Failure modes. A mismatch can arise in several ways. The profile may have been measured with different hardware than the final board, for example before the shunt, amplifier gain or measurement boundary was finalized. Temperature and warm-up state change shunt resistance, amplifier offset and regulator behavior, so a profile captured on a cold board can drift out of tolerance as the board warms. A different ESP32 unit, firmware version, supply source or clock setting can also shift the baseline. A profile that is captured with too few samples or too few repetitions may represent one moment instead of the typical behavior. Finally, an incorrect or partially written configuration over I²C can leave the detector holding a valid-looking but wrong profile.

Impact. A permanently asserted alert makes the monitor unusable. The core demonstrable functions of detecting abnormal behavior and generating a flag

cannot be shown, and the false-positive rate and F1 score cannot be evaluated meaningfully. The failure is easy to see in testing. It does not damage any hardware, and it can be corrected without a board revision, so the impact is rated High on demonstration value but recoverable.

Mitigation. The fix is to remeasure the golden profile and flash a corrected one. The team will make this repeatable instead of one-off:

- capture profiles on the assembled board, with the same ESP32, firmware, supply and measurement boundary that will be used for the demonstration;
- allow a defined warm-up period and record the ambient and board temperature with each capture;
- capture multiple repetitions and compare them, setting thresholds with margin from the observed run-to-run variation instead of from a single capture;
- read the profile back over I²C and verify its CRC before enabling monitoring, so that a failed write is detected as a configuration fault instead of appearing as an anomaly;
- keep the capture, conversion and flashing procedure documented and scripted so that a new profile can be produced quickly when a part, setting or workload changes.

Target resolution. The remeasure-and-reflash procedure should be working by the current-capture testing milestone in January 2027, after the assembled board is available. The profile should be reviewed again whenever the shunt, gain, firmware or ESP32 workload changes.

Table 8.2: Risk Summary for Golden Profile Mismatch

Item	Detail
Component	Golden profile stored in the MCU and FPGA detector
Failure mode	Measured profile differs from the live signature; board constantly reports Critical or Alert
Likely causes	Different hardware or settings at capture time, temperature drift, too few samples or repetitions, failed configuration write
Impact	Detection cannot be demonstrated; metrics cannot be evaluated; no hardware damage
Likelihood / impact / level	Medium / High / High

Mitigation	Remeasure on the assembled board with warm-up, repeat captures, set thresholds with margin, verify with readback and CRC, then flash the new profile
Target resolution	[Team: confirm date, January 2027 current-capture testing]

8.2.2 USB-C Connector Durability (R2)

NG PULSE uses USB-C connectors between the computer and the board and between the board and the ESP32-C6. USB-C was chosen because it is bidirectional and universal, it is reversible, and it also supports UART communication. The tradeoff is mechanical. A USB-C receptacle is smaller and less rigid on the board than a USB-A connector, and the repeated insertion needed during development and testing puts stress on its solder joints and pads.

Failure modes. Repeated plugging and unplugging, or a cable pulling sideways while connected, can crack solder joints, lift pads or tear the receptacle from the board. The effect can be a complete failure, or it can be intermittent contact. Intermittent contact is the more dangerous case for this project. A momentary interruption in the power or ground path would show up as a sudden current change that the detector may correctly classify as an anomaly, so a mechanical fault could be mistaken for an attack or workload change. Because the receptacle is also part of the monitored power path, a poor joint adds contact resistance to the measurement.

Impact. A failed connector disables communication or power to the board or to the ESP32 until it is repaired. Repair of a lifted pad is difficult on a hand-assembled board and could require a replacement board. Intermittent faults can also corrupt test results and waste characterization time. Because the connector is a standard part and the failure does not damage the sensing circuit, the impact is rated Medium.

Mitigation. The team will reinforce the connectors and reduce the number of insertion cycles:

- apply non-conductive epoxy or putty around the base of each receptacle to carry mechanical load away from the solder joints, taking care that it does not enter the contacts or cover test points;
- choose a receptacle with through-hole mounting tabs or stronger shield anchors where the selected part allows it, since these carry more of the load than surface-mount pads alone;
- use short cables with strain relief, and avoid hanging the weight of the cable from the board;
- keep the board and the ESP32 connected during testing instead of unplugging them between runs, and use the programming header for repeated reflashing where possible;
- complete a simple plug-cycle check on the assembled board, with a continuity check after each set of cycles, before relying on the connector during characterization;
- keep spare connectors and, if budget allows, a spare assembled board.

Target resolution. The connector footprint and mounting choice should be resolved with the PCB layout before the November 27, 2026 order date. The epoxy or putty reinforcement and the plug-cycle check should be completed at assembly, in January 2027, before current-capture testing.

Table 8.3: Risk Summary for USB-C Connector Durability

Item	Detail
Component	USB-C receptacles (computer to board, board to ESP32)
Failure mode	Connector loosens, cracks solder joints or detaches after repeated plug and unplug cycles
Likely causes	Small, less rigid connector than USB-A; repeated insertion; side load on the cable
Impact	Loss of power or communication; intermittent faults that mimic anomalies; possible board rework
Likelihood / impact / level	Medium / Medium / Medium
Mitigation	Non-conductive epoxy or putty reinforcement, strong mounting tabs, strain relief, fewer insertion cycles, plug-cycle test, spare parts
Target resolution	[Team: confirm date, before Nov 27 order and at January assembly]

8.2.3 Power Injection and Component Damage (R3)

NG PULSE measures current, and its testing deliberately disturbs the supply to the monitored target. That makes the board's own components exposed to excess current or voltage during characterization. The planned sensing range of up to 20 A, the choice of shunt, and the disturbance injection method set how much stress the board must withstand. As noted in Chapter 4, the USB-C path cannot carry 20 A, so high-current characterization needs a separate bench supply, load, wiring, connectors and protection.

Failure modes. Injecting more current or voltage than a component is rated for can destroy it. The most exposed parts are the shunt resistor (continuous and pulse power rating), the current-sense amplifier (input and common-mode limits), the ADC input of the MCU (which must stay within the supply rails), and the regulators that supply the 3.3 V and 1.2 V rails. A calculation error, such as a wrong gain, a wrong shunt power rating or a wrong unit, can cause damage the first time power is applied. A wiring mistake or short in the test fixture can have the same effect. Damage can also be partial: a stressed amplifier or shunt may keep working but with a shifted offset or gain, which would silently invalidate the golden profile.

Impact. A damaged component can stop the board working, and replacing fine-pitch parts on a hand-assembled board takes time. Damage during the final testing window could also threaten the demonstration schedule. Because the

failure is both likely to happen during the riskiest phase of testing and costly to repair, the impact is rated High.

Mitigation. The team's main mitigation is to verify the design before applying power, and to keep spares in case a component fails anyway:

- check every rating calculation at least twice, and have a second team member review them independently: shunt voltage and power at the maximum current, amplifier gain against the ADC input range, common-mode limits at the supply voltage, regulator dissipation, and the current rating of connectors, traces and vias;
- reconcile the preliminary gain values and the 10 mΩ and 50 mΩ shunt figures before layout, as noted in Chapter 4;
- use a current-limited bench supply and a staged test plan, increasing current in steps from low values and watching temperature and output before each step;
- provide an accessible disconnect and protected test connections, and do not rely on the anomaly detector as the only protection against a short;
- run a power-on checklist after assembly: rail-to-ground resistance, supply voltages and the sensing path before any disturbance test;
- buy backup components for the parts most at risk (shunt, amplifier, regulators, connectors), and consider extra PCBs or a spare assembled board, within the sponsor budget limit of $1,000.

Target resolution. The calculations should be reviewed and the spare-parts list added to the bill of materials before the hardware selection milestone on November 6, 2026 and the PCB order on November 27, 2026. The staged test plan should be written before first power-up and the high-current tests in January 2027.

Table 8.4: Risk Summary for Power Injection and Component Damage

Item	Detail
Component	Shunt, current-sense amplifier, ADC input, regulators and power path
Failure mode	Components damaged or degraded by excess current or voltage during testing
Likely causes	Calculation error, wrong component rating, wiring or fixture fault, uncontrolled disturbance injection
Impact	Board failure or silent drift of the measurement; rework time; schedule delay
Likelihood / impact / level	Medium / High / High
Mitigation	Repeated independent checks of the math, current-limited supply, staged tests, accessible disconnect, spare components and spare boards

Target resolution	[Team: confirm date, calculations by Nov 6 / Nov 27, test plan before January testing]

8.2.4 Risk Comparison

The three risks differ in cause and in how they can be corrected. R1 is a software and calibration risk that can be corrected after the fact by remeasuring and reflashing. R2 is a mechanical risk that is cheap to address in the layout and at assembly. R3 is the only one that can destroy hardware, so it is mitigated before power is ever applied, and the spare-parts plan exists mainly for it. The two High risks (R1 and R3) are given the earliest review in the schedule.

Chapter 9 – System Fabrication/Prototype Construction

9.1 Prototype Architecture and Modular Design

NG PULSE will use a modular stacked-PCB architecture centered on a custom main board. The Nordic microcontroller, FPGA, and clock circuitry will be soldered directly to this board, together with their required support components and connectors. The main PCB will serve as the electrical interconnection and mechanical base for the system. Separate daughterboards will implement current capture and voltage regulation, while an ESP32-C6 development board will serve as the monitored target.

The daughterboards will mount vertically above the main PCB through pin headers. This arrangement is intended to reduce the overall footprint, provide clear functional boundaries, and permit individual modules to be removed during development. The 3.3 V regulator, 1.2 V regulator, current capture board, and ESP32-C6 target will all use this stacking approach. Header locations and board outlines will be coordinated during PCB layout to prevent physical interference and preserve access to USB-C connectors and debugging points.

The target will receive power through the capture board’s monitored USB-C path. Its operating-mode information will be transmitted to the Nordic MCU using one UART transmit signal. A shared electrical ground reference is also required for reliable UART communication; additional header contacts may be reserved for structural support rather than data exchange. The Nordic MCU will digitize the capture-board signal and interface with the FPGA, which will perform comparison logic and produce the system’s anomaly indication.

A separately designed digital ASIC will complement the FPGA implementation as a design-only deliverable. It will be developed in Cadence through RTL design, functional simulation, and synthesis, but will not be fabricated or mounted in the physical prototype.

9.2 Main PCB Design and Layout

9.2.1 Main Board Components

The main PCB will integrate the Nordic MCU, FPGA, clock source, power distribution connections, programming and debug interfaces, and daughterboard headers. Unlike a carrier board that hosts complete processing development modules, this design will place the MCU and FPGA devices directly onto the custom PCB. This decision will require attention to package footprints, solderability, local bypassing, signal pin assignments, and board-level bring-up provisions.

The MCU will receive the amplified analog current signal from the capture board at an ADC-capable input. It will also receive target mode information over UART and exchange the data needed by the FPGA comparison subsystem. The FPGA will be placed to facilitate short, direct interconnections with the MCU and the clock circuitry. The final clock frequency, FPGA pin assignments, and detailed programming interface will be established as the digital design and component selection are finalized.

9.2.2 Component Placement and Routing

Component placement will begin with the main ICs, clock circuit, power inputs, and daughterboard connectors. The layout will reserve adequate space around each package for decoupling capacitors and escape routing. FPGA and MCU supply bypass capacitors will be placed near their associated power pins, with low-impedance return paths to the ground system. Clock routing will be kept short and separated where practical from noisy switching nodes and high-current supply paths.

The analog ADC-IN trace from the capture daughterboard will be routed with attention to noise pickup from the FPGA, digital interfaces, and switching regulators. A continuous ground reference and deliberate return-current paths will be prioritized. Routing width, via count, copper area, and layer stackup will be selected based on the final current requirements and manufacturer capabilities rather than fixed in advance.

The board layout will also incorporate test access for major supply rails, ground, ADC-IN, UART, clock, and selected MCU–FPGA signals. These points will support continuity checks and staged commissioning before the complete system is powered. Final placement will be reviewed against component datasheets, PCB design rules, and assembly clearances.

9.2.3 Power and Clock Integration

The main PCB will receive regulated 3.3 V and 1.2 V power from separate daughterboards. The 3.3 V rail is intended for compatible control and sensing circuitry, including the Nordic MCU and current-sense amplifier, while the 1.2 V rail is intended for the FPGA core. Additional FPGA I/O supply requirements will be verified against the selected device before the PCB is finalized; a 1.2 V core

supply alone must not be assumed to power every FPGA bank or auxiliary function.

Clock circuitry will be implemented on the main board to support the FPGA and any other device requiring an external timing reference. The clock component, supply filtering, routing, and termination strategy will be selected according to the device specifications. The final layout will minimize coupling between clock lines and the sensitive analog acquisition path.

9.3 Daughterboard Design and Integration

9.3.1 Voltage Regulator Daughterboards

Two separate regulator PCBs will provide the required low-voltage rails. Each regulator board will accept the protected 5 V supply from the TPS2553 current-limited branch and provide its regulated output through the main-board header interface. The 3.3 V regulator design is based on the Texas Instruments TLV62569A adjustable buck converter, with a 4.7 µF input capacitor, 1 µH inductor, 22 µF output capacitor, and a 453 kΩ/100 kΩ feedback divider. Its nominal feedback calculation gives approximately 3.318 V. Component ratings and the physical switching-loop layout will be checked against the applicable TI recommendations.

The 1.2 V regulator will follow the same modular mounting concept, but its final device selection, passive values, and layout are not yet established in this chapter. The regulator boards will be designed with sufficient clearance for inductors and capacitors and with connectors rated for the expected supply current. Their switching nodes and high-current loops will be kept compact to reduce electromagnetic interference.

9.3.2 Current Capture Daughterboard

The current capture board will contain the USB-C power interfaces, high-side shunt resistor, INA180A4 current-sense amplifier, and TPS2553 current-limiting branch. The shunt will be located in series with the ESP32-C6 target's positive supply path, while the TPS2553 will protect NG PULSE's supporting electronics rather than the target power path. The amplifier output will connect to the Nordic ADC input through the stacked header interface.

The capture board layout will prioritize a short, controlled shunt-current path and appropriately routed sense connections. The USB-C connector locations will be chosen to preserve cable access after stacking. The mechanical design will also consider the insertion and removal forces imposed by USB-C cables, since relying only on slender pin headers for structural restraint may be inadequate.

9.3.3 ESP32-C6 Target Board

The ESP32-C6 target board will be mounted above the main PCB using a dedicated header arrangement. It will remain the device under observation and will not execute the NG PULSE comparison algorithm. Its USB-C power will pass

through the capture-board sensing path, allowing the monitoring hardware to measure supply-current variations associated with target operation.

The target-to-MCU information interface will consist of a single UART TX signal from the ESP32-C6 to a Nordic UART RX input, together with a common ground reference. The UART message format, logic-level compatibility, baud rate, and mode encoding will be finalized during firmware integration. Additional header positions may be included for alignment and mechanical stability, but these contacts will not be assigned electrical functions unless required by the final design.

9.4 Pin Header Interfaces and Interconnections

Pin headers will provide the removable electrical and mechanical connections between the main board and its daughterboards. Connector pitch, mating height, number of contacts, and orientation will be chosen during mechanical and PCB layout. Keying or clearly marked pin-1 indicators will be considered to reduce the risk of reversed installation. Power contacts will be assigned according to the required current and connector ratings, and unused contacts will not be assumed to provide electrical grounding.

Table 9.1: Pin Header Interfaces

Module	Preliminary interface signals	Integration notes
3.3 V regulator	Protected 5 V, GND, 3.3 V	Verify supply current and header rating
1.2 V regulator	Protected 5 V, GND, 1.2 V	Finalize regulator and FPGA rail requirements
Current capture	ADC-IN, GND, required power/control	Preserve USB-C access; confirm final pinout
ESP32-C6 target	UART TX → Nordic RX, GND	Other pins may provide mechanical support

The connector assignments above represent functional requirements, not final numbered pinouts. The completed schematic and PCB layout will define the exact pin numbering, signal direction, ground allocation, and connector footprint. A pinout review will be performed before fabrication to ensure that each daughterboard mates correctly with the main PCB.

9.5 Mechanical Assembly and Structural Support

The prototype will use a stacked arrangement in which daughterboards extend above the main PCB. Mechanical CAD models will be used to establish board outlines, mounting heights, connector alignment, and clearance envelopes. The assembly model will include the heights of the regulator inductors, USB-C connectors, FPGA package, and any components located beneath the daughterboards.

Although pin headers will provide the primary mating interface, additional standoffs or mounting hardware may be incorporated if needed to withstand handling and cable loads. This is particularly important for the capture and ESP32-C6 boards, where USB-C connections may generate lateral forces. Board-to-board spacing will also allow access for inspection, probing, and replacement without damaging adjacent components.

The final CAD assembly will be checked for interference between boards and for accessibility of programming connectors, status indicators, and test points. Mechanical dimensions and hardware specifications will be recorded once the layout is complete.

9.6 PCB Fabrication and Component Assembly

After schematic completion, each custom PCB will be laid out and subjected to electrical-rule and design-rule checks. Manufacturing files will include Gerber layers, drill files, board outlines, and any assembly documentation required by the selected PCB manufacturer. Footprints will be checked against component datasheets, and the bill of materials will be reviewed for package compatibility and component availability.

Fabrication will be followed by board inspection and component assembly. Because the MCU and FPGA will be mounted directly to the main PCB, their package requirements will influence the soldering method and inspection approach. Small leadless and fine-pitch packages may require stencil-applied solder paste and controlled reflow assembly. Each board will be inspected for solder bridges, missing parts, incorrect polarity, and connector alignment before power is applied.

The initial power-up sequence will be staged. Regulator daughterboards will first be checked independently for correct output voltage and abnormal current draw. The current capture board will then be inspected and tested for supply continuity and amplifier behavior. The main PCB will be powered only after the intended rails, ground connections, and header pinouts have been confirmed. The order and acceptance criteria for these checks will be documented in the system verification plan.

9.7 Digital ASIC Design and Virtual Implementation

In parallel with the physical FPGA-based prototype, NG PULSE will include a digital application-specific integrated circuit (ASIC) design deliverable. The ASIC will not be manufactured, packaged, or installed on the PCB assembly. Instead, the design will remain in the Cadence electronic design automation environment and will demonstrate a possible dedicated-logic implementation of the relevant digital processing functions.

The planned ASIC workflow will include RTL development, functional simulation, and logic synthesis. The RTL description will define the selected comparison-related digital functions and their interfaces. Functional simulation will be used to check the expected behavior for representative inputs, normal

operating cases, and selected abnormal conditions. Logic synthesis will translate the verified RTL into a technology-mapped or otherwise supported gate-level representation, depending on the available library and Cadence setup.

Simulation waveforms, synthesis reports, and design hierarchy views will be retained as evidence of the digital implementation. If available, area and timing estimates may be reported with their library and constraint assumptions clearly stated. These estimates will not be presented as measured silicon performance. Physical place-and-route, tape-out, fabrication, and laboratory ASIC testing are outside the planned scope.

The FPGA will remain the hardware platform for the working NG PULSE prototype. The Cadence ASIC will therefore be evaluated as a separate virtual design outcome, and any comparison between FPGA and ASIC implementations will distinguish simulated or synthesized results from physical prototype measurements.

9.8 Prototype Integration and Initial Inspection

System integration will proceed incrementally after the individual boards are fabricated and inspected. The regulator outputs will be verified before the main board is populated with all daughterboards. The MCU and FPGA will then be brought up with their required power rails and clock signals. Basic communication checks will confirm the MCU–FPGA interface and the UART mode-data link from the ESP32-C6.

The capture-board ADC output will be checked against known or controlled target operating conditions before the complete anomaly-detection workflow is exercised. Particular attention will be given to grounding, connector orientation, regulator noise, and the possibility of digital switching activity affecting the analog current measurement. Integration observations and corrective changes will be recorded for the final prototype documentation.

This chapter describes the planned construction approach rather than completed test outcomes. Final PCB renders, CAD drawings, fabrication photographs, as-built connector pinouts, and inspection records will be added after implementation. Detailed performance results and detection metrics will be reported separately in the verification and testing chapter.

Chapter 10 – System Testing and Evaluation

[TO DO: Target 5–10 pages. Not required for the midterm report. Cover hardware and software testing, performance evaluation, and overall integration. Text below is carried over from the Divide and Conquer.]

10.1 Preliminary Verification and Test Strategy

Verification will proceed from golden-profile acquisition through virtual RTL verification and finally to breadboard prototype testing. Normal workloads will be used to establish the reference set and evaluate false-positive behavior, while defined attack trials will be used to evaluate detection rate, latency, F1 score, and shutdown functionality. Processor benchmarks will be run with and without NG PULSE attached to determine the impact on computing efficiency. Current-sensing performance and signal quality will also be evaluated to ensure that the acquired current waveforms are suitable for reliable anomaly detection.

- Reference-waveform test: For all four (4) selected processes, collect and average 1024 current samples to establish each golden profile. Verify that a golden waveform is successfully acquired for each selected process.
- Efficiency test: Run a defined RISC-V benchmark with and without NG PULSE attached. Pass if there is no reduction in computing efficiency (0% efficiency loss).
- Latency test: Measure the time from a detectable attack/anomaly to assertion of the alert flag. Pass if the detection latency is ≤1 s.
- Detection-rate test: Run repeated, defined attack trials. Pass if ≥95% of attacks are correctly detected and flagged.
- False-positive test: Run repeated normal-workload trials. Pass if ≤5% of normal trials are incorrectly identified as attacks.
- Current-sensing range test: Verify that the current-sensing system operates over the specified range of 500 mA to 20 A. Minimum-current detection will be verified within the specified accuracy, and a thermal imaging camera will be used under high-load conditions to verify safe operation.
- Signal-to-noise ratio (SNR) test: Compare acquired current waveforms to the established golden profile to evaluate signal quality relative to measurement noise. Pass if the measured SNR is ≥20 dB.
- F1 score test: Run repeated defined attack trials and normal-operation trials and calculate the F1 score from the resulting detection performance. Pass if the F1 score is ≥0.85.
- Long-time overcurrent shutdown test: Apply a sustained current surge of 1.05–1.3 pu and verify that system shutdown occurs within 20 s.
- Short-time overcurrent shutdown test: Apply a current surge of 1.3–2 pu and verify that system shutdown occurs within 5 s.

Chapter 11 – Administrative Content

11.1 Budget and Financing

NG PULSE is funded by Northrop Grumman with a maximum project budget of $1,000. The team will use existing UCF Senior Design and DRACO laboratory equipment where available to reduce costs. The project bill of materials will be updated as exact components are selected. Table 11.1 summarizes the estimated cost by category.

Table 11.1: Cost Breakdown

Budget Item	Example Contents	Estimated Cost	Status
Current sensing	Sensor/amplifier, shunt, passives	[TBD]	To be selected
Data conversion	12-bit ADC and support components	[TBD]	To be selected
Detection/support	Comparator, logic/support ICs	[TBD]	To be selected
Power	Regulators, protection, filtering	[TBD]	Investigating
Prototype materials	Breadboard, wiring, connectors, headers	[TBD]	Planned
PCB/fabrication	Prototype PCB and assembly as required	[TBD]	Future
Contingency	Replacement parts/shipping/unplanned needs	[TBD]	Reserved
TOTAL	Must remain within sponsor funding	≤ $1,000	Requirement

11.2 Bill of Materials

Table 11.2: BOM

Item	Component / board	Manufacturer / supplier	Part number / description	Qty	Status / notes
1	FPGA	Lattice Semiconductor	iCE40HX4K-TQ144 (speed/temperature grade TBD)	1	144-pin TQFP
2	Microcontroller (MCU)	Nordic Semiconductor	nRF52832; package variant TBD	1	Nordic MCU
3	Current-sense amplifier	Texas Instruments	INA180A4IDBVR	1	200 V/V; SOT-23-5

4	3.3V buck regulator	Texas Instruments	TLV62569ADRLR	1	Adjustable; 3.3V design
5	1.2V buck regulator	Texas Instruments	TPS62160DGKT	1	Adjustable; 1.2V design
6	Current-limiting switch	Texas Instruments	TPS2553DBVR-1	1	Protected power branch
7	Target development board	Seeed Studio	XIAO ESP32C6; presoldered variant	1	USB-C target;
8	Main processing PCB	JLCPCB	Custom PCB: FPGA + Nordic MCU + clock	1 design	Prototype PCB; layers/dimensions TBD
9	Current capture PCB	JLCPCB	Custom PCB: USB-C, shunt, amplifier, limiter	1 design	Prototype PCB; layers/dimensions TBD
10	3.3V regulator PCB	JLCPCB	Custom PCB: TLV62569A circuit	1 design	Prototype PCB; layers/dimensions TBD
11	1.2V regulator PCB	JLCPCB	Custom PCB: TPS62160 circuit	1 design	Prototype PCB; layers/dimensions TBD

PCB quantity clarification: Four distinct PCB designs are planned. JLCPCB fabrication order quantities may exceed one board per design (for example, five pieces per design), depending on the selected fabrication service and quote.

Pricing: Unit costs and extended totals are intentionally omitted until exact orderable component variants and JLCPCB Gerber-based quotations are available.

Scope: This is a major-component BOM, not a complete manufacturing BOM. Passive components, clock oscillator, USB-C connectors, pin headers, decoupling capacitors, shunt resistor, and other supporting components are not included.

11.3 Project Milestones

11.3.1 Senior Design I

SD1 will focus on establishing the technical foundation needed to begin prototyping immediately in SD2. By the end of SD1, the team intends to have all major hardware selected, the golden profile obtained, a schematic of the proposed ASIC/system completed, and at least 80% of the Verilog code complete. The SD1 milestones are listed in Table 11.3.

Table 11.3: SD1 Milestones

SD1 Milestone	Target	Completion Evidence
System architecture and requirements	10/20/2026	Approved architecture/specifications
Hardware component selection	11/6/2026	Selected Parts/BOM
Hardware schematic V1	11/13/2026	Fusion360 screenshots
Verilog code major functions determined (not written)	11/13/2026	Verilog code files
Hardware schematic V2	11/20/2026	Fusion360 screenshots
High-risk hardware testing complete	11/20/2026	Test results logged, working hardware obtained if possible
Verilog development ≥80% complete	11/27/2026	Code files
PCB(s) laid out in Fusion360, ready to order	11/27/2026	Fusion360 screenshots

11.3.2 Senior Design II

Physical prototyping will begin immediately at the start of SD2. Initial work will focus on constructing the breadboard detector, integrating the FPGA and RISC-V target, and establishing basic end-to-end operation. The remaining approximately two months of SD2 will be devoted primarily to troubleshooting, refinement, final specification testing, data collection, and preparation for the final demonstration and presentation. The SD2 milestones are listed in Table 11.4.

Table 11.4: SD2 Milestones

SD2 Milestone	Target	Completion Evidence
Complete soldering for PCB(s)	1/16/2027	Complete PCB ready for testing
Complete Verilog code(s) ready to upload	1/16/2027	Code files
Test current capture accuracy	1/23/2027	Lab multimeter
Use RTL to design ASIC	1/30/2027	Cadence cloud files
Acquire golden profile and begin full system testing	1/30/2027	MCU Memory, system test results
Troubleshooting period	1/30 - 4/30/2027	Recorded pass/fail results for key specifications

11.4 Team Organization and Work Distribution

Table 11.5 lists each member's primary and secondary responsibilities.

Table 11.5: Work Distribution

Team Member	Primary Responsibility	Secondary Responsibility
Joey Esposito	ADC and supporting circuitry selection, comparator/protection circuitry, power distribution requirements and hardware component/BOM selection.	Hardware schematic development, FPGA/processor hardware interface, prototype assembly, electrical verification and specification testing
Dylan Ruiz	Current-sense design, current-sensor/amplifier selection, shunt/passive component selection, analog signal conditioning and power/voltage regulator design	Hardware schematic development, analog/ADC interface verification, breadboard prototype construction, hardware testing and construction
Sean Kenney	RISC-V/workload simulation, golden model and golden profile generation, windowed feature extraction RTL and Verilog module development	RTL testbenches, simulation/data analysis, ADC to digital logic integration, FPGA implementation and debugging
Jacob Daleandro	Threshold/classifier engine, attack detection and attack flag logic, PMBus/I²C/status interface logic and Verilog module development	Golden-profile comparison logic, RTL verification, FPGA/RISC-V integration, digital-system testing and debugging

Both Dylan and Joey are electrical engineering majors and as such will select all hardware for this project together, including, but not limited to the current capture circuit, ADC, and voltage regulator. They also will be responsible for producing the schematic and verifying all power and connection requirements. Sean and Jacob are both computer engineering majors so they will be creating a virtual model of the RISC-V processor and obtain the golden profile from their simulation. They will write the verilog code to program the FPGA and Nordic board as well.

Chapter 12 – Conclusion

NG PULSE is a hardware-security monitor that checks the integrity of a processor's workload by watching its supply-rail current. A fixed golden profile defines what normal operation looks like for each operating mode, and a sustained departure from it raises an error flag. The monitor sits outside the target's software and control path, so a compromised target cannot easily hide from it. This chapter summarizes where the project stands at the midpoint of Senior Design I, measured against the goal tiers of the Divide and Conquer (D&C), and what remains before the SD1 final report and the start of SD2.

12.1 Work Completed at the Midterm

In the first half of SD1 the team turned the D&C concept into a defined architecture with selected major parts and a first version of the detection logic.

• Architecture. The prototype is two boards. A shunt-based current-sense board sits in series with the ESP32-C6 target's USB-C supply. An FPGA/MCU board carries a Lattice iCE40HX4K-TQ144 FPGA and a Raytac MDBT42Q (nRF52832) module. The FPGA does the fast, stateless detection. The nRF52832 digitizes the current, loads and protects the golden profiles, holds the target in reset until the monitor is ready, and handles the error flag. At Northrop Grumman's request the same detection RTL is also being carried through the DRACO Laboratory's Cadence flow as a simulated ASIC.

• Research and part selection (Chapter 3). Current-sensing technologies, the microcontroller, the FPGA, the MCU firmware platform, the power supply, the clock source, the PCB tools and the ASIC tools have each been compared and selected. Several selections were driven by practical constraints rather than raw performance: the requirement for hand-solderable packages led to an LQFP-144 FPGA and a castellated BLE module, the need for a maintained toolchain ruled out the Spartan-6, and the decision to keep the monitor's silicon different from the target's ruled out a second ESP32.

• Standards and constraints (Chapter 4) relevant to the design have been identified and their effect on it described.

• Hardware design (Chapter 6). The current-sense board and the power distribution are designed.

• Software and RTL (Chapter 7). The nRF52832 firmware is designed down to its boot sequence, sampling chain (TIMER → PPI → SAADC → EasyDMA), SPI sample stream, I²C register map, policy state machine and pin assignment. The detection RTL (windowed mean, golden-profile store and comparison logic) is written. On 11/6/2026 it was revised to use the team's agreed mode_changing signal in place of mode_valid. The revised RTL passes all five testbench checks, and a new latency testbench shows alerts within 6 to 18 samples of an anomaly, far inside the 1 s detection-latency specification. The ASIC flow and its constraints are planned.

• Risk, fabrication and administration (Chapters 8, 9 and 11). The three highest risks are analysed with mitigations, the construction approach for the prototype is described, and the budget framework and milestone schedule are in place.

12.2 Status by Goal Tier

Basic goal. Monitoring one work mode plus the Not-in-Use state, raising an error flag, keeping the monitor a separate system, demonstrating a defined voltage-surge attack, and the Cadence digital design of the comparison logic and flag. Everything the basic goal needs in the FPGA logic is coded and simulated, and the firmware that supports it is designed. What remains is hardware: the boards must be built, and the golden values for Not-in-Use and one work mode

must be measured on them. The attack scenario still has to be defined in detail, and the Cadence synthesis has to be run.

Advanced goal. Four work modes plus Not-in-Use, automatic golden-profile selection from the reported mode, and continuous monitoring across mode transitions. The RTL already selects its golden value from op_mode and blanks comparisons during and just after a mode change. Two changes are still needed: op_mode must grow from 2 to 3 bits to carry five states, and the four work modes must be shown to be distinguishable above noise and drift once they are characterized.

Stretch goals. A single custom PCB, Bluetooth status reporting, automatic ESP32-C6 shutdown from the error flag, and ADC, memory storage and current capture in the ASIC. The firmware design already covers the BLE service and the reset-and-power-kill escalation, and both are labelled as stretch wherever they appear. They will be implemented only after the basic and advanced goals work. The ASIC stretch items are mixed-signal or memory blocks outside the current digital flow and have not been started.

12.3 Open Items Before the End of SD1

The most important items to close before the SD1 final report are:

• reconciling the current-sense design with the specification. The shunt value and amplifier gain in Chapter 6 must be checked against the 500 mA to 20 A range and the 10 mΩ and 50 mΩ analysis in Chapter 4, then finalized together with the amplifier bandwidth;

• completing the FPGA/MCU board design (Section 6.2) and verifying every FPGA and module pin against the datasheets, including moving I²C off the radio-adjacent nRF52832 pins for Rev 2;

• completing the bill of materials and costs (Chapter 11);

• fixing the mode interface across all chapters: GPIO lines op_mode plus mode_changing, a 3-bit op_mode for the five states, and code 000 as Not-in-Use;

• adding the sample-valid strobe, input synchronizers and the other RTL changes listed in Section 7.1.8, using distinct golden values so that mode selection can be tested and the ASIC results are meaningful;

• confirming the ASIC process design kit and DRACO tool access, and running a first Genus synthesis;

• choosing the three specifications that will be demonstrated, and defining the attack scenario for the basic-goal demonstration.

12.4 Risks and Path to Senior Design II

The largest technical risk remains the golden profile itself (Risk R1, Chapter 8). The approach only works if each operating mode's average current is stable enough, and different enough from the others, to separate normal operation from

an attack. A secondary risk is the power path and sensing (R3). The fixed blanking window after a mode change is also a known, bounded blind spot, and the team will decide whether to keep, shorten or adapt it.

The SD1 target is unchanged: all major hardware selected, the PCB laid out in Fusion 360 and ready to order, and at least 80% of the Verilog complete by 11/27/2026. SD2 begins with board assembly and bring-up in January 2027. It continues with the capture-accuracy test, the golden-profile characterization and full system test, and the RTL-to-ASIC run, followed by an extended test period against the detection-rate, false-positive, F1 and latency specifications. The overcurrent shut-off specifications depend on the stretch-goal shutdown path and on the current limiter, so they will be evaluated only if that path is built. At the midterm, the design direction is settled and the detection logic has been shown to work in simulation. The remaining work is to build the hardware and to measure the golden profiles on it.

Appendix A – References

Course guideline: EEL 4914 Senior Design I, Divide and Conquer Guideline. Additional technical references and product datasheets must be added as the prior-work review and component selection are completed.

[1] Texas Instruments “SBOS790 Data sheet | TI.com.” https://www.ti.com/document-viewer/INA233/datasheet

[2] Analog Devices, Inc., “LTC2945: Wide Range I²C Power Monitor,” Rev. D. https://www.analog.com/media/en/technical-documentation/data-sheets/ltc2945.pdf

[3] Analog Devices, Inc., “ADM1293/ADM1294: Digital Power Monitor with PMBus Interface,” Rev. B, 2015. https://www.analog.com/media/en/technical-documentation/data-sheets/ADM1293_1294.pdf

[4] Nordic Semiconductor, "nRF52832 Product Specification," v1.9, Dec. 2023. https://docs.nordicsemi.com/bundle/ps_nrf52832/page/keyfeatures_html5.html

[5] Nordic Semiconductor, "nRF52840 Product Specification," v1.11, Oct. 2024. https://docs.nordicsemi.com/bundle/ps_nrf52840/page/keyfeatures_html5.html

[6] Espressif Systems, "ESP32-C6 Series Datasheet," v1.5, Mar. 2026. https://www.espressif.com/sites/default/files/documentation/esp32-c6_datasheet_en.pdf

[7] STMicroelectronics, "STM32G431x6/x8/xB Datasheet," DS12589 Rev 6, Oct. 2021. https://www.st.com/resource/en/datasheet/stm32g431rb.pdf

[8] Raspberry Pi Ltd., "RP2040 Datasheet," build 2025-02-20. https://datasheets.raspberrypi.com/rp2040/rp2040-datasheet.pdf

[9] Raytac Corporation, "MDBT42Q & MDBT42Q-P Module Specification," Version P, Jul. 2025.

https://www.raytac.com/upload/download_files/38a8a4a0aff945d8484507d60058109b.pdf

[10] Lattice Semiconductor, "iCE40 LP/HX Family Data Sheet," DS1040.

[11] AMD/Xilinx, "Spartan-6 Family Overview," DS160.

[12] AMD/Xilinx, "7 Series FPGAs Data Sheet: Overview," DS180.

[13] Nordic Semiconductor, "nRF Connect SDK Documentation." https://docs.nordicsemi.com/bundle/ncs-latest/page/nrf/index.html

[14] Reddit r/FPGA, "Where to program Spartan 6 FPGA," https://www.reddit.com/r/FPGA/comments/1gh5rk8/where_to_program_spartan_6_fpga/

[15] Winbond, "W25Q32JV Datasheet."

[16] IPC, IPC-2221C, Generic Standard on Printed Board Design https://www.nextpcb.com/blog/ipc-2221

[17] IPC, IPC-2152, Standard for Determining Current Carrying Capacity in Printed Board Design, 2009. https://www.electronics.org/TOC/IPC-2152.pdf

[18] Texas Instruments, How to Lay Out a PCB for High-Performance, Low-Side Current-Sensing Designs, SSZT805, February 2018. https://www.ti.com/document-viewer/lit/html/SSZT805#GUID-5F278960-BA94-4362-9099-87ABC753A4D6

[19] NXP Semiconductors, I²C-Bus Specification and User Manual, UM10204, revision 7.0, October 1, 2021, Sections 3, 4.3, 6, and 7. https://www.nxp.com/docs/en/user-guide/UM10204.pdf

[20] Texas Instruments, I²C Bus Pullup Resistor Calculation, SLVA689, February 2015. https://www.ti.com/lit/an/slva689a/slva689a.pdf?ts=1793891346140

[21] USB Implementers Forum, USB Type-C Cable and Connector Specification, https://www.usb.org/document-library/usb-type-cr-cable-and-connector-specification-release-25

[22] N. Enos and B. Gosselin, A Primer on USB Type-C and USB Power Delivery Applications and Requirements, Texas Instruments, SLYY109A, March 2022. https://www.ti.com/lit/wp/slyy109b/slyy109b.pdf?ts=1793908995397&ref_url=https%253A%252F%252Fchatgpt.com%252F

[23] IEEE, IEEE 1800-2023, Standard for SystemVerilog Unified Hardware Design, Specification, and Verification Language, published February 28, 2024. https://standards.ieee.org/ieee/1800/7743/

[24] IPC, IPC J-STD-001J, Requirements for Soldered Electrical and Electronic Assemblies, March 2024. https://www.electronics.org/TOC/IPC-J-STD-001J_TOC.pdf

[25] IPC, IPC-A-610J, Acceptability of Electronic Assemblies, March 2024. https://www.electronics.org/TOC/IPC-A-610J_TOC.pdf

[26] EOS/ESD Association, ANSI/ESD S20.20-2021, Standard for the Development of an Electrostatic Discharge Control Program. https://www.esda.org/store/search/product/314/ansiesd-s20-20-2021?l=en_US

[27] IEC, IEC 61010-1:2010+AMD1:2016, Safety Requirements for Electrical Equipment for Measurement, Control, and Laboratory Use, Part 1. https://cdn.standards.iteh.ai/samples/22220/a6c5cee6104548a68e3719a79066c2c4/IEC-61010-1-2010-AMD1-2016.pdf

[28] IEC, IEC 61326-1:2020, Electrical Equipment for Measurement, Control and Laboratory Use, EMC Requirements, Part 1. https://cdn.standards.iteh.ai/samples/101977/104f40be88de493d8f12852106314196/IEC-61326-1-2020.pdf

[29] Nordic Semiconductor, nRF52832 Product Specification, SAADC chapter, including reference selection, input range, and acquisition time. https://docs.nordicsemi.com/r/bundle/ps_nrf52832/page/saadc.html

[30] Nordic Semiconductor, nRF52832 Product Specification, SPIM chapter, including EasyDMA and TXD.MAXCNT. https://docs.nordicsemi.com/r/bundle/ps_nrf52832/page/spim.html

[31] Lattice Semiconductor, iCE40 sysCLOCK PLL Design and User Guide, FPGA-TN-02052-1.4, April 2022, Section 3.5. https://0x04.net/~mwk/sbdocs/ice40/FPGA-TN-02052-1-4-iCE40-sysCLOCK-PLL-Design-User-Guide.pdf

[32] European Commission, Restriction of Hazardous Substances in Electrical and Electronic Equipment, official RoHS overview. https://environment.ec.europa.eu/topics/waste-and-recycling/rohs-directive_en

Appendix B – Copyright Permissions

[TO DO: Written authorization (emails, letters or explicit permission citations) for every copyrighted figure, table or text used. Remove any material whose request was rejected.]

Appendix C – Data Sheets

[TO DO: If necessary. Datasheet material in total is limited to two pages.]

Appendix D – Software Code

This appendix holds every code listing referenced in the body of the report. The body describes the code but does not reproduce it.

D.1 Golden-profile record structure (nRF52832 firmware, Section 7.2.9)

```
struct golden_record { u16 version; u16 n_modes;
                       struct { u16 mean; u16 dev; u16 t_minor; u16
t_major; } mode[5];
                       u8 window_log2; u8 persist; u16 blank; u32 crc32;
};
```

D.2 Detection RTL and testbenches (Section 7.1)

The listings below are the RTL described in Section 7.1, revised on 11/6/2026 to replace the mode_valid input with mode_changing (mode_changing = ~mode_valid). D.2.1 to D.2.4 are the synthesizable modules shared by the FPGA and the ASIC flow (Section 7.4). D.2.5 is the top-level testbench (results in Table 7.4) and D.2.6 is the latency testbench added on 11/6/2026. All six files compile and pass in Icarus Verilog 12.0.

D.2.1 ng_pulse_top.sv

```
// ng_pulse_top.sv
// Top-level NG PULSE comparison logic.
// Instantiates windowed_mean, golden_store, and comparison_logic.


`default_nettype none


module ng_pulse_top #(
    parameter WIDTH         = 12,
    parameter N             = 16,
    parameter N_LOG2        = 4,
    parameter THRESHOLD     = 50,
    parameter PERSIST_COUNT = 4,
    parameter BLANK_CYCLES  = 256
)(
    input  wire                 clk,
    input  wire                 rst_n,
    input  wire [WIDTH-1:0]     adc_in,
    input  wire [1:0]           op_mode,
    input  wire                 mode_changing,  // 1 = ESP32-C6
switching modes: blank comparisons
    output wire                 alert,
    output wire [WIDTH-1:0]     deviation
);


    wire [WIDTH-1:0] live_mean;
    wire             mean_valid;
    wire [WIDTH-1:0] golden;


    windowed_mean #(
        .N      (N),
        .N_LOG2 (N_LOG2),
        .WIDTH  (WIDTH)
    ) u_windowed_mean (
```

```
        .clk        (clk),
        .rst_n      (rst_n),
        .sample_in  (adc_in),
        .mean_out   (live_mean),
        .mean_valid (mean_valid)
    );


    golden_store #(
        .WIDTH (WIDTH)
    ) u_golden_store (
        .op_mode    (op_mode),
        .golden_out (golden)
    );


    comparison_logic #(
        .WIDTH         (WIDTH),
        .THRESHOLD     (THRESHOLD),
        .PERSIST_COUNT (PERSIST_COUNT),
        .BLANK_CYCLES  (BLANK_CYCLES)
    ) u_comparison_logic (
        .clk        (clk),
        .rst_n      (rst_n),
        .live_mean  (live_mean),
        .mean_valid (mean_valid),
        .golden     (golden),
        .op_mode    (op_mode),
        .mode_changing (mode_changing),
        .alert      (alert),
        .deviation  (deviation)
    );

endmodule


`default_nettype wire
```

D.2.2 windowed_mean.sv

```
// windowed_mean.sv
// Computes a running mean over the last N samples.
// N must be a power of 2 (divide implemented as right shift by N_LOG2).


`default_nettype none


module windowed_mean #(
    parameter N      = 16,
    parameter N_LOG2 = 4,
    parameter WIDTH  = 12
)(
    input  wire                 clk,
    input  wire                 rst_n,
    input  wire [WIDTH-1:0]     sample_in,
    output reg  [WIDTH-1:0]     mean_out,
    output reg                  mean_valid
```

```
);

    // Shift register holding last N samples
    reg [WIDTH-1:0] window [0:N-1];

    // Running sum wide enough for N * (2^WIDTH - 1)
    reg [WIDTH+N_LOG2-1:0] running_sum;

    // Fill counter
    reg [4:0] fill_count;   // sized for N up to 16

    integer i;

    always @(posedge clk or negedge rst_n) begin
        if (!rst_n) begin
            running_sum <= 0;
            fill_count  <= 0;
            mean_out    <= 0;
            mean_valid  <= 0;
            for (i = 0; i < N; i = i + 1)
                window[i] <= 0;
        end else begin
            // Update running sum: subtract oldest, add newest
            running_sum <= running_sum - window[N-1] + sample_in;

            // Shift window — index 0 is newest
            for (i = N-1; i > 0; i = i - 1)
                window[i] <= window[i-1];
            window[0] <= sample_in;

            // Track fill
            if (fill_count < N)
                fill_count <= fill_count + 1;

            mean_valid <= (fill_count >= N - 1);

            // Divide by N via right shift
            mean_out <= (running_sum - window[N-1] + sample_in) >>
N_LOG2;
        end
    end

endmodule

`default_nettype wire
```

D.2.3 golden_store.sv

```
// golden_store.sv
```

```
// Hardcoded ROM for golden power profiles per op mode.
// PLACEHOLDER VALUES — replace with real measured means after
// characterization campaign on the physical ESP32-C6 board.


`default_nettype none


module golden_store #(
    parameter WIDTH = 12
)(
    input  wire [1:0]       op_mode,
    output reg  [WIDTH-1:0] golden_out
);


    localparam [WIDTH-1:0] GOLDEN_IDLE    = 12'h800;
    localparam [WIDTH-1:0] GOLDEN_COMPUTE = 12'h800;
    localparam [WIDTH-1:0] GOLDEN_CRYPTO  = 12'h800;
    localparam [WIDTH-1:0] GOLDEN_RADIO   = 12'h800;


    always @(*) begin
        case (op_mode)
            2'b00:   golden_out = GOLDEN_IDLE;
            2'b01:   golden_out = GOLDEN_COMPUTE;
            2'b10:   golden_out = GOLDEN_CRYPTO;
            2'b11:   golden_out = GOLDEN_RADIO;
            default: golden_out = GOLDEN_IDLE;
        endcase
    end


endmodule


`default_nettype wire
```

D.2.4 comparison_logic.sv

```
// comparison_logic.sv
// Compares live windowed mean against the golden profile.
// Alert fires only after PERSIST_COUNT consecutive anomalous comparisons
// (one comparison per clock once the window is full).
// Blanking suppresses alerts while mode_changing is high and for
// BLANK_CYCLES clocks after every op_mode change.


`default_nettype none


module comparison_logic #(
    parameter WIDTH         = 12,
    parameter THRESHOLD     = 50,
    parameter PERSIST_COUNT = 4,
    parameter BLANK_CYCLES  = 256
)(
    input  wire                 clk,
    input  wire                 rst_n,
```

```
    input  wire [WIDTH-1:0]      live_mean,
    input  wire                  mean_valid,
    input  wire [WIDTH-1:0]      golden,
    input  wire [1:0]            op_mode,
    input  wire                  mode_changing,
    output reg                   alert,
    output reg  [WIDTH-1:0]      deviation
);


    reg [8:0]  blank_count;
    reg [1:0]  op_mode_prev;
    reg [2:0]  alarm_count;
    reg [WIDTH-1:0] abs_diff;


    // Blank while the post-change counter runs or while the target
reports
    // that it is changing modes (mode_changing replaces the old
mode_valid
    // strobe; mode_changing = ~mode_valid).
    wire blanking = (blank_count > 0) || mode_changing;


    always @(posedge clk or negedge rst_n) begin
        if (!rst_n) begin
            blank_count  <= 0;
            op_mode_prev <= 0;
            alarm_count  <= 0;
            alert        <= 0;
            deviation    <= 0;
        end else begin
            op_mode_prev <= op_mode;


            // Blanking counter on mode change
            if (op_mode != op_mode_prev)
                blank_count <= BLANK_CYCLES;
            else if (blank_count > 0)
                blank_count <= blank_count - 1;


            // Only run comparison when window is filled and not blanking
            if (mean_valid && !blanking) begin
                // Compute absolute difference
                if (live_mean >= golden)
                    abs_diff = live_mean - golden;
                else
                    abs_diff = golden - live_mean;


                deviation <= abs_diff;


                // Increment or clear persistence counter
                if (abs_diff > THRESHOLD) begin
                    if (alarm_count < PERSIST_COUNT)
                        alarm_count <= alarm_count + 1;
                end else begin
```

```
                    alarm_count <= 0;
                end


                // Alert when persistence threshold reached
                alert <= (alarm_count >= PERSIST_COUNT - 1) && (abs_diff
> THRESHOLD);


            end else if (blanking) begin
                // Clear everything during blanking — do NOT clear during
!mean_valid
                alarm_count <= 0;
                deviation   <= 0;
                alert       <= 0;
            end
            // When !mean_valid and not blanking: hold state, let window
fill
        end
    end


endmodule


`default_nettype wire
```

D.2.5 tb_ng_pulse_top.sv

```
// tb_ng_pulse_top.sv
// Testbench for ng_pulse_top.
// Scenarios:
//   1. Clean rail  — no alert expected
//   2. Mode transition blanking — no false alert during switch
//   3. Injected anomaly — alert must fire
//   4. Recovery — alert clears when rail returns to normal


`timescale 1ns/1ps


module tb_ng_pulse_top;


    parameter WIDTH         = 12;
    parameter N             = 16;
    parameter N_LOG2        = 4;
    parameter THRESHOLD     = 50;
    parameter PERSIST_COUNT = 4;
    parameter BLANK_CYCLES  = 256;


    localparam [WIDTH-1:0] GOLDEN_VAL = 12'h800;


    reg                 clk;
    reg                 rst_n;
    reg  [WIDTH-1:0]    adc_in;
    reg  [1:0]          op_mode;
    reg                 mode_changing;
```

```
wire                alert;
wire [WIDTH-1:0]    deviation;


integer i;
integer noise;


ng_pulse_top #(
    .WIDTH         (WIDTH),
    .N             (N),
    .N_LOG2        (N_LOG2),
    .THRESHOLD     (THRESHOLD),
    .PERSIST_COUNT (PERSIST_COUNT),
    .BLANK_CYCLES  (BLANK_CYCLES)
) dut (
    .clk        (clk),
    .rst_n      (rst_n),
    .adc_in     (adc_in),
    .op_mode    (op_mode),
    .mode_changing (mode_changing),
    .alert      (alert),
    .deviation  (deviation)
);


// 100MHz clock
initial clk = 0;
always #5 clk = ~clk;


// Timeout watchdog
initial begin
    #10_000_000;
    $display("TIMEOUT");
    $finish;
end


initial begin
    $dumpfile("ng_pulse_top.vcd");
    $dumpvars(0, tb_ng_pulse_top);


    // Reset
    rst_n      = 0;
    adc_in     = 0;
    op_mode    = 2'b01;
    mode_changing = 1;
    repeat(4) @(posedge clk);
    #1;
    rst_n = 1;
    @(posedge clk);
    #1;


    $display("[%0t] Reset complete", $time);
```

```
// ------------------------------------------------
// SCENARIO 1: Clean rail — no alert expected
// ------------------------------------------------
$display("[%0t] S1: Clean rail", $time);
mode_changing = 0;


for (i = 0; i < N + 32; i = i + 1) begin
    noise  = ($random % 11);         // -5 to +5
    adc_in = GOLDEN_VAL + noise;
    @(posedge clk);
    #1;
end


if (alert)
    $display("FAIL S1: alert on clean rail");
else
    $display("PASS S1: no alert on clean rail");


// ------------------------------------------------
// SCENARIO 2: Mode transition blanking
// ------------------------------------------------
$display("[%0t] S2: Mode transition", $time);
mode_changing = 1;
op_mode    = 2'b10;


for (i = 0; i < 64; i = i + 1) begin
    adc_in = 12'h500;   // anomalous during transition
    @(posedge clk);
    #1;
end


if (alert)
    $display("FAIL S2: alert during blanking");
else
    $display("PASS S2: blanking suppressed alert");


// Settle into new mode — must outlast the BLANK_CYCLES window
// started at the op_mode change, or S3 lands inside blanking
mode_changing = 0;
for (i = 0; i < BLANK_CYCLES + N + 32; i = i + 1) begin
    noise  = ($random % 11);
    adc_in = GOLDEN_VAL + noise;
    @(posedge clk);
    #1;
end


if (alert)
    $display("FAIL S2b: alert after settle");
else
    $display("PASS S2b: clean after mode settle");
```

```
        // ------------------------------------------------
        // SCENARIO 3: Injected anomaly — alert must fire
        // Need enough samples to:
        //   - flush the old clean mean out of the window (N samples)
        //   - accumulate PERSIST_COUNT full anomalous windows
(N*PERSIST_COUNT)
        //   - plus a few extra cycles for the alert register to update
        // ------------------------------------------------
        $display("[%0t] S3: Injected anomaly", $time);


        for (i = 0; i < N * (PERSIST_COUNT + 4) + 10; i = i + 1) begin
            adc_in = 12'h500;   // well below golden (deviation = 0x300 =
768 >> THRESHOLD)
            @(posedge clk);
            #1;
        end


        if (!alert)
            $display("FAIL S3: alert did not fire");
        else
            $display("PASS S3: alert fired (deviation=%0d)", deviation);


        // ------------------------------------------------
        // SCENARIO 4: Recovery — alert must clear
        // ------------------------------------------------
        $display("[%0t] S4: Recovery", $time);


        for (i = 0; i < N * (PERSIST_COUNT + 4) + 10; i = i + 1) begin
            noise  = ($random % 11);
            adc_in = GOLDEN_VAL + noise;
            @(posedge clk);
            #1;
        end


        if (alert)
            $display("FAIL S4: alert still set after recovery");
        else
            $display("PASS S4: alert cleared");


        $display("[%0t] Simulation complete", $time);
        $finish;
    end


endmodule
```

D.2.6 tb_latency.sv

```
// tb_latency.sv
// Directed latency measurements for ng_pulse_top (added 2026-11-06).
// Measures clocks from an anomaly step to alert, the threshold boundary,
// recovery time, and blanking when op_mode changes without
mode_changing.
```

```
`timescale 1ns/1ps

module tb_latency;

    reg         clk = 0;
    reg         rst_n;
    reg         mode_changing;
    reg  [11:0] adc_in;
    reg  [1:0]  op_mode;
    wire        alert;
    wire [11:0] deviation;
    integer     i, lat;

    ng_pulse_top dut (
        .clk(clk), .rst_n(rst_n), .adc_in(adc_in), .op_mode(op_mode),
        .mode_changing(mode_changing), .alert(alert),
.deviation(deviation)
    );

    always #5 clk = ~clk;

    // Apply a constant level and count clocks until alert (-1 = never)
    task step_to(input [11:0] level, output integer clocks);
        begin
            adc_in = level;
            clocks = -1;
            for (i = 0; i < 200 && clocks < 0; i = i + 1) begin
                @(posedge clk); #1;
                if (alert) clocks = i + 1;
            end
        end
    endtask

    task settle;
        begin
     adc_in = 12'h800;
            repeat (300) @(posedge clk);
            #1;
        end
    endtask

    initial begin
        rst_n = 0; op_mode = 2'b01; mode_changing = 1; adc_in = 12'h800;
        repeat (4) @(posedge clk);
        #1 rst_n = 1; mode_changing = 0;
        settle;

        step_to(12'h800 - 12'd768, lat); $display("step -768 LSB : alert
after %0d clocks", lat); settle;
```

```
        step_to(12'h800 - 12'd100, lat); $display("step -100 LSB : alert
after %0d clocks", lat); settle;
        step_to(12'h800 + 12'd60,  lat); $display("step  +60 LSB : alert
after %0d clocks", lat); settle;
        step_to(12'h800 + 12'd50,  lat); $display("step  +50 LSB : alert
after %0d clocks (-1 = never; THRESHOLD is strict)", lat); settle;


        // Recovery: hold an anomaly, then return to the golden level
        adc_in = 12'h500;
        repeat (100) @(posedge clk);
        #1 adc_in = 12'h800; lat = -1;
        for (i = 0; i < 100 && lat < 0; i = i + 1) begin
            @(posedge clk); #1;
            if (!alert) lat = i + 1;
        end
        $display("recovery      : alert cleared after %0d clocks", lat);


        // op_mode change with mode_changing held low: only BLANK_CYCLES
protects
        settle;
        op_mode = 2'b10; adc_in = 12'h500; lat = -1;
        for (i = 0; i < 400 && lat < 0; i = i + 1) begin
            @(posedge clk); #1;
            if (alert) lat = i + 1;
        end
        $display("mode change, no mode_changing, immediate anomaly: alert
after %0d clocks", lat);
        $finish;
    end


endmodule
```

Appendix E – ChatGPT Prompts and Outcomes

#	Section	Prompt (paste the exact text)	Outcome
E.1	5.4.1	[Prompt asking for an LQFP alternative to the Artix-7, 10/8/2026]	Spartan-6 proposed, later rejected; iCE40HX4K-TQ144 selected
E.2	5.4.2	[Prompt or response describing the HX8K-TQ144 upgrade path, 10/8/2026]	Part does not exist; caught by Jacob
E.3	5.4.3	[Prompts with the S3 simulation failure, 10/22/2026]	Two wrong hypotheses; root cause in the testbench
E.4	5.4.4	[Prompt about recalculating the golden profile, 10/21/2026]	Immutable golden adopted
E.5	5.4.4	[Prompt asking for a review of the current-sense schematic, 10/12/2026]	Gain discrepancy found

Appendix F – Supporting Material

[TO DO: Any other supporting material.]